SPiiPlus Python Library Reference

Programmer's Guide

May 2024

Document Revision: 3.14.01

ACS MOTION CONTROL

SPiiPlus Python Library Reference

Release Date: May 2024

COPYRIGHT

© ACS Motion Control Ltd., 2024. All rights reserved.

Changes are periodically made to the information in this document. Changes are published as release notes and later incorporated into revisions of this document.

No part of this document may be reproduced in any form without prior written permission from ACS Motion Control.

TRADEMARKS

Windows and Intellisense are trademarks of Microsoft Corporation.

EtherCAT® is registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany.

Any other companies and product names mentioned herein may be the trademarks of their respective owners.

PATENTS

Israel Patent No. 235022
US Patent Application No. 14/532,023
Europe Patent application No.15187586.1
Japan Patent Application No.: 2015-193179
Chinese Patent Application No.: 201510639732.X
Taiwan(R.O.C.) Patent Application No. 104132118
Korean Patent Application No. 10-2015-0137612

www.acsmotioncontrol.com

support@acsmotioncontrol.com

sales@acsmotioncontrol.com

NOTICE

The information in this document is deemed to be correct at the time of publishing. ACS Motion Control reserves the right to change specifications without notice. ACS Motion Control is not responsible for incidental, consequential, or special damages of any kind in connection with using this document.

Revision History

Date	Revision	Description
May 2024	3.14.01	Clarify Linux support Add FRF functions
March 2024	3.14a	NURBS parameter warning
February 2024	3.14	Initial Version

Related Documentation

Documents listed in the following table provide additional information related to this document.

Authorized users can download the latest versions of the documents from ACS Downloads.

Document	Description
SPiiPlus Setup Guide	Communication, configuration and adjustment procedures for SPiiPlus motion control products.
SPiiPlus Command & Variable Reference Guide	Command and variables of high level language for programming SPiiPlus controllers.
SPiiPlus C Library Reference Guide	C Methods, Properties, and Events for Communication with the Controller
SPiiPlus Utilities User Guide	The SPiiPlus Utilities described in this document are: > SPiiPlus User Mode Driver (UMD) > SPiiPlus Upgrader > SPiiPlus Simulator

Conventions Used in this Guide

Text Formats

Format	Description
Bold	Names of GUI objects or commands
BOLD + UPPERCASE	ACSPL+ variables and commands
`Monospace + grey background`	Code example
Italic	Names of other documents
Blue	Hyperlink
[ ]	In commands indicates optional item(s)
|	In commands indicates either/or items

Flagged Text

	**Note** - includes additional information or programming tips.
	**Caution** - describes a condition that may result in damage to equipment.
	**Warning** - describes a condition that may result in serious bodily injury or death.
	**Model** - highlights a specification, procedure, condition, or statement that depends on the product model
	**Advanced** - indicates a topic for advanced users.

Table of Contents

1. Introduction 19
2. Operational Environment and Other Requirements 20
3. Package Installation and Removal 21
   3.1 Package Installation 21
   3.2 Package Removal 23
   3.3 Supplied Components 23
4. SPiiPlus C Library Compatibility 25
   4.1 Unsupported C Library Functions 25
   4.2 Function Naming Convention 25
5. Python Library Functions 26
   5.1 Communications Functions 26
      5.1.1 OpenCommSerial 26
      5.1.2 OpenCommEthernetTCP 26
      5.1.3 OpenCommEthernetUDP 27
      5.1.4 OpenCommSimulator 28
      5.1.5 CloseSimulator 28
      5.1.6 GetEthernetCardsExt 29
      5.1.7 SetServerExtLogin 30
      5.1.8 CloseComm 31
      5.1.9 Transaction 32
      5.1.10 Command 33
      5.1.11 GetResult 34
      5.1.12 CancelOperation 35
      5.1.13 GetConnectionsList 36
      5.1.14 GetConnectionInfo 36
      5.1.15 TerminateConnection 37
   5.2 Service Communication Functions 37
      5.2.1 GetLastError 37
      5.2.2 GetErrorString 38
      5.2.3 SetQueueOverflowTimeout 39
      5.2.4 GetQueueOverflowTimeout 39
      5.2.5 SetTimeout 40

5.2.6 GetTimeout 40
5.2.7 GetDefaultTimeout 41
5.2.8 SetIterations 41
5.2.9 SetCommOptions 42
5.2.10 GetCommOptions 43
5.2.11 ClearBreakpoints 43
5.2.12 SetBreakpoint 44
5.2.13 GetBreakpointsList 44
5.2.14 GetLibraryVersion 45
5.3 ACSPL+ Program Management Functions 46
5.3.1 LoadBuffer 46
5.3.2 LoadBuffer 46
5.3.3 LoadBuffersFromFile 47
5.3.4 AppendBuffer 48
5.3.5 UploadBuffer 49
5.3.6 ClearBuffer 49
5.3.7 CompileBuffer 50
5.3.8 RunBuffer 51
5.3.9 StopBuffer 52
5.3.10 SuspendBuffer 53
5.4 Read and Write Variables Functions 54
5.4.1 ReadInteger 54
5.4.2 WriteInteger 55
5.4.3 ReadReal 57
5.4.4 WriteReal 57
5.4.5 ReadString 59
5.4.6 WriteString 60
5.4.7 ReadStruct 62
5.4.8 WriteStruct 62
5.5 Load/Upload Data To/From Controller Functions 63
5.5.1 LoadDataToController 63
5.5.2 UploadDataFromController 65
5.6 Multiple Thread Synchronization Functions 67
5.6.1 CaptureComm 67

5.6.2 ReleaseComm 68
5.7 History Buffer Management Functions 69
5.7.1 OpenHistoryBuffer 69
5.7.2 CloseHistoryBuffer 69
5.7.3 GetHistory 70
5.8 Unsolicited Messages Buffer Management Functions 71
5.8.1 OpenMessageBuffer 71
5.8.2 GetMessage 71
5.8.3 GetSingleMessage 72
5.9 Log File Management Methods 73
5.9.1 OpenLogFile 73
5.9.2 CloseLogFile 73
5.9.3 WriteLogFile 74
5.9.4 FlushLogFile 75
5.9.5 GetLogData 75
5.10 SPiiPlusSC Log File Management Functions 76
5.10.1 OpenSCLogFile 76
5.10.2 CloseSCLogFile 76
5.10.3 WriteSCLogFile 77
5.10.4 FlushSCLogFile 77
5.11 System Configuration Functions 78
5.11.1 SetConf 78
5.11.2 GetConf 79
5.11.3 GetVolatileMemoryUsage 80
5.11.4 GetVolatileMemoryTotal 80
5.11.5 GetVolatileMemoryFree 81
5.11.6 GetNonVolatileMemoryUsage 82
5.11.7 GetNonVolatileMemoryTotal 82
5.11.8 GetNonVolatileMemoryFree 83
5.11.9 SysInfo 84
5.12 Setting and Reading Motion Parameters Functions 84
5.12.1 SetVelocity 84
5.12.2 GetVelocity 85
5.12.3 SetAcceleration 86

5.12.4 GetAcceleration ... 87
5.12.5 SetDeceleration ... 88
5.12.6 GetDeceleration ... 89
5.12.7 SetJerk ... 90
5.12.8 GetJerk ... 91
5.12.9 SetKillDeceleration ... 92
5.12.10 GetKillDeceleration ... 93
5.12.11 SetVelocityImm ... 94
5.12.12 SetAccelerationImm ... 95
5.12.13 SetDecelerationImm ... 96
5.12.14 SetJerkImm ... 97
5.12.15 SetKillDecelerationImm ... 98
5.12.16 SetFPosition ... 99
5.12.17 GetFPosition ... 99
5.12.18 SetRPosition ... 100
5.12.19 GetRPosition ... 101
5.12.20 GetFVelocity ... 102
5.12.21 GetRVelocity ... 103
5.13 Axis/Motor Management Functions ... 104
5.13.1 CommutExt ... 104
5.13.2 Enable ... 105
5.13.3 EnableM ... 106
5.13.4 Disable ... 107
5.13.5 DisableAll ... 108
5.13.6 DisableExt ... 108
5.13.7 DisableM ... 109
5.13.8 Group ... 110
5.13.9 Split ... 111
5.13.10 SplitAll ... 112
5.14 Motion Management Functions ... 113
5.14.1 Go ... 113
5.14.2 GoM ... 114
5.14.3 Halt ... 115
5.14.4 HaltM ... 116

5.14.5 Kill 118
5.14.6 KillM 118
5.14.7 KillAll 119
5.14.8 KillExt 120
5.14.9 Break 121
5.14.10 BreakM 123
5.15 Point-to-Point Motion Functions 124
5.15.1 ToPoint 124
5.15.2 ToPointM 125
5.15.3 ExtToPoint 126
5.15.4 ExtToPointM 128
5.16 Augmented Point-to-Point Motion 129
5.16.1 BoostedPointToPointMotion 129
5.16.2 SmoothPointToPointMotion 132
5.17 Track Motion Control Functions 133
5.17.1 Track 133
5.17.2 SetTargetPosition 134
5.17.3 GetTargetPosition 135
5.18 Jog Functions 136
5.18.1 Jog 136
5.18.2 JogM 138
5.19 Slaved Motion Functions 139
5.19.1 SetMaster 139
5.19.2 Slave 140
5.19.3 SlaveStalled 142
5.20 Multi-Point Motion Functions 144
5.20.1 MultiPoint 144
5.20.2 MultiPointM 145
5.21 Arbitrary Path Motion Functions 147
5.21.1 Spline 147
5.21.2 SplineM 149
5.22 PVT Functions 150
5.22.1 AddPVPoint 150
5.22.2 AddPVPointM 151

5.22.3 AddPVTPoint 153
5.22.4 AddPVTPointM 154
5.23 Segmented Motion Functions 155
5.23.1 ExtendedSegmentedMotionV2 155
5.23.2 SegmentLineV2 164
5.23.3 SegmentArc1V2 168
5.23.4 SegmentArc2V2 172
5.23.5 Stopper 176
5.23.6 Projection 177
5.23.7 SmoothTransitionPointToPointMotion 178
5.24 Blended Segmented Motion Functions 180
5.24.1 BlendedSegmentMotion 180
5.24.2 BlendedLine 182
5.24.3 BlendedArc1 183
5.24.4 BlendedArc2 185
5.25 NURBS and Smooth Path Motion 187
5.25.1 NurbsMotion 187
5.25.2 NurbsPoints 190
5.25.3 SmoothPathMotion 192
5.25.4 SmoothPathSegment 195
5.26 Points and Segments Manipulation Functions 200
5.26.1 AddPoint 200
5.26.2 AddPointM 201
5.26.3 ExtAddPoint 202
5.26.4 ExtAddPointM 204
5.26.5 EndSequence 205
5.26.6 EndSequenceM 206
5.27 Data Collection Functions 207
5.27.1 DataCollectionExt 207
5.27.2 StopCollect 208
5.27.3 WaitCollectEndExt 209
5.28 Status Report Functions 210
5.28.1 GetMotorState 210
5.28.2 GetAxisState 211

5.28.3 GetIndexState 212
5.28.4 ResetIndexState 212
5.28.5 GetProgramState 214
5.29 Input/Output Access Functions 214
5.29.1 GetInput 214
5.29.2 GetInputPort 215
5.29.3 GetOutput 216
5.29.4 GetOutputPort 217
5.29.5 GetAnalogInputNT 217
5.29.6 GetAnalogOutputNT 218
5.29.7 SetAnalogOutputNT 219
5.29.8 GetExtInput 220
5.29.9 GetExtInputPort 221
5.29.10 GetExtOutput 222
5.29.11 GetExtOutputPort 223
5.29.12 SetExtOutput 223
5.29.13 SetExtOutputPort 224
5.30 Safety Control Functions 225
5.30.1 GetFault 225
5.30.2 SetFaultMask 226
5.30.3 GetFaultMask 227
5.30.4 EnableFault 228
5.30.5 DisableFault 229
5.30.6 SetResponseMask 230
5.30.7 GetResponseMask 231
5.30.8 DisableResponse 232
5.30.9 GetSafetyInput 233
5.30.10 GetSafetyInputPort 235
5.30.11 GetSafetyInputPortInv 236
5.30.12 SetSafetyInputPortInv 237
5.30.13 FaultClear 238
5.30.14 FaultClearM 239
5.31 Wait-for-Condition Functions 240
5.31.1 WaitMotionEnd 240

5.31.2 WaitLogicalMotionEnd 241
5.31.3 WaitForAsyncCall 242
5.31.4 WaitMotorEnabled 243
5.31.5 WaitInput 244
5.31.6 WaitUserCondition 244
5.31.7 WaitMotorCommutated 245
5.32 Callback Registration Functions 246
5.32.1 InstallCallback 246
5.32.2 InstallCallbackExt 247
5.32.3 SetCallbackMask 248
5.32.4 SetCallbackMaskExt 249
5.32.5 GetCallbackMask 249
5.32.6 GetCallbackMaskExt 250
5.32.7 SetCallbackPriority 250
5.33 Variables Management Functions 251
5.33.1 DeclareVariable 251
5.33.2 ClearVariables 252
5.34 Service Functions 253
5.34.1 GetFirmwareVersion 253
5.34.2 GetSerialNumber 254
5.34.3 GetBuffersCount 255
5.34.4 GetAxesCount 255
5.34.5 GetDBufferIndex 256
5.34.6 GetUMDVersion 257
5.35 Error Diagnostic Functions 257
5.35.1 GetMotorError 257
5.35.2 GetMotionError 258
5.35.3 GetEtherCATError 259
5.35.4 GetProgramError 260
5.36 Shared Memory Functions 260
5.36.1 GetSharedMemoryAddress 260
5.36.2 ReadSharedMemoryInteger 261
5.36.3 WriteSharedMemoryInteger 262
5.36.4 ReadSharedMemoryReal 262

5.36.5 WriteSharedMemoryReal 263
5.37 EtherCAT Functions 264
5.37.1 GetEtherCATState 264
5.37.2 MapEtherCATInput 264
5.37.3 MapEtherCATOutput 265
5.37.4 UnmapEtherCATInputsOutputs 266
5.37.5 GetEtherCATSlaveIndex 267
5.37.6 GetEtherCATSlaveOffsetV2 268
5.37.7 GetEtherCATSlaveVendorIDV2 269
5.37.8 GetEtherCATSlaveProductIDV2 270
5.37.9 GetEtherCATSlaveRevisionV2 271
5.37.10 GetEtherCATSlaveType 272
5.37.11 GetEtherCATSlaveRegister 273
5.37.12 GetEtherCATSlaveStateV2 274
5.37.13 GetEtherCATSlavesCount 275
5.38 Position Event Generation (PEG) Functions 276
5.38.1 AssignPegNTV2 276
5.38.2 AssignPegOutputsNT 277
5.38.3 AssignFastInputsNT 278
5.38.4 PegIncNTV2 279
5.38.5 PegRandomNTV2 285
5.38.6 WaitPegReadyNT 288
5.38.7 StartPegNT 289
5.38.8 StopPegNT 290
5.39 Dynamic Error Compensation 291
5.39.1 DynamicErrorCompensationOn 291
5.39.2 DynamicErrorCompensationOff 292
5.39.3 DynamicErrorCompensation1D 292
5.39.4 DynamicErrorCompensationN1D 294
5.39.5 DynamicErrorCompensationA1D 296
5.39.6 DynamicErrorCompensation2D 297
5.39.7 DynamicErrorCompensationN2D 300
5.39.8 DynamicErrorCompensationA2D 302
5.39.9 DynamicErrorCompensation3D2 303

5.39.10 DynamicErrorCompensation3D3 306
5.39.11 DynamicErrorCompensation3D5 309
5.39.12 DynamicErrorCompensation3DA 313
5.39.13 DynamicErrorCompensationN3D2 317
5.39.14 DynamicErrorCompensationN3D3 320
5.39.15 DynamicErrorCompensationN3D5 323
5.39.16 DynamicErrorCompensationN3DA 326
5.39.17 DynamicErrorCompensationRemove 330
5.40 Emergency Stop Functions 331
5.40.1 RegisterEmergencyStop 331
5.40.2 UnregisterEmergencyStop 332
5.41 Application Save/Load Functions 333
5.41.1 AnalyzeApplication 333
5.41.2 LoadApplication 333
5.41.3 SaveApplication 334
5.41.4 FreeApplication 335
5.42 Reboot Functions 336
5.42.1 ControllerReboot 336
5.42.2 ControllerFactoryDefault 336
5.43 Host-Controller File Operations 337
5.43.1 CopyFileToController 337
5.43.2 DeleteFileFromController 337
5.44 Save to Flash 338
5.44.1 ControllerSaveToFlash 338
5.45 SPiiPlusSC Management 340
5.45.1 StartSPiiPlusSC 340
5.45.2 StopSPiiPlusSC 340
5.46 Servo Processor Data Collection 341
5.46.1 SPDataCollectionStart 341
5.46.2 SPDataCollectionStop 343
5.46.3 GetSPAddress 343
5.46.4 WaitSPDataCollectionEnd 344
5.47 Controller Protection Functions 345
5.47.1 DefineControllerProtection 345

5.47.2 RemoveControllerProtection 346
5.47.3 TemporarilyDisableVariableProtection 347
5.47.4 RestoreVariableProtection 348
5.48 FRF Functions 349
5.48.1 FRF_Measure 349
5.48.2 FRF_Stop 350
5.48.3 FRF_CalculateMeasurementDuration 352
5.48.4 FRF_InitInput 352
5.48.5 FRF_ReadServoParameters 353
5.48.6 FRF_CalculateControllerFRD 353
5.48.7 FRF_CalculateOpenLoopFRD 354
5.48.8 FRF_CalculateClosedLoopFRD 355
5.48.9 FRF_CalculateStabilityMargins 356
5.48.10 FFT 356
5.48.11 JitterAnalysis 357
5.48.12 FRF_CrossCouplingMeasure 358
5.48.13 FRF_CalculateSLVRAT 359

List Of Figures

Figure 5-1. Emergency Stop Button 331

List of Tables

1. Introduction

The Python programming language has consistently maintained its position as one of the most popular languages in recent years.

Python has a wide selection of libraries for different purposes , providing users with the flexibility to accomplish tasks with notable ease.

This package will give the user the ability to communicate with the ACS controller with Python, like the SPiiPlus DotNet library.

2. Operational Environment and Other Requirements

The SPiiPlus Python Package supports all the Windows operating systems that are also supported by the C library. Linux is also supported, see below for supported versions

The SPiiPlus Python package relies on the "NumPy" package.

The package is also requires the Dynamic library of the C library (For example ACSCL_x64.dll for Windows 64-bit OS).

The communication channels supported by the Python library are the same as those supported by the C library.

SPiiPlus ADK Suite Version 3.14.01 supports the following Microsoft® Windows environments with Python versions 3.10, 3.11, and 3.12:

- Windows 10 (x64)
- Windows 11 (x64)
- Windows Server 2016 R2 (x64)
- Windows Server 2019 (x64)
- Windows Server 2022 (x64)

The following Linux distributions are supported by the SPiiPlus Python package.

- Ubuntu - version 22.04, python 3.10 version
- Ubuntu - version 18.04, python 3.6 version
- KUbuntu - version 18.04, python 3.6 version
- RedHat Enterprise Linux Server - version 7.9, python 3.6 version
- CentOS - version 7.9, python 3.6 version

3. Package Installation and Removal

3.1 Package Installation



For Linux users, employ 'sudo' when globally installing or removing the SPiiPlusPython package.

1. Go to the folder containing the files with the extension .whl

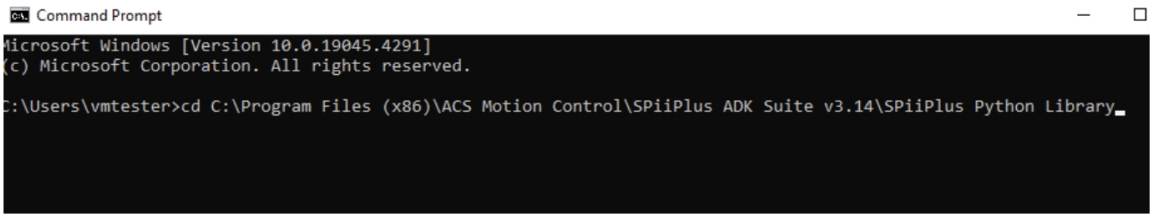
   Default .whl files path in Windows: C:\Program Files (x86)\ACS Motion Control\SPiiPlus ADK Suite v{MMI version}\SPiiPlus Python Library

   Default .whl file path in Linux: /lib/ACSMotionControl

2. Open CMD/Bash in the SPiiPlusPython folder

   Alternatively, open CMD/Bash and use the 'cd' command to navigate to the folder.

   Example:



3. Check the default Python version on the computer with the command:

In Windows

```
python --version
```

In Linux

```
python3 --version
```

If the default Python version differs from the version required by the distribution, adjust the default version to match the version supported by the library.

In windows, to change the default version of Python, follow these steps:

a. Find the directory where the python.exe file for the supported Python version is installed .

b. Open ‘edit the system environment variables’ .

c. Press ‘environment variable’ button in ‘Advanced’ .

d. Press edit of 'Path' variable in the user variables or system variables .

e. Define a path value to store the path of the directory containing the python.exe file, and another path value to store the path of the 'Scripts' folder located within the same directory of python.exe.

f. Make sure the environment variables have higher priority than other python variables in the list. If you define the paths in user variables, make sure no python variables are stored in system variables.

g. Reopen CMD and check again with 'python --version' to verify that the default python version has been set.

4. Run in CMD/Bash, filling in the variable sections indicated in curly braces:

In Windows

```
pip install SPiiPlusPython-{ppv}-cp{pv}-cp{abi}-{platform}.whl
```

In Linux

```
pip3 install SPiiPlusPython-{ppv}-cp{pv}-cp{abi}-{platform}.whl
```

Choose the file that corresponds to both the Python version and the PC operating system platform you use.

Key to version name:

{ppv} - Python Package Version

{pv} – Python version. 310, 311, etc.

{abi} – Application binary interface

{platform} – Operation system platform. Currently supports Windows and Linux platforms specified in Chapter 2.

For example, this command will install SPiiPlus Python version 3.14 on Python version 3.12, windows x86-64:

```
pip install SPiiPlusPython-3.14.0.0-cp312-cp312-win_amd64.whl
```

3.2 Package Removal

1. Open CMD/Bash
2. Run the following command:

In Windows

```
pip uninstall SPiiPlusPython
```

In Linux

```
pip3 uninstall SPiiPlusPython
```

3.3 Supplied Components

The SPiiPlus Python Package will be delivered as a compressed file with the extension of ".whl".

After installation, the package will be found in the folder "site-packages" as a folder named SPiiPlusPython.

The installation contains a binary file (with the extensions .pyd or .so depending on the OS) which serves as the interface to the C library. Additionally, there will be a directory named "SPiiPlusDefs" which contains the definitions of objects that the package relies upon.

Path	Component Name	Description
%SystemRoot%\System32	ACSCL_x64.dll	ACS C Dynamic Library for Windows 64-bit OS
/usr/lib/ACSMotionControl	libACSCL_x64.so	ACS C Dynamic Library for Linux 64-bit OS
.../site-packages/SPiiPlusPython to find the prefix path write in shell the command: python -m site --user-site	SPiiPlusPython folder	SPiiPlus package library after installation
.../SPiiPlusPython /SPiiPlusWarper.py	SPiiPlusWarper.py	Used as the API between the user and the package. Wrap the function inside the .pyd to give the ability to the user to see the function inside the package.

Path	Component Name	Description
.../SPiiPlusPython /SPiiPlusBinary.pyd	SPiiPlusBinary.pyd	The API between the C library and the Python package. Contains the calls for the C library functions
.../SPiiPlusPython /SPiiPlusDefs/SPiiPlusStructs.py	SPiiPlusStructs.py	Contains the definitions of struct objects which the user can use
.../SPiiPlusPython /SPiiPlusDefs/SPiiPlusEnums.py	SPiiPlusEnums.py	Contains the definitions of enum objects which the user can use
.../SPiiPlusPython /SPiiPlusDefs/GlobalVariables.py	GlobalVariables.py	Contains additional objects which the user can use

4. SPiiPlus C Library Compatibility

4.1 Unsupported C Library Functions

The version 3.14 of the SPiiPlus Python Package is compatible with the version 7.4.0.0 of the SPiiPlus C Library, excluding the functions listed below.

The following functions are **not** supported by the SPiiPlus Python Package:

> All FRF library functions

4.2 Function Naming Convention

All the functions names were taken from the ACSC.h header file, excluding the "acsc_" prefix.

Except for the functions listed in the Python Library Functions section, every function in the package implements the same functionality as the corresponding function in the C library and the function parameters share the same meaning as the corresponding parameter with the same name in the C library function.

Parameters which use to return values by reference (as a pointer to the value) to the caller function in the C library, will be returned by value by the Python SPiiPlus function.

Example: The C result stored in the "Values" parameter of the function "acsc_ReadReal" C library will be returned by value to the caller in the Python package.

```
import SPiiPlusPython as sp

hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
print(f"CTIME: {sp.ReadReal(hc, -1, 'CTIME', 0, 0, 0, 0,
sp.SYNCHRONOUS)}")
# CTIME: 0.5
```

5. Python Library Functions

5.1 Communications Functions

5.1.1 OpenCommSerial

Description

The method opens communication with the controller via a serial port.

Syntax

```
def OpenCommSerial(channel, rate):
```

Arguments

channel	Communication channel: 1 corresponds to COM1, 2 – to COM2, etc.
rate	Communication rate in bits per second (baud). This parameter must be equal to the controller variable IOBAUD to link successfully with the controller.

Return Value

int

The result refers to the to the open communication channel. If the function succeeds, the return value is zero or greater number. If the function fails, the return value is -1.

Remarks

Get extended error information by calling GetLastError.

After a channel is open, any SPiiPlus Python function works with the channel irrespective of the physical nature of the channel.

5.1.2 OpenCommEthernetTCP

Description

The method opens communication with the controller via Ethernet using the UDP protocol.

Syntax

```
OpenCommEthernetUDP(address, port)
```

Arguments

address	string that contains the network address of the controller in symbolic or TCP/IP dotted form.
port	You can use either: EthernetCommOption.ACSC_SOCKET_STREAM_PORT or define it using the ACSPL+ variable: UDPPORT (see SPiiPlus ACSPL+ Command & Variable Reference Guide)

Return Value

int

The result refers to the to the open communication channel. If the function succeeds, the return value is zero or greater number. If the function fails, the return value is -1.

Remarks

Get extended error information by calling GetLastError.

After a channel is open, any SPiiPlus Python function works with the channel irrespective of the physical nature of the channel.

Example

```
ip = "10.0.0.89"
port = 701
hc = sp.OpenCommEthernetTCP(ip, port)
if hc == -1:
   print("Failed to connect")
```

5.1.3 OpenCommEthernetUDP

Description

The method opens communication with the controller via Ethernet using the UDP protocol.

Syntax

OpenCommEthernetUDP(address, port)

Arguments

address	string that contains the network address of the controller in symbolic or TCP/IP dotted form.
port	You can use either: EthernetCommOption.ACSC_SOCKET_DGRAM_PORT or Define it using the ACSPL+ variable: **UDPPORT** (see *SPiiPlus ACSPL+ Command & Variable Reference Guide*)

Return Value

None

Get extended error information by calling GetLastError.

Remarks

After a channel is open, any SPiiPlus Python function works with the channel irrespective of the physical nature of the channel.

Example

```
ip = "10.0.0.89"
port = 701
hc = sp.OpenCommEthernetUDP(ip, port)
if hc == -1:
   print("Failed to connect")
```

5.1.4 OpenCommSimulator

Description

The function executes the SPiiPlus stand-alone simulator via SPiiPlus User ModeDriver in the case that it is not running.

The function connects to the simulator using the TCP/IP protocol.

Syntax

OpenCommSimulator()

Get extended error information by calling GetLastError.

Comments

Error codes (in case of failure):

ErrorCodes.ACSC_SIMULATOR_RUN_EXT - if the ports set for simulator in UMD are taken by another application.

ErrorCodes.ACSC_SIMULATOR_NOT_SET - in case a simulator execution attempt was made without setting a simulator executable and default simulator executable was not found.

Example

```
hc = sp.OpenCommSimulator (ip, port)
if hc == -1:
   print("Failed to connect")
```

5.1.5 CloseSimulator

Description

The function stops the SPiiPlus simulator via UMD in the case that it is running.

Syntax

CloseSimulator(failure_check=False)

Get extended error information by calling GetLastError.

Comments

Error codes (in case of failure):

ErrorCodes.ACSC_SIMULATOR_NOT_RUN - an attempt to stop simulator was made without it running.

Example

```
sp.CloseSimulator(failure_check=True)
```

5.1.6 GetEthernetCardsExt

Description

The function detects available controllers through a standard Ethernet connection and retrieves the IP address, serial number, part number, and firmware version, which are stored in an ACSC_CONTROLLER_INFO class. By default, the function searches the local network segment. The default setting can be changed, as described below.

Syntax

```
def GetEthernetCardsExt(controllerInfoSize, max, broadcastAddress,
failure_check=False):
```

Arguments

controllerInfoSize	Size of the of an object ACSC_CONTROLLER_INFO object. Can use the command: sys.getsizeof(SPiiPlusPython.ACSC_CONTROLLER_INFO)
max	Max length of the return list contains ACSC_CONTROLLER_INFO objects
broadcastAddress	IP address for broadcasting. Normally should be GeneralDefinition.ACSC_NONE. See comments below for more details.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors .

Return Value

List of ACSC_CONTROLLER_INFO objects.

Comments

Get extended error information by calling GetLastError.

Remarks

The function proceeds as follows:

1. Broadcasts a message to the network.
2. Collects all received replies.
3. Filters out all replies sent by nodes other than the SPiiPlus controller.
4. The function then stores controller's IP addresses in the IP address field in the ACSC_CONTROLLER_INFO list.

5. Broadcasts a private message to each ip address field in ACSC_CONTROLLER_INFO list. The messages ask for the controller serial number, part number and firmware version, then the function stores them in the relevant field and returns.

To obtain the Controller's IP address, you can utilize the inet_ntoa function from the built-in socket module. For instance, if you have a variable named 'card' of type ACSC_CONTROLLER_INFO, the user can retrieve the controller's IP address by calling

```
socket.inet_ntoa(struct.pack("!L", socket.htonl(int(card.IpAddress.s_
addr))))
```

This will provide the IP address associated with the specified controller.

Example

```
IPAddresses = sp.GetEthernetCardsExt(sys.getsizeof( sp.ACSC_APPSL_INFO_
c), 10, socket.INADDR_BROADCAST, True)
process_ip = sp.GetConnectionInfo(hc, True).EthernetIP
for ip_addr in IPAddresses:
   ip = socket.inet_ntoa(struct.pack("!L", socket.htonl(int(ip_
addr.IpAddress.s_addr))))
      print("Card data:")
      print(f"Controller ip: {ip}")
      print(f"Controller FW version: {ip_addr.Version}")
      print(f"Controller serial number: {ip_addr.SerialNumber}\n\n")
```

5.1.7 SetServerExtLogin

Description

The function defines the User Mode Driver (UMD) host IP address, and port along with passing login data.

Syntax

SetServerExtLogin(iP, port, username, password, domain, failure_check=False)

iP	IP address where to look for server.
port	Port number to connect over.
username	String of valid username.
password	String of valid password.
domain	String of domain or workgroup.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

Use the function only if the application needs to establish communication with a controller through a remote computer.

If the function is not called, by default, all connections are established through the local computer. Only a controller connected to the local computer by a serial cable, PCI bus or Ethernet can be accessed.

The function sets the IP address and port number of the User Mode Driver (UMD) host and logs the user in. Once the function is called all **OpenCommxxx** calls will attempt to establish communication via the UMD that was specified in the most recent **SetServrExtLogin** call.

In order to establish communication via a different UMD host **SetServrExtLogin** must be called again.

Example

```
userName = "myUser"
password = "1234"
domain = "myDomain"
acsApi.SetServerExtLogin("10.0.0.6",9999,userName,password,
                         domain,True)
```

5.1.8 CloseComm

Description

The function closes communication via the specified communication channel.

Syntax

```
def CloseComm(handle, failure_check=False):
```

Arguments

handle	Communication handle
failure_check	If True when the function detected an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

Get extended error information by calling GetLastError.

The function closes the communication channel and releases all system resources related to the channel. If the function closes communication with the Simulator, the function also terminates the Simulator. Each OpenComm*** call in the application must make the corresponding CloseComm call in order to return the resources to the system.

Example

```
# Open communication channel
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Close communication channel
sp.CloseComm(hc)
```

5.1.9 Transaction

Description

The function executes one transaction with the controller, i.e. it sends a command and receives a controller response.

Syntax

```
def Transaction(handle, outBuf, outCount, inCount, wait, failure_
check=False):
```

Arguments

handle	Communication handle
outBuf	string that contains the command to be sent.
outCount	Number of characters in the command
inCount	Number of characters in the return buffer
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is set to IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

String type. The function returns a string containing the controller response. If the controller has no response the return string will be empty.

Comments

Get extended error information by calling GetLastError.

Calling **Transaction** executes the following steps:

1. Send outCount characters from outBuf to the controller.
2. Waits until the controller response is received, or the timeout occurs. In the case of timeout, store error value and return.

3. Analyzes the controller response, and set the error value if the response indicates an error.
4. Store the controller response in response string. If the controller response is longer than InCount, store only InCount characters.

If the `wait` argument is SYNCHRONOUS, the call blocks and the function does not return until the full operation is finished. If the `wait` argument points to a valid ACSC_WAITBLOCK, the call is non-waiting, and the function returns immediately after the first step. An internal library thread executes the rest of the operation. The calling thread can check if the operation finished and can retrieve the operation result using the GetResult function.

Get extended error information by calling GetLastError.

Example

```
buf = "?VR\r"
fw = sp.Transaction(hc, buf, len(buf), 256, sp.SYNCHRONOUS, True)
print(f"firmware verions: {fw}")
```

5.1.10 Command

Description

The function sends a command to the controller and analyzes the controller's response.

Syntax

```
Command(handle, outBuf, outCount, wait, failure_check=False)
```

Arguments

handle	Communication handle
outBuf	String that contains the command to be sent.
outCount	Number of characters in the command
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors .

Return Value

None

Comments

Get extended error information by calling GetLastError.

The function is similar to Transaction except that the controller response is not transferred to the calling thread. The function is used mainly for the commands that the controller responds to with a prompt. In this case, the exact characters that constitute the prompt are irrelevant for the calling thread. The function provides analysis of the prompt, and if the operation fails, the calling thread can obtain the error code.

If the `wait` is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the `wait` object before calling WaitForAsyncCall or GetResult.

5.1.11 GetResult

Description

GetResult provides the ability to retrieve the results of the previous asynchronous operation.

The **GetResult** function collects the input buffer of the last asynchronous call, then converts the buffer to a specific type which was saved on the system (int type, float type, string type, etc.) and returns the converted buffer to the caller.

Syntax

```
def GetResult(handle, wait, timeout, shape):
```

Arguments

handle	Communication handle
wait	Instance of ACSC_WAITBLOCK. Wait block which was used by the Async method
timeout	Number of milliseconds to wait for result
shape	Convert the return value to the input shape. In case the user does not provide a shape, the function will yield a single item. Should the user provide shape input with one or two dimensions, the function will yield a numpy.ndarray of the same dimension, preserving the item type as maintained by the system.
struct_type	When the asynchronous function should return class representing struct object, the user needs to send through struct_type the class type. If the function should return a user defined class ,Like with the case of ReadStruct, The user is required to provide their custom-defined class, which must inherit from ctypes.Structure.

Return Value

int. float, str, np.ndarray, depending on the parameter type sent by the asynchronous call

Comments

GetResult executes the last asynchronous function call which used the given ACSC_WAITBLOCK.

The function returns a value only when the last asynchronous call using the given ACSC_WAITBLOCK is ready to return a value to the user.

When the last asynchronous is not intended to return a value, **GetResult** will return None.

When the final asynchronous function is expected to return a value, **GetResult** will convert the stored buffer to a type that corresponds to the expected return type of the asynchronous function.

If the user doesn't modify the shape parameter, the **GetResult** converts one item from the stored buffer and returns it to the user.

If the user modifies the shape to a list containing a up to 2 items, representing the expected shape to be obtained from the function, the function will attempt to transform the stored buffer into a list with the specified shape.

Example

```
sp.ReadReal(hc, -1, "MyGlobalReal", -1, -1, -1, -1, wait, True)
result = sp.GetResult(hc, wait) # The result will be float
print(result)
sp.ReadReal(hc, -1, "MyGlobalInt", 0, 1, 0, 1, wait, True)
result = sp.GetResult(hc, wait, shape=(2, 2))
# The result will array with the shape of (2,2)
```

5.1.12 CancelOperation

Description

The function cancels any asynchronous (non-waiting) call or all operations with the specified communication handle.

Syntax

```
def CancelOperation(handle, wait, failure_check=False):
```

Arguments

handle	Communication handle
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

Get extended error information by calling GetLastError.

5.1.13 GetConnectionsList

Description

The function retrieves all currently opened connections on the active server and their details.

Syntax

GetConnectionsList(maxNumConnections, failure_check=False)

Arguments

maxNumConnections	Number of ConnectionsList array elements.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

List of **ACSC_CONNECTION_DESC** objects.

Get extended error information by calling GetLastError.

Comments

Up to MaxNumConnections connections are stored in the array.

This function can be used to check if there are some unused connections that remain from an application that did not close the communication channel or were not gracefully closed (terminated or killed).

Each connection from returned list can be terminated only by the TerminateConnection function.



Using the SetServerExtLogin function makes a previously returned connections list irrelevant because it changes the active server.

Example

```
connection_list = sp.GetConnectionsList(100, True)
   for item in connection_list:
      print(f"Process name: {item.Application}")
      print(f"Process id: {item.ProcessId}")
```

5.1.14 GetConnectionInfo

Description

The function is used to retrieve the details of opened communication channel.

Syntax

GetConnectionInfo(handle, failure_check=False)

Arguments

handle	Communication handle
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

ACSC_CONNECTION_INFO object.

Get extended error information by calling GetLastError.

5.1.15 TerminateConnection

Description

The function terminates a given communication channel (connection) of the active server.

Syntax

TerminateConnection(connection, failure_check=False)

Arguments

connection	The connection to terminate defined by ACSC_CONNECTION_DESC object. Each connection is defined by the ACSC_CONNECTION_DESC object. The GetConnectionsList function is responsible for initializing the list with ACSC_CONNECTION_DESC.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Example

```
connection_list = sp.GetConnectionsList(100, True)
# Terminate last connection
sp.TerminateConnection(connection_list[-1], True)
```

5.2 Service Communication Functions

5.2.1 GetLastError

Description

The function returns the calling thread's last error code value. The last error code is maintained on a per-thread basis. Multiple threads do not overwrite each other's last error code.

Syntax

```
def GetLastError():
```

Return Value

The return value is the calling thread's last-error code value. (int type)

Remarks

This function will return the error code only when the last function was call with failure_check is False. All of the functions rewrite the error code value when they are called. For a complete List of Error Codes, see Error Codes.

Example

```
sp.ReadReal(hc, -1, "varName", -1, -1, -1, -1, sp.SYNCHRONOUS)
err = sp.GetLastError()
if err != 1064:
   print("variable name is undefined")
```

5.2.2 GetErrorString

Description

The function retrieves the explanation of an error code.

Syntax

```
GetErrorString(handle, errorCode, count, failure_check=False):
```

Arguments

handle	Communication handle
errorCode	Error code.
count	Max Size of the returned string.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors .

Return Value

The return value is the calling thread's last-error string explanation. (string type).

Comments

Get extended error information by calling GetLastError.

The function retrieves the string that contains the text explanation of the error code returned by the GetLastError, GetMotorError, and GetProgramError functions.

The function will not copy more than `count` characters to the return string. If the buffer is too small, the error explanation can be truncated.

For the SPiiPlus Python Library error codes, the function returns immediately with the text explanation. For the controller's error codes, the function refers to the controller to get the text explanation.

5.2.3 SetQueueOverflowTimeout

Description

The function sets the Queue Overflow Timeout.

Syntax

```
def SetQueueOverflowTimeout(handle, timeout, failure_check=False):
```

Arguments

handle	Communication handle
timeout	Timeout value in milliseconds
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Comments

Get extended error information by calling GetLastError.

5.2.4 GetQueueOverflowTimeout

Description

The function retrieves the Queue Overflow Timeout.

Syntax

```
def GetQueueOverflowTimeout(handle, failure_check=False):
```

handle	Communication handle
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

If the function succeeds, the return value is the current Queue Overflow Timeout value in milliseconds. If the function fails, the return value is -1.

Comments

Get extended error information by calling GetLastError.

5.2.5 SetTimeout

Description

The function retrieves the Queue Overflow Timeout.

Syntax

```
def SetTimeout(handle, timeout, failure_check=False):
```

Arguments

handle	Communication handle
timeout	Maximum waiting time in milliseconds.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Comments

Get extended error information by calling GetLastError.

The function sets the communication timeout.

All of the subsequent waiting calls of the functions will wait for the controller response timeout in milliseconds. If the controller does not respond to the sent command during this time, SPiiPlus Python functions return with zero value.

In this case, the call of GetLastError will return the **ErrorCodes.ACSC_TIMEOUT** error.

5.2.6 GetTimeout

Description

The function retrieves communication timeout.

Syntax

GetTimeout(handle, failure_check=False)

Arguments

handle	Communication handle
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors .

Example

```
ip = "10.0.0.89"
port = 701
```

```
hc = sp.OpenCommEthernetTCP(ip, port)
# Get current timeout
timeout = sp.GetTimeout(hc)
# Add 500 milliseconds to communication timeout
sp.SetTimeout(hc2, timeout+500, True)
```

5.2.7 GetDefaultTimeout

Description

The function retrieves default communication timeout.

Syntax

```
GetDefaultTimeout(handle, failure_check=False)
```

Arguments

handle	Communication handle
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type.
If the function succeeds, the return value is the default time-out value in milliseconds.
If the function fails, the return value is zero.

Remarks

The value of the default time-out depends on the type of established communication channel.
Timeout also depends on the baud rate value for serial communication.

Get extended error information by calling GetLastError.

5.2.8 SetIterations

Description

The function sets the number of iterations (command repetitions) in one transaction.

Syntax

def SetIterations(handle, iterations, failure_check=False):

Arguments

handle	Communication handle
iterations	Number of iterations or command repitions
failure_check	If True when the function detected error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

If, after the transmission of command to the controller, the controller response is not received during the predefined time, the library repeats the transmission of command.

The number of those iterations is defined by the Iterations parameter for each communication channel independently.

Most of the SPiiPlus Python functions perform communication with the controller by transactions (i.e., they send commands and wait for responses) that are based on the Transaction function. Therefore, the changing of number of iterations can have an influence on the behavior of the user application.

The default number of iterations for all communication channels is 2.

Get extended error information by calling GetLastError.

5.2.9 SetCommOptions

Description

The function sets the communication options.

Syntax

def SetCommOptions(handle, options, failure_check=False):

Arguments

handle	Communication handle
Options	Communication options to be set Bit-mapped parameter that can include one of the following flags: CommOptions.ACSC_COMM_USE_CHECKSUM: the communication mode used when each command is sent to the controller with checksum and the controller also responds with checksum. CommOptions.ACSC_COMM_AUTORECOVER_HW_ERROR: When a hardware error is detected in the communication channel and this bit is set, the library automatically repeats the transaction, without counting iterations.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Remarks

The function sets the communication options. To get current communication option, call GetCommOptions.
To add some communication options to the current configuration, modify an option parameter that has been filled in by a call to GetCommOptions. This ensures that the other communication options will have same values.

Get extended error information by calling GetLastError.

5.2.10 GetCommOptions

Description

The function retrieves the communication options.

Syntax

GetCommOptions(handle, failure_check=False)

Arguments

handle	Communication handle
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type
The return value is bit-mapped to represent the current communication options.

Remarks

The function retrieves the current communication options. To set the communication option call SetCommOptions.

Get extended error information by calling GetLastError.

5.2.11 ClearBreakpoints

Description

Remove any saved breakpoints in the designated line and buffer.

Syntax

ClearBreakpoints(handle, buffer, line, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
line	Line to save the breakpoint
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

5.2.12 SetBreakpoint

Description

Place a breakpoint at the indicated line within the designated buffer.

Syntax

SetBreakpoint(handle, buffer, line, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
line	Line on which to to place breakpoint
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResultfunction to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Set breakpoint in buffer 1 line 2
sp.SetBreakpoint(hc, 1, 2, sp.SYNCHRONOUS, True)
```

5.2.13 GetBreakpointsList

Description

Function retrieves all breakpoints of a buffer up to arraySize.

Syntax

def GetBreakpointsList(handle, buffer, arraySize, wait, failure_check=False):

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
arraySize	Maximum length of the return list.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

list of int - a list of numbers representing the lines in which the breakpoints are located

Get extended error information by calling GetLastError.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Get a list of up to 20 breakpoints from buffer 1
Breakpoint_list = sp.GetBreakpointsList(hc, 1, 20, sp.SYNCHRONOUS, True)
```

5.2.14 GetLibraryVersion

Description

The function retrieves the SPiiPlus C Library version number.

Syntax

GetLibraryVersion()

Return Value

Int type.
The return value is an integer value, which specifies the binary version number.

Comments

The SPiiPlus C Library version consists of four (or fewer) numbers separated by points: #.#.#.#. The binary version number is represented by 32-bit unsigned integer value. Each byte of this value specifies one number in the following order: high byte of high word – first number, low byte of high

word – second number, high byte of low word – third number and low byte of low word – forth number. For example version "2.10" has the following binary representation (hexadecimal format): 0x020A0000. The first two numbers in the string form are required. Any release version of the library consists of two numbers. The third and fourth numbers specify an alpha or beta version, special or private build, or other version, if relevant.

5.3 ACSPL+ Program Management Functions

5.3.1 LoadBuffer

5.3.2 LoadBuffer

Description

The function clears the specified program buffer and then loads ACSPL+ program to this buffer.

Syntax

LoadBuffer(handle, buffer, text, count, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
text	String containing an ACSPL+ program(s).
count	Length of the input string.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function clears the specified program buffer and then loads ACSPL+ program to this buffer. No compilation or syntax check is provided during downloading. Any text, not only a correct program, can be inserted into a buffer. To compile the program and check its accuracy, the compile command must be executed after downloading.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
buf = '''GLOBAL Arr1(100)\
         \nGLOBAL string strType(30) = \"check for debug\"\
         \nSTOP'''

# load 'buf' string to buffer 2
sp.LoadBuffer(hc, 2, buf, len(buf), sp.SYNCHRONOUS, failure_check=True)
```

5.3.3 LoadBuffersFromFile

Description

The function opens a file that contains one or more ACSPL+ programs allocated to several buffers and downloads the programs to the corresponding buffers.

Syntax

LoadBuffersFromFile(handle, filename, wait, failure_check=False)

Arguments

handle	Communication handle
filename	Path of the file
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function analyzes the file, determines which program buffers should be loaded, clears them and then loads ACSPL+ programs to those buffers.
SPiiPlus software tools save ACSPL+ programs in the following format:

```
# Header: Date, Firmware version etc.
#Buf1 (buffer number)
ACSPL+ program of Buf1
#Buf2 (buffer number)
ACSPL+ program of Buf2
#Buf3 (buffer number)
ACSPL+ program of Buf3 etc.
```

The number of buffers in a file may change from 0 to 63 (depending on controller), without any default order.
No compilation or syntax check is provided during downloading. Any text, not only a correct program, can be inserted into a buffer.
In order to compile the program and check its accuracy, the compile command must be executed after downloading.
If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.

5.3.4 AppendBuffer

Description

The function appends one or more ACSPL+ lines to the program in the specified buffer.

Syntax

AppendBuffer(handle, buffer, text, count, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
text	String containing an ACSPL+ program(s).
count	Length of the input string.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Remarks

The function appends one or more ACSPL+ lines to the program in the specified buffer. If the buffer already contains any program, the new text is appended to the end of the existing program.
No compilation or syntax check is provided during downloading. In fact, any text, not only a correct program, can be inserted into a buffer.
In order to compile the program and check its accuracy, the compile command must be executed after downloading.
If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.

5.3.5 UploadBuffer

Description

The function uploads ACSPL+ program from the specified program buffer.

Syntax

UploadBuffer(handle, buffer, offset, count, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
text	String containing an ACSPL+ program(s).
count	Length of the input string.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

String - Representing the buffer code.

Get extended error information by calling GetLastError.

5.3.6 ClearBuffer

Description

The function deletes the specified ACSPL+ program lines in the specified program buffer.

Syntax

ClearBuffer(handle, buffer, fromLine, toLine, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
fromLine	These Arguments specify a range of lines to be deleted. fromLine starts c.
fromLine	If toLine is larger than the total number of lines in the specified program buffer, the range includes the last program line.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function deletes the specified ACSPL+ program lines in the specified program buffer.
If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or WaitForAsyncCall function.

5.3.7 CompileBuffer

Description

The function compiles ACSPL+ program in the specified program buffer(s).

Syntax

CompileBuffer(handle, buffer, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller). Use GeneralDefinition.ACSC_NONE instead of the buffer number, to compile all programs in all buffers.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

In order to get information about compilation results, use ReadInteger to read **PERR** [X], which contains the last error that occurred in buffer X. If **PERR** [X] is zero, the buffer was compiled successfully.
Otherwise, **PERR** [X] tells you about the error that occurred during the compilation.
The function compiles ACSPL+ program in the specified program buffer or all programs in all buffers if the parameter Buffer is GeneralDefinition.ACSC_NONE.
The function deletes the specified ACSPL+ program lines in the specified program buffer.
If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.



If attempting to compile the D-Buffer, all other buffers will be stopped and put in a noncompiled state.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
buf = '''ENABLE 0\
         \njog 0, +\
         \nSTOP'''

# Load and compile buffer 2
sp.LoadBuffer(hc, 2, buf, len(buf), sp.SYNCHRONOUS, failure_check=True)
sp.CompileBuffer(hc, 2, sp.SYNCHRONOUS, failure_check=True)
```

5.3.8 RunBuffer

Description

The function starts up an ACSPL+ program in the specified buffer.

Syntax

RunBuffer(handle, buffer, label, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
label	Label in the program that the execution starts from. If None is specified instead of a String, the execution starts from the first line.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function starts up ACSPL+ program in the specified buffer. The execution starts from the specified label, or from the first line if the label is not specified.
If the program was not compiled before, the function first compiles the program and then starts it. If an error was encountered during compilation, the program does not start.
If the program was suspended by the SuspendBuffer function, the function resumes the program execution from the point where the program was suspended.
The function can wait for the controller response or can return immediately as specified by the wait argument. The controller response indicates that the program in the specified buffer was started successfully. The function does not wait for the program end. To wait for the program end, use the WaitProgramEnd function.
If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.

5.3.9 StopBuffer

Description

The function stops the execution of ACSPL+ program in the specified buffer(s).

Syntax

StopBuffer(handle, buffer, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function stops ACSPL+ program execution in the specified buffer or in all buffers if the parameter Buffer is GeneralDefinition.ACSC_NONE. The function has no effect if the program in the specified buffer is not running.
If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.

5.3.10 SuspendBuffer

Description

The function suspends the execution of ACSPL+ program in the specified program buffer(s).

Syntax

SuspendBuffer(handle, buffer, wait, failure_check=False)

Arguments

handle	Communication handle
buffer	Buffer number, from 0 to 63 (depending on controller).
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

The function suspends ACSPL+ program execution in the specified buffer or in all buffers if the parameter buffer is GeneralDefinition.ACSC_NONE. The function has no effect if the program in the specified buffer is not running. To resume execution of the program in the specified buffer, call the RunBuffer function.
If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.

5.4 Read and Write Variables Functions

5.4.1 ReadInteger

Description

The function reads value(s) from integer variable.

Syntax

ReadInteger(handle, nBuf, var, from1, to1, from2, to2, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Buffer number, from 0 to 63 (depending on controller).
var	string that contains a name of the variable
from1, to1	Index range (first dimension).
from2, to2	Index range (second dimension).
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int, or np.array with dtype is int in accordance with the return object shape.

Get extended error information by calling GetLastError.

Comments

The function reads specified integer variable or array.
The variable can be a standard controller variable, a user global variable, or a user local variable.
Standard and user global variables have global scope. Therefore, parameter nbuf must be GeneralDefinition.ACSC_NONE (-1) for these classes of variables.
User local variable exists only within a buffer. The buffer number must be specified for user local variable.
If the variable is scalar, all indexes from1, to1, from2, to2 must be GeneralDefinition.ACSC_NONE. The function reads the requested value and returns it to user.
If the variable is a one-dimensional array, from1, to1 must specify the index range and from2, to2 must be GeneralDefinition.ACSC_NONE. Array Values must be of size to1-from1+1 at least. The function reads all requested values from index from1 to index to1 inclusively and returns them. If the variable is a two-dimensional array, from1, to1 must specify the index range of the first dimension and from2, to2 must specify the index range of the second dimension. Array Values must be of size (To1-From1+1)x(To2-From2+1) values at least.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.4.2 WriteInteger

Description

The function writes value(s) to integer variable.

Syntax

WriteInteger(handle, nBuf, var, from1, to1, from2, to2, values, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Buffer number, from 0 to 63 (depending on controller).
var	string that contains a name of the variable.
from1, to1	Index range (first dimension).
from2, to2	Index range (second dimension).
values	Object contained values that must be written. For scalar input - the object should be convertible to pyhon int type. For multi-item input - the object should be convertible to numpy.ndarry type.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes to a specified integer variable or array.
The variable can be a standard controller variable, a user global variable, or a user local variable. Standard and user global variables have global scope. Therefore, parameter nbuf must be GeneralDefinition.ACSC_NONE (-1) for these classes of variables.
User local variable exists only within a buffer. The buffer number must be specified for user local variable.
If the variable is scalar, all indexes from1, to1, from2, to2 must be GeneralDefinition.ACSC_NONE. The function writes the requested value.
If the variable is a one-dimensional array, from1, to1 must specify the index range and from2, to2 must be GeneralDefinition.ACSC_NONE. Array Values must be of size to1-from1+1 at least. The function writes all requested values from index from1 to index to1 inclusively. If the variable is a two-dimensional array, from1, to1 must specify the index range of the first dimension and from2, to2 must specify the index range of the second dimension. Array Values must be of size (To1-From1+1)x (To2-From2+1) values at least.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
val = 0.86
# change V0 value to 0.86
vals = [[1,2], [3,4]]
sp.WriteInteger(hc, 1, "myarray", 0, 1, 0, 1, vals, sp.SYNCHRONOUS, True)
# should return array with the shape of (2, 2)
result = sp.ReadInteger(hc, 1, "myarray", 0, 1, 0, 1, sp.SYNCHRONOUS,
True)
print(result)
```

5.4.3 ReadReal

Description

The function writes value(s) to the real variable.

Syntax

ReadReal(handle, nBuf, var, from1, to1, from2, to2, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Buffer number, from 0 to 63 (depending on controller).
var	string that contains a name of the variable.
from1, to1	Index range (first dimension).
from2, to2	Index range (second dimension).
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float, or np.array with dtype is np.double in accordance with the return object shape.

Get extended error information by calling GetLastError.

Comments

If Wait is a SYNCHRONOUS object, the function returns when the controller response is received.
If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result.
If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

5.4.4 WriteReal

Description

The function writes value(s) to the real variable.

Syntax

WriteReal(handle, nBuf, var, from1, to1, from2, to2, values, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Buffer number, from 0 to 63 (depending on controller).
var	string that contains a name of the variable.
from1, to1	Index range (first dimension).
from2, to2	Index range (second dimension).
values	Object contained values that must be written. For scalar input - the object should be convertible to pyhon float type. For multi-item input - the object should be convertible to numpy.ndarry type.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function writes to a specified real variable or array.
The variable can be a standard controller variable, a user global variable, or a user local variable.
Standard and user global variables have global scope. Therefore, parameter nbuf must be GeneralDefinition.ACSC_NONE (-1) for these classes of variables.
User local variable exists only within a buffer. The buffer number must be specified for user local variable.
If the variable is scalar, all indexes from1, to1, from2, to2 must be GeneralDefinition.ACSC_NONE. The function writes the requested value.
If the variable is a one-dimensional array, from1, to1 must specify the index range and from2, to2 must be GeneralDefinition.ACSC_NONE. Array Values must be of size to1-from1+1 at least. The function writes all requested values from index from1 to index to1 inclusively. If the variable is a two-dimensional array, from1, to1 must specify the index range of the first dimension and from2, to2 must specify the index range of the second dimension. Array Values must be of size (To1-From1+1)x (To2-From2+1) values at least.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
val = 0.86
# change V0 value to 0.86
sp.WriteReal(hc, -1, "V0", -1, -1, -1, -1, val, sp.SYNCHRONOUS, True)
# Function Should return 0.86 float
res = sp.ReadReal(hc, -1, "V0", -1, -1, -1, -1, sp.SYNCHRONOUS, True)
print(res)
```

5.4.5 ReadString

Description

The function writes value(s) to the real variable.

Syntax

ReadString(handle, nBuf, var, from1, to1, from2, to2, stringSize, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Buffer number, from 0 to 63 (depending on controller).
var	string that contains a name of the variable
from1, to1	Index range (first dimension).
from2, to2	Index range (second dimension). (Not supported)
stringSize	The maximum length of each string.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

string, list of strings in accordance with the return object shape.

Comments

The function reads specified real variable or array.
The variable can be a standard controller variable, a user global variable, or a user local variable. Standard and user global variables have global scope. Therefore, parameter nbuf must be GeneralDefinition.ACSC_NONE (-1) for these classes of variables.
User local variable exists only within a buffer. The buffer number must be specified for user local variable.
If the variable is single string, all indexes from1, to1, from2, to2 must be GeneralDefinition.ACSC_NONE. The function reads the requested value and returns it to user.
If the variable is a one-dimensional array, from1, to1 must specify the index range and from2, to2 must be GeneralDefinition.ACSC_NONE. Array Values must be of size to1-from1+1 at least. The function reads all requested values from index from1 to index to1 inclusively and returns them. If the variable is a two-dimensional array, from1, to1 must specify the index range of the first dimension and from2, to2 must specify the index range of the second dimension. Array Values must be of size (To1-From1+1)x(To2-From2+1) values at least.

If Wait is a SYNCHRONOUS object, the function returns when the controller response is received.
If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result.
If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

5.4.6 WriteString

Description

The function writes values to ACSPL standard or user defined String variables or arrays.

Syntax

WriteString(handle, nBuf, var, from1, to1, from2, to2, strings, stringSize, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Buffer number, from 0 to 63 (depending on controller).
var	string that contains a name of the variable.
from1, to1	Index range (first dimension).
from2, to2	Index range (second dimension).
strings	String object or list of strings, to write on variable of array of string.
stringSize	The maximum length of each string.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

None.

Comments

The function writes to a specified string variable or array.
The variable can be a standard controller variable, a user global variable, or a user local variable. Standard and user global variables have global scope. Therefore, parameter nbuf must be GeneralDefinition.ACSC_NONE (-1) for these classes of variables.
User local variable exists only within a buffer. The buffer number must be specified for user local variable.
If the variable single string, all indexes from1, to1, from2, to2 must be GeneralDefinition.ACSC_NONE. The function writes the requested value.
If the variable is a one-dimensional array, from1, to1 must specify the index range and from2, to2 must be GeneralDefinition.ACSC_NONE. Array Values must be of size to1-from1+1 at least. The function writes all requested values from index from1 to index to1 inclusively. If the variable is a two-dimensional array, from1, to1 must specify the index range of the first dimension and from2, to2 must specify the index range of the second dimension. Array Values must be of size (To1-From1+1)x (To2-From2+1) values at least.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
vals = ["check string1", "check string2"]
max_length = 20
sp.WriteString(hc, axis, "myString2D", 0, 1, -1, -1, vals, max_length,
sp.SYNCHRONOUS, True)
string_list = sp.ReadString(hc, axis, "myString2D", 0, 1, -1, -1, max_
length, sp.SYNCHRONOUS, True)
for item in string_list:
   print(item)
```

5.4.7 ReadStruct

Description

The function retrieves Struct values from ACSPL+ Struct variables or arrays.

Syntax

ReadStruct(handle, nBuf, var, from, to, valuesSize, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Number of program buffer for local struct variable or GeneralDefinition.ACSC_NONE (-1) for global struct variable.
var	String that contains a name of the variable.
From, to	Index range.
valuesSize	Size of the buffer to which the data will be copied
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

ctypes.c_voip_p - cast to the desired object after call

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.4.8 WriteStruct

Description

The function writes struct values to ACSPL+ struct variables or arrays.

Syntax

WriteStruct(handle, nBuf, var, from, to, values, valuesSize, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Number of program buffer for local struct variable or GeneralDefinition.ACSC_NONE (-1) for global struct variable.
var	String that containing the name of the variable.
From, to	Index range.
valuesSize	Size of the buffer to which the data will be copied
values	List that contains the data to be written to the buffer. Need to be list of object inherited from ctypes.Structure items.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.5 Load/Upload Data To/From Controller Functions

5.5.1 LoadDataToController

Description

The function writes value(s) from text file to SPiiPlus controller (variable or file).

Syntax

LoadDataToController(handle, dest, destName, from1, to1, from2, to2, srcFileName, srcNumFormat, bTranspose, wait, failure_check=False)

Arguments

handle	Communication handle.
dest	Number of program buffer for local variable, GeneralDefinition.ACSC_NONE for global and standard variable. GeneralDefinition.ACSC_FILE for loading directly to file on flash memory (only arrays can be written directly into controller files).
destName	String that contains name of the Variable or File
from1, to1	Index range (first dimension)
From2, to2	Index range (second dimension)
srcFileName	Filename (including path) of the source text data file
srcNumFormat	Format of number(s) in source file. Use: GeneralDefinition.ACSC_INT_BINARY for integers, and GeneralDefinition.ACSC_REAL_BINARY for real
bTranspose	If TRUE, then the array will be transposed before being loaded; otherwise, this parameter has no affect.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

Comments

The function writes to a specified variable (scalar/array) or directly to a binary file on the controller's flash memory. The variable can be an ACSPL+ variable, user global or user local.

The input file (stored by the srcFileName argument) must be in ANSI format, otherwise error 168, ACSC_INVALID_FILE_FORMAT, is returned.

ACSPL+ and user global variables have global scope. Therefore, dest must be ACSC_NONE (-1) for these classes of variables. User local variable exists only within a buffer. The buffer number must be specified for user local variable.

If dest is ACSC_NONE (-1) and there is no global variable with the name specified by destName, it would be defined. Arrays will be defined with dimensions (to1+1, to2+1).

If performing loading directly to a file, from1, to1, from2 and to2 are meaningless. If the variable is scalar, the from1, to1, from2, and to2 arguments must be ACSC_NONE (-1).

The function writes the value from the file specified by srcFileName to the variable specified by Name. If the variable is a one-dimensional array, from1, to1 must specify the index range and the from2, to2 must be ACSC_NONE (-1). The text file, stored by the srcFileName argument, must contain to1 to from1+1 values, at least. The function writes the values to the specified variable from the from1 index to the to1 index inclusively. If the variable is a two-dimensional array, from1, to1 must specify the index range of the first dimension and from2, to2 must specify the index range of the second dimension. The text file, stored by the srcFileName argument, must contain ((to1- from1+1) x (to2 - from2+1)) values, at least; otherwise, an error will occur. The function uses the text file as follows: 1. The function retrieves the to2 - from2+1 values and writes them to row from1 of the specified controller variable 2. Then retrieves next to2 - from2+1 values and writes them to row from1+1 of the specified controller variable, etc.

file as follows: 1. The function retrieves the to2 - from2+1 values and writes them to row from1 of the specified controller variable 2. Then retrieves next to2 - from2+1 values and writes them to row from1+1 of the specified controller variable, etc.

The function uses the text file as follows:

- The function retrieves the to2 - from2+1 values and writes them to row from1 of the specified controller variable.
- Then retrieves next to2 - from2+1 values and writes them to row from1 +1 of the specified controller variable, etc.

If bTranspose is TRUE, the function actions are inverted. It takes to1 - from1 +1 values and writes them to column from2 of the specified controller variable, then retrieves next to1 - from1 +1 values and writes them to column from2+1 of the specified controller variable, etc.

The text file is processed line-by-line; any characters except numbers, dots, commas and exponent 'e' are translated as separators between the numbers. A line that starts with no digits is considered as comment and ignored.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.



The flash memory is rated to support about 100,000 write operations.
ACS recommends against frequent, repeated writing to the flash memory.

5.5.2 UploadDataFromController

Description

This method writes value(s) from SPiiPlus controller (variable or file) to text file.

Syntax

UploadDataFromController(handle, src, srcName, srcNumFormat, from1, to1, from2, to2, destFileName, destNumFormat, bTranspose, wait, failure_check=False)

Arguments

handle	Communication handle.
src	Number of program buffer for local variable, GeneralDefinition.ACSC_NONE for global and standard variable. GeneralDefinition.ACSC_FILE for loading directly to file on flash memory (only arrays can be written directly into controller files).
srcName	String that contains name of the Variable or File
srcNumFormat	Format of number(s) in source file. Use: GeneralDefinition.ACSC_INT_BINARY for integers, and GeneralDefinition.ACSC_REAL_BINARY for real
from1, to1	Index range (first dimension)
From2, to2	Index range (second dimension)
destFileName	Filename (including path) of the destination text data file
destNumFormat	Format of number(s) in destination file. Use: GeneralDefinition.ACSC_INT_BINARY for integers, and GeneralDefinition.ACSC_REAL_BINARY for real
bTranspose	If TRUE, then the array will be transposed before being loaded; otherwise, this parameter has no affect.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

None.

Comments

The function writes data to the specified file from a specified variable (scalar/array) or directly from a binary file in the controller's flash memory.
The variable can be a standard ASCPL+ variable, user global or user local. The ASCPL+ and user global

variables have global scope. In this case, the src argument has to be ACSC_NONE (-1) for these classes of variables. User local variable exists only within a buffer. The buffer number has to be specified for user local variable. If the variable (or file) with the name specified by srcName does not exist, an error occurs. If loading directly from the file, from1, to1, from2 and to2 are meaningless. If the variable is a scalar, all from1, to1, from2 and to2 values must be ACSC_NONE (-1). The function writes the value from variable specified by srcName to the file specified by destFileName. If the variable is a one-dimensional array, From1, To1 must specify the index range, and from2, to2 must be ACSC_NONE (-1). The function writes the values from the specified variable from the From1 value to the to1 value, inclusively, to the file specified by destFileName. If the variable is a two-dimensional array, from1, to1 must specify the index range of the first dimension, and from2, to2 must specify the index range of the second dimension. The function uses the variable as follows: first, the function retrieves the to2 - from2 +1 values from the row specified in From1 and writes them to the file specified by destFileName. It then retrieves to2 - from2 +1 values from the From1+1 row and writes them, and so forth. If destFileName is TRUE, the function actions are inverted. It takes To1-From1+1 values from the row specified by from2 and writes them to first column of the destination file. Then retrieves the next to1- from1 values from row from2 +1 and writes them to the next column of the file. The destination file's format can be determined by string specified by destFileName. This string will be used as argument in *printf function.

If Wait is a SYNCHRONOUS object, the function returns when the controller response is received.
If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result.
If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

5.6 Multiple Thread Synchronization Functions

5.6.1 CaptureComm

Description

The function captures a communication channel.

Syntax

CaptureComm(handle, failure_check=False)

Arguments

handle	Communication handle.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

None.

Comments

The function captures the communication handle for the calling thread and prevents access to this communication handle from other threads.
If one thread captures the communication handle and another thread calls one of functions using

the same handle, the second thread will be delayed until the first thread executes ReleaseComm. The function provides ability to execute a sequence of functions without risk of intervention from other threads.

5.6.2 ReleaseComm

Description

The function releases a communication channel.

Syntax

ReleaseComm(handle, failure_check=False)

Arguments

handle	Communication handle.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

None.

Comments

The function releases the communication handle captured by CaptureComm and allows other threads to communicate through the channel.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
def EnableAxis(hc):
    print("In second thread")
    sp.Enable(hc, 0, sp.SYNCHRONOUS, True)
    print("After Enable function")

sp.Disable(hc, 0, sp.SYNCHRONOUS, True)
sp.CaptureComm(hc, True)
myThread = threading.Thread(target=EnableAxis, args=(hc,))
myThread.start()
time.sleep(0.25)
command = "?MST0.0\r"
# reply is 0 means the axis is diabled
reply = int(sp.Transaction(hc, command, len(command), 256,
sp.SYNCHRONOUS, True).strip())
# Axis 0 should be enabled after this line
sp.ReleaseComm(hc, True)
```

5.7 History Buffer Management Functions

5.7.1 OpenHistoryBuffer

Description

The function opens a history buffer.

Syntax

OpenHistoryBuffer(handle, size)

Arguments

handle	Communication handle.
size	Required size of the buffer in bytes.

Return Value

Get extended error information by calling GetLastError.

None.

Comments

The function allocates a history buffer that stores all commands sent to the controller and all responses and unsolicited messages received from the controller.
Only one history buffer can be open for each communication handle.
The buffer works as a cyclic buffer. When the amount of the stored data exceeds the buffer size, the newly stored data overwrites the earliest data in the buffer.

5.7.2 CloseHistoryBuffer

Description

The function closes the history buffer and discards all stored history.

Syntax

CloseHistoryBuffer(handle, failure_check=False)

Arguments

handle	Communication handle.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

None.

Comments

The function closes the history buffer and releases the used memory. All information stored in the buffer is discarded.

5.7.3 GetHistory

Description

The function retrieves the contents of the history buffer.

Syntax

GetHistory(handle, count, bClear, failure_check=False)

Arguments

handle	Communication handle.
count	Size of the buffer in bytes.
bClear	If TRUE, the function clears contents of the history buffer. If FALSE, the history buffer content is not cleared.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

numpy.chararray

This array will return the data in bytes.

Get extended error information by calling GetLastError.

Comments

The function retrieves the communication history from the history buffer and stores it in the returned buffer.
The communication history includes all commands sent to the controller and all responses and unsolicited messages received from the controller. The amount of received data does not exceed the size of the history buffer. The history buffer works as a cyclic buffer: when the amount of the stored data exceeds the buffer size, the newly stored data overwrites the earliest data in the buffer. Therefore, as a rule, the retrieved communication history includes only the recently sent commands and receives responses and unsolicited messages. The depth of the retrieved history depends on the history buffer size.
The history data is retrieved in historical order, i.e. the earliest message is stored at the beginning of buf. The first retrieved message in buf can be incomplete, because of being partially overwritten in the history buffer.
If the size of the buf is less than the size of the history buffer, only the most recent part of the stored history is retrieved.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
sp.OpenHistoryBuffer(hc, 10000)
command = "#SI\r"
sp.Transaction(hc, command, len(command), 256, sp.SYNCHRONOUS, True)
# return result in byte array
```

```
reply = sp.GetHistory(hc, 10000, True, True)
sp.CloseComm(hc, True)
```

5.8 Unsolicited Messages Buffer Management Functions

5.8.1 OpenMessageBuffer

Description

The function opens an unsolicited message buffer

Syntax

OpenMessageBuffer(handle, size)

Arguments

handle	Communication handle.
size	Required size of the buffer in bytes.

Return Value

Get extended error information by calling GetLastError.

None.

Comments

The function allocates a buffer that stores unsolicited messages from the controller.
Unsolicited messages are messages that the controller sends on its own initiative and not as a response to command. For example, the DISP command in an ACSPL+ program forces the controller to send an unsolicited message.
Only one message buffer can be open for each communication handle.
If the message buffer has been open, the library separates unsolicited messages from the controller responses and stores them in the message buffer. In this case, the GetMessage function retrieves unsolicited messages. If the user application does not call GetMessage and uses only Transaction and Command, the unsolicited messages are discarded. The message buffer works as a FIFO buffer: GetMessage extracts the earliest message stored in the buffer. If GetMessage extracts the messages slower than the controller produces them, buffer overflow can occur, and some messages will be lost. Generally, the greater the buffer, the less likely is buffer overflow to occur.

5.8.2 GetMessage

Description

The function retrieves unsolicited messages from the buffer. This function only works if you setup a buffer using OpenMessageBuffer

Syntax

GetMessage(handle, count, bClear, failure_check=False)

Arguments

handle	Communication handle.
count	Size of the buffer in bytes.
bClear	If TRUE, the function clears the contents of the unsolicited messages buffer after retrieving the message. If FALSE, the unsolicited messages buffer is not cleared.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

String Type.

Returns a string containing the unsolicited message data.

Comments

The function retrieves all stored unsolicited messages from the message buffer.
The function always returns immediately. If no, unsolicited message is received, the function returns empty string.
Parameter count specifies the buffer size. If a received message contains more than count characters, the function transfers to buffer only count character.
The remaining characters of the message are removed from the message buffer. If the count is equal to or more than the length of the message, the function transfers the whole message to buffer and assigns variable Received with a number of characters that were actually transferred.

5.8.3 GetSingleMessage

Description

The function retrieves a single unsolicited message from the buffer. This function only works if you setup a buffer using OpenMessageBuffer. If there is no message in the buffer, the function waits until the message arrives, or timeout expires.

Syntax

GetSingleMessage(handle, count, timeout, failure_check=False)

Arguments

handle	Communication handle.
count	Size of the buffer in bytes.
timeout	Timeout in milliseconds.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Get extended error information by calling GetLastError.

String Type.

Returns a string containing the unsolicited message data.

Comments

If the timeout expires, the function exits with the ErrorCodes.ACSC_TIMEOUT error.

5.9 Log File Management Methods

5.9.1 OpenLogFile

Description

The function opens a log file.

Syntax

OpenLogFile(handle, fileName, failure_check=False)

Arguments

handle	Communication handle.
fileName	String containing the name or path and name of the log file.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function opens a binary file that stores all communication history.
Only one log file can be open for each communication handle.
If the log file has been opened, the library writes all incoming and outgoing messages to the specified file. The messages are written to the file in binary format, i.e., exactly as they are received and sent, including all service bytes.
Unlike the history buffer, the log file cannot be read within the library. The main usage of the log file is for debug purposes.

5.9.2 CloseLogFile

Description

The function closes the log file.

Syntax

CloseLogFile(handle, failure_check=False)

Arguments

handle	Communication handle.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

An application should always call the **CloseLogFile** function before it exits, otherwise the data written to the file might be lost.

5.9.3 WriteLogFile

Description

The function writes to log file.

Syntax

WriteLogFile(handle, buf, count, failure_check=False)

Arguments

handle	Communication handle.
buf	The string to be written to log file.
count	Length of the input string.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes data from a buffer to log file.



The log file must have previously been opened using OpenLogFile.

5.9.4 FlushLogFile

Description

The method flushes the log to file.

Syntax

FlushLogFile(filename, failure_check=False)

Arguments

filename	String specifying the text file into which the buffer is to be flushed.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If Continuous Log is active, the function will fail.

5.9.5 GetLogData

Description

The function is used to retrieve the data of firmware log.

Syntax

GetLogData(handle, count, wait, failure_check=False)

Arguments

handle	Communication handle.
count	Length of the input string.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

String containing the log data

Get extended error information by calling GetLastError.

5.10 SPiiPlusSC Log File Management Functions

5.10.1 OpenSCLogFile

Description

The function opens the SPiiPlusSC log file.

Syntax

OpenSCLogFile(handle, fileName, failure_check=False)

Arguments

handle	Communication handle.
filename	String specifying the text file into which the buffer is to be flushed.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function opens a binary file that stores all SPiiPlusSC log history.
The messages are written to the file in binary format, i.e., exactly as they are received and sent, including all service bytes.
The main use of the log file is for debug purposes.

5.10.2 CloseSCLogFile

Description

The function closes the SPiiPlusSC log file.

Syntax

CloseSCLogFile(handle, failure_check=False)

Arguments

handle	Communication handle.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

An application must always call the **CloseSCLogFile** before it exits; otherwise, the data written to the file might be lost.

5.10.3 WriteSCLogFile

Description

The function writes to the SPiiPlusSC log file.

Syntax

WriteSCLogFile(handle, buf, count, failure_check=False)

Arguments

handle	Communication handle
buf	The string to be written to the log file
count	Length of the input string
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes data from a buffer to a log file.

The log file must have been opened previously using OpenSCLogFile

5.10.4 FlushSCLogFile

Description

The function enables flushing the SPiiPlusSC internal binary buffer to a specified text file.

Syntax

FlushSCLogFile(filename, bClear, failure_check=False)

Arguments

filename	String specifying the text file into which the buffer is to be flushed.
bClear	Can be TRUE or FALSE: TRUE - the function clears contents of the log buffer FALSE - the log buffer content is not cleared

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If Continuous Log is active, the function will fail.

5.11 System Configuration Functions

5.11.1 SetConf

Description

The function writes system configuration data.

Syntax

SetConf(handle, key, index, value, wait, failure_check=False)

Arguments

handle	Communication handle.
key	Configuration key, that specifies the configured feature. Assigns value of key argument in ACSPL+ SETCONF function.
index	Specifies corresponding axis or buffer number. Assigns value of index argument in ACSPL+ SETCONF function.
value	Value to write to specified key. Assigns value of value argument in ACSPL+ SETCONF function.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes system configuration data. The Key parameter specifies the feature number and the Index parameter defines axis or buffer to which it should be applied. Use ConfigKey.ACSC_CONF_XXX constants in the value field. For complete details of system configuration see the description of the SETCONF function in the *SPiiPlus ACSPL+ Programmer's Guide*.

5.11.2 GetConf

Description

The method reads system configuration data.

Syntax

GetConf(handle, key, index, wait, failure_check=False)

Arguments

handle	Communication handle.
key	Configuration key, that specifies the configured feature. Assigns value of key argument in ACSPL+ **SETCONF** function.
index	Specifies corresponding axis or buffer number. Assigns value of index argument in ACSPL+ **SETCONF** function.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - system configuration data.

Comments

The key specifies the feature number and index defines axis or buffer to which it should be applied. For detailed Description of system configuration see *SPiiPlus Command & Variable Reference Guide*.

Get extended error information by calling GetLastError.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Writes system configuration data of axis 1 to 1 with key 204
sp.SetConf(hc, sp.ConfigKey.ACSC_CONF_MFLAGS9_KEY,
           sp.Axis.ACSC_AXIS_1, 1, sp.SYNCHRONOUS, True)
```

```
# reply should be 1.0 float
reply = sp.GetConf(hc, sp.ConfigKey.ACSC_CONF_MFLAGS9_KEY,
                   sp.Axis.ACSC_AXIS_1, sp.SYNCHRONOUS, True)
```

5.11.3 GetVolatileMemoryUsage

Description

The function retrieves the volatile memory load in percentage.

Syntax

GetVolatileMemoryUsage(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - percentage of the volatile memory load

Get extended error information by calling GetLastError.

5.11.4 GetVolatileMemoryTotal

Description

The function retrieves the amount of total volatile memory in bytes.

Syntax

GetVolatileMemoryTotal(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - total amount of volatile memory in bytes

Get extended error information by calling GetLastError.

5.11.5 GetVolatileMemoryFree

Description

The function retrieves the amount of free volatile memory in bytes.

Syntax

GetVolatileMemoryFree(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - quantity of free volatile memory in bytes

Get extended error information by calling GetLastError.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
total = sp.GetVolatileMemoryTotal(hc, sp.SYNCHRONOUS, failure_check=True)
free = sp.GetVolatileMemoryFree(hc, sp.SYNCHRONOUS, failure_check=True)
usg = sp.GetVolatileMemoryUsage(hc, sp.SYNCHRONOUS, failure_check=True)
# the ratio should be close to 1
if (free + usg/(100)*total)/total > 0.99 and 1.01 > (free + usg/
(100)*total)/total:
   print("The memory results create proper equation")
else:
   print("The memory results are not accurate")
```

5.11.6 GetNonVolatileMemoryUsage

Description

The function retrieves the percentage of the non-volatile memory load.

Syntax

GetNonVolatileMemoryUsage(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - percentage of non-volatile memory in bytes

Get extended error information by calling GetLastError.

5.11.7 GetNonVolatileMemoryTotal

Description

The function retrieves the amount of total non-volatile memory.

Syntax

GetNonVolatileMemoryTotal(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - total non-volatile memory in bytes

Get extended error information by calling GetLastError.

5.11.8 GetNonVolatileMemoryFree

Description

The function retrieves the amount of free non-volatile memory in bytes.

Syntax

GetNonVolatileMemoryFree(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - free non-volatile memory in bytes

Get extended error information by calling GetLastError.

5.11.9 SysInfo

Description

The function returns certain system information based on the argument that is specified.

Syntax

SysInfo(handle, key, wait, failure_check=False)

Arguments

handle	Communication handle.
key	Configuration Key, specifies the configured feature. Assigns value of key argument in ACSPL+ **SYSINFO** function.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type

5.12 Setting and Reading Motion Parameters Functions

5.12.1 SetVelocity

Description

The function defines a value of motion velocity.

Syntax

SetVelocity(handle, axis, velocity, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
velocity	The value specifies required motion velocity. The value will be used in the subsequent motions except for the master-slave motions and the motions activated with the MotionFlags.ACSC_AMF_VELOCITY flag.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Comments

The function writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The function affects the motions initiated after the function call. The function has no effect on any motion that was started or planned before the function call. To change velocity of an executed or planned motion, use the SetVelocityImm function.

The function has no effect on the master-slave motions and the motions activated with the MotionFlags.ACSC_ AMF_VELOCITY flag.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Set velocity of axis 0 to 3000
sp.SetVelocity(hc, 0, 3000, sp.SYNCHRONOUS, True)
```

5.12.2 GetVelocity

Description

The function retrieves a value of motion velocity.

Syntax

GetVelocity(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type -retrieves the value of the motion velocity

Get extended error information by calling GetLastError.

Comments

The retrieved value is the value defined by a previous call of SetVelocity or the default value if the method was not previously called.
If the method fails, the Error object is filled with the Error Description.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# receives velocity of axis 0
Velocity = sp.GetVelocity(hc, 0, sp.SYNCHRONOUS, True)
```

5.12.3 SetAcceleration

Description

The function defines a value of motion acceleration.

Syntax

SetAcceleration(handle, axis, acceleration, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

acceleration	The value specifies required motion acceleration. The value will be used in the subsequent motions except the master-slave motions.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects the motions initiated after the function call. The function has no effect on any motion that was started or planned before the function call. To change acceleration of an executed or planned motion, use the SetAccelerationImm function.
The function has no effect on the master-slave motions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Set acceleration of axis 0 to 1000
sp.SetAcceleration(hc, 0, 1000, sp.SYNCHRONOUS, True)
```

5.12.4 GetAcceleration

Description

The function retrieves a value of motion acceleration.

Syntax

GetAcceleration(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - retrieves the value of the motion acceleration

Get extended error information by calling GetLastError.

Comments

The function retrieves the value of the motion acceleration. The retrieved value is a value defined by a previous call of the SetAcceleration function, or the default value if the function was not called before.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.5 SetDeceleration

Description

The function defines a value of motion deceleration.

Syntax

SetDeceleration(handle, axis, deceleration, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
deceleration	The value specifies a required motion deceleration. The value will be used in the subsequent motions except the master-slave motions.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects the motions initiated after the function call. The function has no effect on any motion that was started or planned before the function call. To change deceleration of an executed or planned motion, use the SetDeceleration function.
The function has no effect on master-slave motions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Get acceleration of axis 0
acc = sp.GetAcceleration(hc, 0, sp.SYNCHRONOUS, True)
```

5.12.6 GetDeceleration

Description

The function retrieves a value of motion deceleration.

Syntax

GetDeceleration(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - The method retrieves the value of the motion deceleration.

Get extended error information by calling GetLastError.

Comments

The function retrieves the value of the motion deceleration. The retrieved value is a value defined by a previous call of the SetDeceleration function, or the default value if the function was not previously called.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.7 SetJerk

Description

The function defines a value of motion jerk.

Syntax

SetJerk(handle, axis, jerk, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
jerk	The value specifies a required motion jerk. The value will be used in the subsequent motions except for the master-slave motions.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects the motions initiated after the function call. The function has no effect on any motion that was started or planned before the function call. To change jerk of an executed or planned motion, use the SetJerkImm function.
The function has no effect on the master-slave motions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.8 GetJerk

Description

The function retrieves a value of motion jerk.

Syntax

GetJerk(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - retrieves the value of the motion jerk

Comments

The function retrieves the value of the motion jerk. The retrieved value is a value defined by a previous call of the SetJerk function, or the default value if the function was not called before.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.9 SetKillDeceleration

Description

The function defines a value of motion kill deceleration.

Syntax

SetKillDeceleration(handle, axis, killDeceleration, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_ AXIS_1 to axis 1, etc.
killDeceleration	The value specifies a required motion kill deceleration. The value will be used in the subsequent motions.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects the motions initiated after the function call. The function has no effect on any motion that was started or planned before the function call.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.10 GetKillDeceleration

Description

The function retrieves a value of motion kill deceleration.

Syntax

GetKillDeceleration(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type - retrieves the value of the motion kill deceleration

Comments

The function retrieves the value of the motion kill deceleration. The retrieved value is a value defined by a previous call of the SetKillDeceleration function, or the default value if the function was not called before.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.11 SetVelocityImm

Description

The function defines a value of motion velocity. Unlike SetVelocity, the function has immediate effect on any executed or planned motion.

Syntax

SetVelocityImm(handle, axis, velocity, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
velocity	The value specifies required motion velocity.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects:

- The currently executed motion. The controller provides a smooth transition from the instant current velocity to the specified new value.

- The waiting motions that were planned before the function call.
- The motions that will be commanded after the function call.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.12 SetAccelerationImm

Description

The function defines a value of motion acceleration. Unlike SetAcceleration, the function has immediate effect on any executed and planned motion.

Syntax

SetAccelerationImm(handle, axis, acceleration, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
acceleration	The value specifies required motion acceleration.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects:

- The currently executed motion. The controller provides a smooth transition from the instant current velocity to the specified new value.
- The waiting motions that were planned before the function call.

> The motions that will be commanded after the function call.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.13 SetDecelerationImm

Description

The function defines a value of motion deceleration. Unlike SetDeceleration, the function has immediate effect on any executed and planned motion.

Syntax

SetDecelerationImm(handle, axis, deceleration, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
deceleration	The value specifies required motion deceleration.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects:

> The currently executed motion. The controller provides a smooth transition from the instant current velocity to the specified new value.

> The waiting motions that were planned before the function call.

> The motions that will be commanded after the function call.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.14 SetJerkImm

Description

The function defines a value of motion jerk. Unlike SetJerk, the function has immediate effect on any executed and planned motion.

Syntax

SetJerkImm(handle, axis, jerk, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
jerk	The value specifies required motion jerk.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects:

> The currently executed motion. The controller provides a smooth transition from the instant current velocity to the specified new value.

> The waiting motions that were planned before the function call.

> The motions that will be commanded after the function call.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.15 SetKillDecelerationImm

Description

The function defines a value of motion kill deceleration. Unlike SetKillDeceleration, the function has an immediate effect on any executed and planned motion.

Syntax

SetKillDecelerationImm(handle, axis, killDeceleration, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
killDeceleration	The value specifies the required motion deceleration.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the controller.
One value can be specified for each axis.
A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.
The function affects:

- The currently executed motion. The controller provides a smooth transition from the instant current velocity to the specified new value.
- The waiting motions that were planned before the function call.
- The motions that will be commanded after the function call.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.16 SetFPosition

Description

The function defines a value of motion kill deceleration. Unlike SetKillDeceleration, the function has an immediate effect on any executed and planned motion.

Syntax

SetFPosition(handle, axis, fPosition, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
fPosition	The value specifies the current value of feedback position.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function assigns a current value to the feedback position. No physical motion occurs. The motor remains in the same position; only the internal controller offsets are adjusted so that the periodic calculation of the feedback position will provide the required results.
For more information see the explanation of the **SET** command in the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.17 GetFPosition

Description

The function retrieves an instant value of the motor feedback position.

Syntax

GetFPosition(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float - retrieves an instant value of the motor feedback position

Get extended error information by calling GetLastError.

Comments

The function retrieves an instant value of the motor feedback position. The feedback position is a measured position of the motor transferred to user units.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.18 SetRPosition

Description

The method assigns a current value of reference position.

Syntax

SetRPosition(handle, axis, rPosition, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
rPosition	The value specifies the current value of reference position.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The method assigns a current value to the reference position. No physical motion occurs. The motor remains in the same position; only the internal controller offsets are adjusted so that the periodic calculation of the reference position will provide the required results.
For more information see explanation of the **SET** command in the *SPiiPlus ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the Error Description.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Set RPOS(0) to -50
sp.SetRPosition(hc, 0, -50, sp.SYNCHRONOUS, True)
```

5.12.19 GetRPosition

Description

The function retrieves an instant value of the motor reference position.

Syntax

GetRPosition(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float - retrieves the current value of the motor reference position

Get extended error information by calling GetLastError.

Comments

The function retrieves an instant value of the motor feedback position. The reference position is a value calculated by the controller as a reference for the motor.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Get RPOS(0)
rpos = sp.GetRPosition(hc, 0, sp.SYNCHRONOUS, True)
```

5.12.20 GetFVelocity

Description

The function retrieves an instant value of the motor feedback velocity. Unlike , the function retrieves the actually measured velocity and not the required value.

Syntax

GetFVelocity(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float

Get extended error information by calling GetLastError.

Comments

The function retrieves an instant value of the motor feedback velocity. The feedback velocity is a measured velocity of the motor transferred to user units.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.12.21 GetRVelocity

Description

The method retrieves an instant value of the motor reference velocity.

Syntax

GetRVelocity(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float - retrieves the current value of the motor reference velocity

Get extended error information by calling GetLastError.

Comments

The function retrieves an instant value of the motor reference velocity. The reference velocity is a value calculated by the controller in the process of motion generation.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.13 Axis/Motor Management Functions

5.13.1 CommutExt

Description

This function initiates a motor commutation.

Syntax

CommutExt(handle, axis, current, settle, slope, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
current	Desired excitation current in percentage 0-100, GeneralDefinition.ACSC_NONE for default value.
settle	Specifies the time it takes for auto commutation to settle, in milliseconds. GeneralDefinition.ACSC_NONE for the default value of 500ms.
slope	Specifies the time it takes for the current to rise to the desired value, GeneralDefinition.ACSC_NONE for default value.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function activates a motor. After the activation, the motor begins to follow the reference and physical motion is available.

Settle can only be set if Current is set. Similarly, Slope can only be set if Settle is set.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Commut axis 0 with default values
sp.CommutExt(hc,sp.Axis.ACSC_AXIS_0, sp.GeneralDefinition.ACSC_NONE,
             sp.GeneralDefinition.ACSC_NONE,
             sp.GeneralDefinition.ACSC_NONE, sp.SYNCHRONOUS, True)
```

5.13.2 Enable

Description

The function activates a motor.

Syntax

Enable(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function activates a motor. After the activation, the motor begins to follow the reference and physical motion is available.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Enable axis 0
sp.Enable(hc, 0, sp.SYNCHRONOUS, True)
```

5.13.3 EnableM

Description

The function activates several motors.

Syntax

EnableM(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function activates several motors. After the activation, the motors begin to follow the corresponding reference and physical motions for the specified motors are available.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Enable axes 0, 2 and 4
sp.EnableM(hc, [0, 2, 4, -1], sp.SYNCHRONOUS, True)
```

5.13.4 Disable

Description

The function shuts off a motor.

Syntax

Disable(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function shuts off a motor. After shutting off the motor cannot follow the reference and remains at idle.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Disable axis 0
sp.Disable(hc, 0, sp.SYNCHRONOUS, True)
```

5.13.5 DisableAll

Description

The function shuts off all motors.

Syntax

DisableAll(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function shuts off all motors. After the shutting off none of motors can follow the corresponding, reference and all motors remain idle.
If no motors are currently enabled, the function has no effect.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# Disable all axes
sp.DisableAll(hc, sp.SYNCHRONOUS, True)
```

5.13.6 DisableExt

Description

The function shuts off a motor and defines the disable reason.

Syntax

DisableExt(handle, axis, reason, wait, failure_check=False)

Arguments

handle	U
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
reason	Integer number that defines the reason of disable. The specified value is stored in the MERR variable in the controller and so modifies the state of the disabled motor.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function shuts off a motor. After shutting off the motor cannot follow the reference and remains at idle.
If Reason specifies one of the available motor termination codes, the state of the disabled motor will be identical to the state of the motor disabled for the corresponding fault. This provides an enhanced implementation of user-defined fault response.
If the second parameter specifies an arbitrary number, the motor state will be displayed as "Kill/disable reason: - customer code. This provides ability to separate different DISABLE commands in the application.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.13.7 DisableM

Description

The function shuts off several specified motors.

Syntax

DisableM(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function shuts off several motors. After the shutting off, the motors cannot follow the corresponding reference and remain idle.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.13.8 Group

Description

The function creates a coordinate system for a multi-axis motion.

Syntax

Group(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function creates a coordinate system for a multi-axis motion. The first element of the axes array specifies the leading axis. The motion parameters of the leading axis become the default motion parameters for the group.
An axis can belong to only one group at a time. If the application requires restructuring the axes, it must split the existing group and only then create the new one. To split the existing group, use **Split** function. To split all existing groups, use the **SplitAll** function

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
axes = [0, 1, 2, -1]
# Group axes 0, 1, 2
sp.Group(hc, axes, sp.SYNCHRONOUS,True)
```

5.13.9 Split

Description

The function breaks down an axis group created before.

Syntax

Split(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function breaks down an axis group created before by the Group function. The axes parameter must specify the same axes as for the **Group** function that created the group. After the splitting up the group no longer exists.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
axes = [0, 1, 2, -1]
# Split axes 0, 1, 2
sp.Split(hc, axes, sp.SYNCHRONOUS,True)
```

5.13.10 SplitAll

Description

The function breaks down all axis groups created before.

Syntax

SplitAll(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function breaks down all axis groups created before by the Group function.
The application may call this function to ensure that no axes are grouped. If no groups are currently existed, the function has no effect.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.14 Motion Management Functions

5.14.1 Go

Description

The function starts up a motion waiting in the specified motion queue.

Syntax

Go(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

A motion that was planned with MotionFlags.ACSC_AMF_WAIT flag does not start until the Go function is executed. Being planned, a motion waits in the appropriate motion queue.
Each axis has a separate motion queue.
A single-axis motion waits in the motion queue of the corresponding axis. A multi-axis motion waits in the motion queue of the leading axis. The leading axis is an axis specified first in the motion command. The Go function initiates the motion waiting in the motion queue of the specified axis. If no motion waits in the motion queue, the function has no effect.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
sp.SetFPosition(hc, 0, 0, sp.SYNCHRONOUS, True)
# set to-point motion with wait condition to start on go command
sp.ToPoint(hc, sp.MotionFlags.ACSC_AMF_WAIT, 0, 10000,
           sp.SYNCHRONOUS, True)
fpos = sp.GetFPosition(hc, 0, sp.SYNCHRONOUS, True)
if fpos != 0:
   raise "Fpos was not set to 0"
# start to-point motion
sp.Go(hc, 0, sp.SYNCHRONOUS, True)
```

5.14.2 GoM

Description

The function synchronously starts up several motions that are waiting in the specified motion queues.

Syntax

GoM(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

A motion that was planned with MotionFlags.ACSC_AMF_WAIT flag does not start until the GoM function is executed. After being planned, a motion waits in the appropriate motion queue.
Each axis has a separate motion queue. A single-axis motion waits in the motion queue of the corresponding axis. A multi-axis motion waits in the motion queue of the leading axis. The leading axis is an axis specified first in the motion command.
The GoM function initiates the motions waiting in the motion queues of the specified axes. If no motion waits in one or more motion queues, the corresponding axes are not affected.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.14.3 Halt

Description

The function terminates a motion using the full deceleration profile.

Syntax

Halt(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates the executed motion that involves the specified axis. The terminated motion can be either single-axis or multi-axis. Any other motion waiting in the corresponding motion queue is discarded and will not be executed.
If no executed motion involves the specified axis, the function has no effect.
The terminated motion finishes using the full third-order deceleration profile and the motion deceleration value.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# halt axis 0
sp.Halt(hc, sp.Axis.ACSC_AXIS_0, sp.SYNCHRONOUS, True)
```

5.14.4 HaltM

Description

The function terminates a motion using the full deceleration profile.

Syntax

HaltM(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates all executed motions that involve the specified axes. The terminated motions can be either single-axis or multi-axis. All other motions waiting in the corresponding motion queues are discarded and will not be executed.
If no executed motion involves a specified axis, the function has no effect on the corresponding axis.
The terminated motions finish using the full third-order deceleration profile and the motion deceleration values.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# halt axis 0
axes = [0, 1, -1]
# halt axes 0 and 1
sp.Halt(hc, axes, sp.SYNCHRONOUS, True)
```

5.14.5 Kill

Description

The function Kill terminates a motion using reduced deceleration profile.

Syntax

Kill(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates the executed motion that involves the specified axis. The terminated motion can be either single-axis or multi-axis. Any other motion waiting in the corresponding motion queue is discarded and will not be executed.
If no executed motion involves the specified axis, the function has no effect.
The terminated motion finishes with the reduced second-order deceleration profile and the kill deceleration value.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.14.6 KillM

Description

The function terminates several motions using reduced deceleration profile.

Syntax

def KillM(handle, axes, wait, failure_check=False):

Arguments

handle	Communication handle.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates all executed motions that involve the specified axes. The terminated motions can be either single-axis or multi-axis. All other motions waiting in the corresponding motion queues are discarded and will not be executed.
If no executed motion involves a specified axis, the function has no effect on the corresponding axis.
The terminated motions finish with the reduced second-order deceleration profile and the kill deceleration values.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.14.7 KillAll

Description

The function terminates all currently executed motions.

Syntax

KillAll(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates all currently executed motions. Any other motion waiting in any motion queue is discarded and will not be executed.
If no motion is currently executed, the function has no effect.
The terminated motions finish with the reduced second-order deceleration profile and the kill deceleration values.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.14.8 KillExt

Description

The function terminates a motion using reduced deceleration profile and defines the kill reason.

Syntax

KillExt(handle, axis, reason, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
reason	Integer number that defines the reason of kill. The specified value is stored in the MERR variable in the controller and so modifies the state of the killed motor.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates the executed motion that involves the specified axis. The terminated motion can be either single-axis or multi-axis. Any other motion waiting in the corresponding motion queue is discarded and will not be executed.
If no executed motion involves the specified axis, the function has no effect.
The terminated motion finishes with the reduced second-order deceleration profile and the kill deceleration value
If Reason specifies one of the available motor termination codes, the state of the killed motor will be identical to the state of the motor killed for the corresponding fault. This provides an enhanced implementation of user-defined fault response.
If the second parameter specifies an arbitrary number, the motor state will be displayed as "Kill/disable reason: - customer code. This provides ability to separate different KILL commands in the application.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.14.9 Break

Description

The function terminates a motion immediately and provides a smooth transition to the next motion.

Syntax

Break(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates the executed motion that involves the specified axis only if the next motion is waiting in the corresponding motion queue. The terminated motion can be either single-axis or multi-axis.
If the motion queue contains no waiting motion, the break command is not executed immediately. The current motion continues instead until the next motion is planned to the same motion queue. Only then is the break command executed.
If no executed motion involves the specified axis, or the motion finishes before the next motion is planned, the function has no effect.
When executing the break command, the controller terminates the motion immediately without any deceleration profile. The controller builds instead a smooth third-order transition profile to the next motion.
Use caution when implementing the break command with a multi-axis motion, because the controller provides a smooth transition profile of the vector velocity. In a single-axis motion, this ensures a smooth axis velocity. However, in a multi-axis motion an axis velocity can change abruptly if the terminated and next motions are not tangent to the junction point. To avoid jerk, the terminated and next motion must be tangent or nearly tangent in the junction point.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# break motion of axis 0
sp.Break(hc, sp.Axis.ACSC_AXIS_0, sp.SYNCHRONOUS, True)
```

5.14.10 BreakM

Description

The function terminates a motion immediately and provides a smooth transition to the next motion.

Syntax

BreakM(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function terminates the executed motions that involve the specified axes. Only those motions are terminated that have the next motion waiting in the corresponding motion queue. The terminated motions can be either single-axis or multi-axis.
If a motion queue contains no waiting motion, the break command does not immediately affect the corresponding axis. The current motion continues instead until the next motion is planned to the same motion queue. Only then, the break command is executed.
If no executed motion involves the specified axis, or the corresponding motion finishes before the next motion is planned, the function does not affect the axis.
When executing the break command, the controller terminates the motion immediately without any deceleration profile. Instead, the controller builds a smooth third-order transition profile to the next motion.
Use caution when implementing the break command with a multi-axis motion, because the controller provides a smooth transition profile of the vector velocity. In a single-axis motion, this ensures a smooth axis velocity, but in a multi-axis motion, an axis velocity can change abruptly if the

terminated and next motions are not tangent in the junction point. To avoid jerk, the terminated and next motion must be tangent or nearly tangent in the junction point.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.15 Point-to-Point Motion Functions

5.15.1 ToPoint

Description

The function initiates a single-axis motion to the specified point.

Syntax

ToPoint(handle, flags, axis, point, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates a single-axis point-to-point motion.
The motion is executed using the required motion velocity and finishes with zero end velocity. The required motion velocity is the velocity specified by the previous call of the SetVelocity function or the default velocity if the function was not called.
To execute multi-axis point-to-point motion, use ToPointM. To execute motion with other motion velocity or non-zero end velocity, use **ToPoint** or ExtToPointM.
The controller response indicates that the command was accepted, and the motion was planned successfully. The function does not wait for the motion end. To wait for the motion end, use WaitMotionEnd function.
The motion builds the velocity profile using the required values of velocity, acceleration, deceleration, and jerk of the specified axis.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.15.2 ToPointM

Description

The function initiates a multi-axis motion to the specified point.

Syntax

ToPointM(handle, flags, axes, point, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function initiates a multi-axis point-to-point motion.
The motion is executed using the required motion velocity and finishes with zero end velocity. The required motion velocity is the velocity specified by the previous call of the SetVelocity function or the default velocity if the function was not called.
To execute single-axis point-to-point motion, use ToPoint. To execute motion with other motion velocity or non-zero end velocity, use ExtToPoint or ExtToPointM.
The controller response indicates that the command was accepted, and the motion was planned successfully. The function does not wait for the motion end. To wait for the motion end, use WaitMotionEnd function.
The motion builds the velocity profile using the required values of velocity, acceleration, deceleration, and jerk of the leading axis. The leading axis is the first axis in the axes array.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
axes = [1, 2, -1]
points = [20000, 20000]
# enable and commut the participating axes
sp.EnableM(hc, axes, sp.SYNCHRONOUS, True)
for axis in range(len(axes)-1):
   sp.WaitMotorEnabled(hc, axis, 1, 5000, True)
   sp.CommutExt(hc, axis, -1, -1, -1, sp.SYNCHRONOUS, True)
   sp.WaitMotorCommutated(hc, axis, 1, 5000, True)
for axis in range(4):
   sp.SetFPosition(hc, axis, 0, sp.SYNCHRONOUS, True)
# start multi axis motion
sp.ToPointM(hc, 0, axes, points, sp.SYNCHRONOUS, True)
```

5.15.3 ExtToPoint

Description

The function initiates a single-axis motion to the specified point using the specified velocity or end velocity.

Syntax

ExtToPoint(handle, flags, axis, point, velocity, endVelocity, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
point	Coordinate of the target point.
velocity	Motion velocity. The argument is used only if the MotionFlags.ACSC_AMF_VELOCITY flag is specified.
endVelocity	Velocity in the target point. The argument is used only if the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates a single-axis point-to-point motion.
If the MotionFlags.ACSC_AMF_VELOCITY flag is specified, the motion is executed using the velocity specified by the velocity argument. Otherwise, the required motion velocity is used. The required motion velocity is the velocity specified by the previous call of the SetVelocity function, or the default velocity if the function was not called.
If the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified, the motion velocity at the final point is

specified by the endVelocity argument. Otherwise, the motion velocity at the final point is zero.
To execute a multi-axis point-to-point motion with the specified velocity or end velocity, use ExtToPointM. To execute motion with default motion velocity and zero end velocity, use ExtToPoint or ToPointM.
The controller response indicates that the command was accepted, and the motion was planned successfully. The function does not wait for the motion end. To wait for the motion end, use WaitMotionEnd function.
The motion builds the velocity profile using the required values of acceleration, deceleration and jerk of the specified axis.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.15.4 ExtToPointM

Description

The function initiates a multi-axis motion to the specified point using the specified velocity or end velocity.

Syntax

ExtToPointM(handle, flags, axes, point, velocity, endVelocity, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
point	Coordinate of the target point.
velocity	Motion velocity. The argument is used only if the MotionFlags.ACSC_AMF_VELOCITY flag is specified.
endVelocity	Velocity in the target point. The argument is used only if the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function initiates a multi-axis point-to-point motion.
If the MotionFlags.ACSC_AMF_VELOCITY flag is specified, the motion is executed using the velocity specified by the Velocity argument. Otherwise, the required motion velocity is used. The required motion velocity is the velocity specified by the previous call of the SetVelocity function, or the default velocity if the function was not called.
If the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified, the motion velocity at the final point is specified by the endVelocity argument. Otherwise, the motion velocity at the final point is zero.
To execute a single-axis point-to-point motion with the specified velocity or end velocity, use ExtToPoint. To execute a motion with default motion velocity and zero end velocity, use ToPoint or ToPointM.
The controller response indicates that the command was accepted, and the motion was planned successfully. The function does not wait for the motion end. To wait for the motion end, use WaitMotionEnd function.
The motion builds the velocity profile using the required values of acceleration, deceleration, and jerk of the leading axis. The leading axis is the first axis in the axes array.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.16 Augmented Point-to-Point Motion

5.16.1 BoostedPointToPointMotion

Description

This function defines a motion profile using the MotionBoost Feature.

Syntax

def BoostedPointToPointMotion(handle, flags, axes, point, velocity, finalVelocity, time, fixedTime, motionDelay, wait, failure_check=False):

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT(/w) plan the motion but do not start it until the function GoM is executed. MotionFlags.ACSC_AMF_VELOCITY(/v) the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_EXT_DELAY_MOTION(/q) Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The switch requires an additional parameter that specifies the motion delay. MotionFlags.ACSC_AMF_TIME (/t) Minimum travel time in seconds, the calculated travel time will be at least the specified value. Incompatible with the MotionFlags.ACSC_AMF_FIXED_TIME switch. ACSC_AMF_FIXED_TIME (/d) Travel Time – specifies the exact travel time for the motion in seconds. All other considerations are ignored, which could cause a safety fault during motion execution. Incompatible with the ACSC_AMF_TIME switch. MotionFlags.ACSC_AMF_ENDVELOCITY The motion comes to the end point with the specified velocity User will enter final, nonzero velocity MotionFlags.ACSC_AMF_RELATIVE Relative motion, The value of the point coordinate is relative to the end point coordinate of the previous motion. MotionFlags.ACSC_AMF_LOCALCS Interpret entered coordinates according to the Local Coordinate System. With this switch, use 2 axes motion coordinate only (X,Y). Using 3 or more coordinates causes a runtime error. MotionFlags.ACSC_AMF_MAXIMUM Use maximum velocity under axis limits. With this suffix, no required velocity should be specified. The required velocity is calculated for each segment individually on the base of segment geometry and axis velocities (VEL values) of the involved axes. MotionFlags.ACSC_AMF_2 Use 20 kHz motion mode. Limited to at most 2 axes in a single function call.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.

velocity	If MotionFlags.ACSC_AMF_VELOCITYflag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
finalVelocity	If MotionFlags.ACSC_AMF_ENDVELOCITY flag was specified, this argument defines finalVelocity . finalVelocity specified the motion velocity at the final point Set this argument to GeneralDefinition.ACSC_NONE if not used.
time	If MotionFlags.ACSC_AMF_TIME flag was specified, this argument defines Time. Minimum travel time in seconds, The calculated travel time will be at least the specified value. Set this argument to GeneralDefinition.ACSC_NONEif not used.
fixedTime	If MotionFlags.ACSC_AMF_FIXED_TIME flag has been specified, this argument defines the fixedTime . specifies the exact travel time for the motion in seconds. Set this argument to GeneralDefinition.ACSC_NONEif not used.
motionDelay	If MotionFlags.ACSC_AMF_DELAY_MOTION flag has been specified, this argument defines the bit motion delay. Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles or 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Set this argument to GeneralDefinition.ACSC_NONEif not used
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

Supported from V3.12.

5.16.2 SmoothPointToPointMotion

Description

The SmoothPointToPointMotion command defines a multi-axes Point-to-Point motion and provides the positioning to specific target and looks like the ToPointM command. Unlike ToPointM, SmoothPointToPointMotion uses a 4th order motion profile.
If the axis is moving when the command is issued, the controller creates the motion and inserts it into the axis motion queue. The motion waits in the queue until all motions before it finish, and only then starts.
The SmoothPointToPointMotion command can be either a single axis or an axis group.
SmoothPointToPointMotion responds to KILL /HALT commands in the same fashion as do other motion commands.

Syntax

BoostedPointToPointMotion(handle, flags, axes, point, velocity, finalVelocity, time, fixedTime, motionDelay, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT plan the motion but do not start it until the function GoM is executed. MotionFlags.ACSC_AMF_VELOCITY the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_EXT_DELAY_MOTION Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The switch requires an additional parameter that specifies the motion delay. MotionFlags.ACSC_AMF_RELATIVE Relative motion, The value of the point coordinate is relative to the end point coordinate of the previous motion. MotionFlags.ACSC_AMF_LOCALCS Interpret entered coordinates according to the Local Coordinate System. With this switch, use 2 axes motion coordinate only (X,Y). Using 3 or more coordinates causes a runtime error. MotionFlags.ACSC_AMF_ENVELOPE Wait for motion termination before executing next command. The switch requires an additional parameter that specifies the motion delay.

axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. 2 or 3 Cartesian axes can participate in the motion. For the axis constants see Axis Definitions.
point	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last –1 element.
velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

5.17 Track Motion Control Functions

5.17.1 Track

Description

The function initiates a single-axis track motion.

Syntax

Track(handle, flags, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The method initiates a single-axis track motion. After the motion is initialized, PTP motion will be generated with every change in the TPOS value.
The controller response indicates that the command was accepted and the motion was planned successfully.

If Wait is a SYNCHRONOUS object, the function returns when the controller response is received.
If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result.
If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

5.17.2 SetTargetPosition

Description

The function assigns a current value of track position.

Syntax

SetTargetPosition(handle, axis, targetPosition, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_ AXIS_1 to axis 1,etc.
targetPosition	The value specifies the current value of track position.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function assigns a current value to the Track position. If the corresponding axis is initialized with track motion, the change of TPOS will cause generation of ptp motion to that new value.
For more information see the explanation of the track command in the SPiiPlus ACSPL+ Programmer's Guide.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.17.3 GetTargetPosition

Description

The function retrieves the instant value of track position.

Syntax

GetTargetPosition(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type.
The function retrieves a current value of motor feedback position.

Get extended error information by calling GetLastError.

Comments

The function reads a current value of the corresponding TPOS variable. If the corresponding axis is initialized with track motion, TPOS controls the motion of the axis.
For more information see the explanation of the track command in the SPiiPlus ACSPL+ Programmer's Guide.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
axis = 0
sp.SetTargetPosition(hc, axis, 50, sp.SYNCHRONOUS, True)
t_pos = sp.GetTargetPosition(hc, axis, sp.SYNCHRONOUS, True)
print(f"target position of axis {axis} is {t_pos}")
```

5.18 Jog Functions

5.18.1 Jog

Description

The function initiates a single-axis jog motion.

Syntax

Jog(handle, flags, axis, velocity, wait, failure_check=False): Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_ AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. 2 or 3 Cartesian axes can participate in the motion. For the axis constants see Axis Definitions.

direction	Array of directions. The number and order of values must correspond to the axes array. The direction array must specify direction for each element of axes except the last –1 element. The constant GeneralDefinition.ACSC_ POSITIVE_DIRECTION in the direction array specifies the correspondent axis to move in positive direction, the constant GeneralDefinition.ACSC_ NEGATIVE_DIRECTION specifies the correspondent axis to move in the negative direction.
velocity	If MotionFlags.ACSC_AMF_VELOCITYflag was specified, the velocity profile is built using the value of velocity. If the MotionFlags.ACSC_AMF_VELOCITY flag is not specified, velocity is not used.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function initiates a multi-axis jog motion. To execute single-axis jog motion, use Jog.
The jog motion is a motion with constant velocity and no defined ending point. The jog motion continues until the next motion is planned, or the motion is killed for any reason.
The motion builds the vector velocity profile using the default values of velocity, acceleration, deceleration and jerk of the axis group. If the MotionFlags.ACSC_AMF_VELOCITY flag is not specified, the default value of velocity is used as well. If the MotionFlags.ACSC_AMF_VELOCITY flag is specified, the value of Velocity is used instead of the default velocity.
The function can wait for the controller response or can return immediately as specified by the wait argument.
The controller response indicates that the command was accepted and the motion was planned successfully. The function cannot wait or validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.18.2 JogM

Description

The function initiates a multi-axis jog motion.

Syntax

JogM(handle, flags, axes, direction, velocity, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. 2 or 3 Cartesian axes can participate in the motion. For the axis constants see Axis Definitions.
direction	Array of directions. The number and order of values must correspond to the axes array. The direction array must specify direction for each element of axes except the last –1 element. The constant GeneralDefinition.ACSC_POSITIVE_DIRECTION in the direction array specifies the correspondent axis to move in positive direction, the constant GeneralDefinition.ACSC_NEGATIVE_DIRECTION specifies the correspondent axis to move in the negative direction.
velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, the velocity profile is built using the value of velocity. If the MotionFlags.ACSC_AMF_VELOCITY flag is not specified, velocity is not used.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function initiates a multi-axis jog motion. To execute single-axis jog motion, use Jog.
The jog motion is a motion with constant velocity and no defined ending point. The jog motion continues until the next motion is planned, or the motion is killed for any reason.
The motion builds the vector velocity profile using the default values of velocity, acceleration, deceleration and jerk of the axis group. If the MotionFlags.ACSC_AMF_VELOCITY flag is not specified, the default value of velocity is used as well. If the MotionFlags.ACSC_AMF_VELOCITY flag is specified, the value of Velocity is used instead of the default velocity.
The function can wait for the controller response or can return immediately as specified by the wait argument.
The controller response indicates that the command was accepted and the motion was planned successfully. The function cannot wait or validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.89", 701)
# axis 0 will move toward the positive direction and axis 1 will
# move in the negative direction
sp.JogM(hc, 0, [0, 1, -1], [1, -1, -1], 25, sp.SYNCHRONOUS, True)
```

5.19 Slaved Motion Functions

5.19.1 SetMaster

Description

The function initiates calculating of master value for an axis.

Syntax

SetMaster(handle, axis, formula, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
formula	ASCII string that specifies a rule for calculating master value.

wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates calculating of master value for an axis.
The master value for each axis is presented in the controller as one element of the **MPOS** array.
Once the **SetMaster** function is called, the controller is calculates the master value for the specified axis each controller cycle.
The **SetMaster** function can be called again for the same axis at any time. At that moment, the controller discards the previous formula and accepts the newly specified formula for the master calculation.
The function can wait for the controller response or can return immediately as specified by the `wait` argument.
The controller response indicates that the command was accepted and the controller starts calculating the master value according to the formula.
The `formula` string can specify any valid ACSPL+ expression that uses any standard or user global variables as its operands.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.19.2 Slave

Description

The function initiates a master-slave motion.

Syntax

Slave(handle, flags, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: **MotionFlags.ACSC_AMF_WAIT**: plan the motion but don't start it until the function Go is executed. **MotionFlags.ACSC_AMF_POSITIONLOCK**: the motion will use position lock. If the flag is not specified, velocity lock is used.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates a single-axis master-slave motion with an unlimited area of following. If the area of following must be limited, use SlaveStalled.

The master-slave motion continues until the motion is killed or the motion fails for any reason.
The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the motion was planned successfully.

The master value for the specified axis must be defined before by the call to SetMaster function. The **SetMaster** function can be called again in order to change the formula of master calculation.

If at this moment the master-slave motion is in progress, the slave can come out from synchrony. The controller then regains synchrony, probably with a different value of offset between the master and slave.

If the **MotionFlags.ACSC_AMF_POSITIONLOCK** flag is not specified, the function activates a velocity-lock mode of slaved motion. When synchronized, the **APOS** axis reference follows the **MPOS** with a constant offset:

```
APOS = MPOS + C
```

The value of **C** is latched at the moment when the motion comes to synchrony, and then remains unchanged as long as the motion is synchronous. If at the moment of motion start the master velocity is zero, the motion starts synchronously and C is equal to the difference between initial values of **APOS** and **MPOS**.

If the **MotionFlags.ACSC_AMF_POSITIONLOCK** flag is specified, the function activates a position-lock mode of slaved motion. When synchronized, the **APOS** axis reference strictly follows the **MPOS**:

```
APOS = MPOS
```

If Wait is a SYNCHRONOUS object, the function returns when the controller response is received.
If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result.
If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

5.19.3 SlaveStalled

Description

The function initiates master-slave motion within predefined limits.

Syntax

SlaveStalled(handle, flags, axis, left, right, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: **MotionFlags.ACSC_AMF_WAIT**: plan the motion but don't start it until the function Go is executed. **MotionFlags.ACSC_AMF_POSITIONLOCK**: the motion will use position lock. If the flag is not specified, velocity lock is used.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
left	Left (negative) limit of the following area.
right	Right (positive) limit of the following area.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates single-axis master-slave motion within predefined limits. Use Slave to initiate unlimited motion. For sophisticated forms of master-slave motion, use slaved variants of segmented and spline motions.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the motion was planned successfully.

The master-slave motion continues until the Kill command is executed, or the motion fails for some reason.

The master value for the specified axis must be defined before by the call to SetMaster function. The **SetMaster** function can be called again in order to change the formula of master calculation.

If at this moment the master-slave motion is in progress, the slave can come out from synchrony. The controller then regains synchrony, probably with a different value of offset between the master and slave.

If the **MotionFlags.ACSC_AMF_POSITIONLOCK** flag is not specified, the function activates a velocity-lock mode of slaved motion. When synchronized, the **APOS** axis reference follows the **MPOS** with a constant offset:

```
APOS = MPOS + C
```

The value of **C** is latched at the moment when the motion comes to synchrony, and then remains unchanged as long as the motion is synchronous. If at the moment of motion start the master velocity is zero, the motion starts synchronously and C is equal to the difference between initial values of **APOS** and **MPOS**.

If the **MotionFlags.ACSC_AMF_POSITIONLOCK** flag is specified, the function activates a position-lock mode of slaved motion. When synchronized, the **APOS** axis reference strictly follows the **MPOS**:

```
APOS = MPOS
```

If Wait is a SYNCHRONOUS object, the function returns when the controller response is received.
If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result.
If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

5.20 Multi-Point Motion Functions

5.20.1 MultiPoint

Description

The function initiates a single-axis multi-point motion.

Syntax

MultiPoint(handle, flags, axis, dwell, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate. MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_CYCLIC: the motion uses the point sequence as a cyclic array. After positioning to the last point it does positioning to the first point and continues.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
dwell	Dwell in each point in milliseconds.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates a single-axis multi-point motion. To execute multi-axis multi-point motion, use MultiPointM.

The motion executes sequential positioning to each of the specified points, optionally with dwell in each point.

The function itself does not specify any point, so that the created motion starts only after the first point is specified. The points of motion are specified by using the AddPoint or ExtAddPoint functions that follow this function.

The motion finishes when the EndSequence function is executed. If the call of **EndSequence** is omitted, the motion will stop at the last point of the sequence and wait for the next point. No transition to the next motion in the motion queue will occur until the function **EndSequence** executes.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the motion was planned successfully. The function cannot wait or validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

During positioning to each point, a velocity profile is built using the default values of acceleration, deceleration, and jerk of the specified axis. If the **MotionFlags.ACSC_AMF_VELOCITY** flag is not specified, the default value of velocity is used as well.

If the **MotionFlags.ACSC_AMF_VELOCITY** flag is specified, the value of velocity specified in subsequent ExtAddPoint function is used.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.20.2 MultiPointM

Description

The function initiates a multi-axis multi-point motion.

Syntax

MultiPointM(handle, flags, axes, dwell, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate. MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_CYCLIC: the motion uses the point sequence as a cyclic array. After positioning to the last point it does positioning to the first point and continues.

axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
dwell	Dwell in each point in milliseconds.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates a multi-axis multi-point motion. To execute single-axis multi-point motion, use MultiPoint.

The motion executes sequential positioning to each of the specified points, optionally dwelling for the specified interval at each point.

The function itself does not specify any point, so that the created motion starts only after the first point is specified. The points of motion are specified by using the AddPointM or ExtAddPointM functions that follow this function.

The motion finishes when the EndSequenceM function is executed. If the call of **EndSequenceM** is omitted, the motion will stop at the last point of the sequence and wait for the next point. No transition to the next motion in the motion queue will occur until the function **EndSequenceM** executes.
The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the motion was planned successfully. The function cannot wait or validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

During positioning to each point, a vector velocity profile is built using the default values of velocity, acceleration, deceleration, and jerk of the axis group. If the **MotionFlags.AFM_VELOCITY** flag is not specified, the default value of velocity is used as well. If the **MotionFlags.AFM_VELOCITY** flag is specified, the value of velocity specified in subsequent ExtAddPointM functions is used.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.21 Arbitrary Path Motion Functions

5.21.1 Spline

Description

The function initiates a single-axis spline motion. The motion follows an arbitrary path defined by a set of points.

Syntax

Spline(handle, flags, axis, period, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate. MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_CYCLIC: the motion uses the point sequence as a cyclic array. After positioning to the last point it does positioning to the first point and continues. MotionFlags.ACSC_AMF_CUBIC: use a cubic interpolation between the specified points (third order spline). If the flag is not specified, linear interpolation is used (first order spline). If the flag is specified and the MotionFlags.ACSC_AMF_VARTIME is not specified, the controller builds a PV spline motion. If the flag is specified and the MotionFlags.ACSC_AMF_VARTIME is specified, the controller builds a PVT spline motion.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
period	Time interval between adjacent points. The parameter is used only if the **MotionFlags.ACSC_ AMF_VARTIME** flag is not specified.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates a single-axis spline motion. To execute multi-axis spline motion, use SplineM.

The motion executes sequential positioning to each of the specified points, optionally with dwell in each point.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the motion was planned successfully. The function cannot wait or validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

The motion does not use the default values of velocity, acceleration, deceleration, and jerk. The points and the time intervals between the points completely define the motion profile.
Points for arbitrary path motion are defined by the consequent calls of AddPoint or ExtAddPoint functions. The EndSequence function terminates the point sequence. After execution of the **EndSequence** function, no AddPVPoint or ExtAddPoint functions for this motion are allowed.

The trajectory of the motion follows through the defined points. Each point presents the instant desired position at a specific moment. Time intervals between the points are uniform, or nonuniform as defined by the **MotionFlags.ACSC_AMF_VARTIME** flag.

This motion does not generate a motion profile. The time intervals between the points are typically short, so that the array of the points implicitly specifies the desired velocity in each point.

If the time interval does not coincide with the controller cycle, the controller provides interpolation of the points according to the **MotionFlags.ACSC_AMF_CUBIC** flag.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Example

```
hc = sp.OpenCommEthernetTCP("192.168.56.1", 701)
# Define spline motion
sp.Spline(hc, sp.MotionFlags.ACSC_AMF_CUBIC, 0,
```

```
          100, sp.SYNCHRONOUS, True)
# Add sequence of points
for i in range(5):
    sp.AddPVPoint(hc, 0, i*10, 200, sp.SYNCHRONOUS, True)
# End sequance
sp.EndSequence(hc, 0, sp.SYNCHRONOUS, True)
```

5.21.2 SplineM

Description

The function initiates a multi-axis spline motion. The motion follows an arbitrary path defined by a set of points.

Syntax

SplineM(handle, flags, axes, period, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate. MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_CYCLIC: the motion uses the point sequence as a cyclic array. After positioning to the last point it does positioning to the first point and continues. MotionFlags.ACSC_AMF_CUBIC: use a cubic interpolation between the specified points (third order spline).
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be located that contains -1 which marks the end of the array.
period	Time interval between adjacent points. The parameter is used only if the MotionFlags.ACSC_ AMF_VARTIME flag is not specified.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function initiates a multi-axis spline motion. To execute a single-axis spline motion, use Spline.

The motion executes sequential positioning to each of the specified points, optionally with dwell in each point.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the motion was planned successfully. The function cannot wait or validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

The motion does not use the default values of velocity, acceleration, deceleration, and jerk. The points and the time intervals between the points define the motion profile completely.

Points for arbitrary path motion are defined by the consequent calls of the AddPVPointM or AddPVTPoint functions. The EndSequence function terminates the point sequence. After execution of the **EndSequenceM** function, no **AddPVPointM** or **AddPVTPointM** functions for this motion are allowed.

The trajectory of the motion follows through the defined points. Each point presents the desired position at a specific moment. Time intervals between the points are uniform, or nonuniform as defined by the **MotionFlags.ACSC_AMF_VARTIME** flag.

This motion does not use a motion profile generation. The time intervals between the points are typically short, so that the array of the points implicitly specifies the desired velocity in each point. If the time interval does not coincide with the controller cycle, the controller provides interpolation of the points according to the **MotionFlags.ACSC_AMF_CUBIC** flag.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.22 PVT Functions

5.22.1 AddPVPoint

Description

The function adds a point to a single-axis PV spline motion and specifies velocity.

Syntax

AddPVPoint(handle, axis, point, velocity, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
point	Coordinate of the added point
velocity	Desired velocity at the point
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

Before this function can be used, PV spline motion must be initiated by calling Spline with the appropriate flags.

The function adds a point to a single-axis PV spline motion with a uniform time and specified velocity at that point.

To add a point to a multi-axis PV motion, use AddPVTPointM. To add a point to a PVT motion with non-uniform time interval, use the AddPVTPoint and AddPVTPointM functions. The function can wait for the controller response or can return immediately as specified by the *wait* argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.22.2 AddPVPointM

Description

The function adds a point to a multiple-axis PV spline motion and specifies velocity.

Syntax

AddPVPointM(handle, axis, point, velocity, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
point	Array of the coordinates of added point. The number and order of values must correspond to the `axes` array. The `point` must specify a value for each element of axes except the last –1 element.
velocity	Array of the velocities of added point. The number and order of values must correspond to the `axes` array. The `velocity` must specify a value for each element of `axes` except the last –1 element.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

Before this function can be used, PVT spline motion must be initiated by calling SplineM with the appropriate flags.

The function adds a point to a multiple-axis PV spline motion with a uniform time and specified velocity at that point.

To add a point to a single-axis PV motion, use AddPVPointM. To add a point to a PVT motion with non-uniform time interval, use the AddPVTPoint and AddPVTPointM functions.
The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function

periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

All axes specified in the `axes` array must be specified before the call of the MultiPointM or SplineM. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of **MultiPointM** or **SplineMfunctions**.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.22.3 AddPVTPoint

Description

The function adds a point to a single-axis PVT spline motion and specifies velocity and motion time.

Syntax

AddPVTPoint(handle, axis, point, velocity, timeInterval, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
point	Coordinate of the added point.
velocity	Desired velocity at the point.
timeInterval	If the motion was activated by the Spline function with the **MotionFlags.ACSC_AMF_VARTIME** flag, this parameter defines the time interval between the previous point and the present one
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

Before this function can be used, PV spline motion must be initiated by calling **Spline** with the appropriate flags.

The function adds a point to a single-axis PVT spline motion with a non-uniform time and specified velocity at that point.

To add a point to a multi-axis PVT motion, use AddPVTPointM. To add a point to a PV motion with uniform time interval, use the AddPVPointand AddPVPointM functions.

The function can wait for the controller response or can return immediately as specified by the wait argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.22.4 AddPVTPointM

Description

The function adds a point to a multiple-axis PVT spline motion and specifies velocity and motion time.

Syntax

AddPVTPointM(handle, axis, point, velocity, timeInterval, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
point	Array of the coordinates of added point. The number and order of values must correspond to the `axes` array. The `point` must specify a value for each element of `axes` except the last –1 element.
velocity	Array of the velocities of added point. The number and order of values must correspond to the `axes` array. The `velocity` must specify a value for each element of `axes` except the last –1 element.
timeInterval	If the motion was activated by the SplineM_ function with the MotionFlags.ACSC_AMF_VARTIME flag, this parameter defines the time interval between the previous point and the present one.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

Before this function can be used, PVT spline motion must be initiated by calling SplineM with the appropriate flags.

The function adds a point to a multiple-axis PVT spline motion with a non-uniform time and specified velocity at that point.

To add a point to a single-axis PVT motion, use AddPVTPoint. To add a point to a PV motion with uniform time interval, use the AddPVPoint and AddPointM functions.

The function can wait for the controller response or can return immediately as specified by the wait argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.23 Segmented Motion Functions

5.23.1 ExtendedSegmentedMotionV2

Description

The function initiates a multi-axis extended segmented motion.

Syntax

ExtendedSegmentedMotionV2(handle, flags, axes, point, velocity, endVelocity, junctionVelocity, angle, curveVelocity, deviation, radius, maxLength, starvationMargin, segments, extLoopType, minSegmentLength, maxAllowedDeviation, outputIndex, bitNumber, polarity, motionDelay, wait, failure_check=False)

Arguments

handle	Communication handle.

flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_ENDVELOCITY: This flag requires an additional parameter that specifies end velocity. The controller decelerates to the specified velocity at the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions. MotionFlags.ACSC_AMF_MAXIMUM: use maximum velocity under axis limits. With this flag, no required velocity should be specified. The required velocity is calculated for each segment individually on the base of segment geometry and axis velocities (VEL values) of the involved axes. MotionFlags.ACSC_AMF_JUNCTIONVELOCITY: Decelerate to corner. This flag requires an additional parameter that specifies corner velocity. The controller detects corners on the path and decelerates to the specified velocity before the corner. The specified value should be less than the required velocity; otherwise the parameter is ignored. If ACSC_AMF_JUNCTIONVELOCITY flag is not specified while ACSC_AMF_ANGLE flag is specified, zero value of corner velocity is assumed. If none of the ACSC_AMF_JUNCTIONVELOCITY and ACSC_AMF_ANGLE flags are specified, the controller provides automatic calculation as described in Automatic corner processing. MotionFlags.ACSC_AMF_ANGLE: Do not treat junction as a corner, if junction angle is less than or equal to the specified value in radians. This flag requires additional parameter that specifies negligible angle in radians. If ACSC_AMF_ANGLE flag is not specified while ACSC_ AMF_ JUNCTIONVELOCITY flag is specified, the controller accepts

default value of 0.01 radians that is about 0.57 degrees.
If none of the ACSC_AMF_JUNCTIONVELOCITY and ACSC_AMF_ ANGLE flags are specified, the controller provides automatic calculation as described in Automatic corner processing.

MotionFlags.ACSC_AMF_AXISLIMIT

Enable velocity limitations under axis limits.
With this flag set, setting the ACSC_AMF_VELOCITY flag will result in the requested velocity being restrained by the velocity limits of all involved axes.

MotionFlags.ACSC_AMF_CURVEVELOCITY

Decelerate to curvature discontinuity point.
This flag requires an additional parameter that specifies velocity at curvature discontinuity points.
Curvature discontinuity occurs in linear-to-arc or arcto-arc smooth junctions.
If the flag is not set, the controller does not decelerate to smooth junction disregarding curvature discontinuity in the junction.
If the flag is set, the controller detects curvature discontinuity points on the path and provides deceleration to the specified velocity.
The specified value should be less than the required velocity; otherwise the parameter is ignored.
The flag can be set together with flags ACSC_AMF_ JUNCTIONVELOCITY and/or ACS_AMF_ANGLE.
If neither of ACSC_AMF_JUNCTIONVELOCITY, ACS_AMF_ ANGLE or ACSC_AMF_CURVEVELOCITY is set, the controller provides automatic calculation of the corner processing.

MotionFlags.ACSC_AMF_CURVEAUTO

If the Flag is specified the controller provides automatic calculations as described in Enhanced automatic corner and curvature discontinuity points processing.

MotionFlags.ACSC_AMF_CORNERDEVIATION

Use a corner rounding option with the specified permitted deviation. This flag requires an additional parameter that specifies maximal allowed deviation of motion trajectory from the corner point. This flag cannot be set together with flags ACSC_AMF_ CORNERRADIUS and ACSC_AMF_CORNERLENGTH.

MotionFlags.ACSC_AMF_CORNERRADIUS

Use a corner rounding option with the specified permitted curvature. This flag requires an additional parameter that specifies maximal allowed rounding radius of the additional segment. This flag cannot be specified together with flags

	ACSC_AMF_ CORNERLENGTH or ACSC_AMF_CORNERDEVIATION MotionFlags.ACSC_AMF_CORNERLENGTH Use automatic corner rounding option. This flag requires an additional parameter that specifies the maximum length of the segment for automatic corner rounding. If a length of one of the segments that built a corner exceeds the specified maximal length, the corner will not be rounded. The automatic corner rounding is only applied to pair of linear segments. If one of the segments in a pair is an arc, the rounding is not applied for this corner. This flag cannot be set together with flags ACSC_AMF_ CORNERDEVIATION or ACSC_AMF_CORNERRADIUS. MotionFlags.ACSC_AMF_EXT_LOOP The switch requires additional parameters that specify the external loop type, the minimum segment length, and the maximum allowed deviation from profile. MotionFlags.ACSC_AMF_EXT_LOOP_SYNC Defines output bit to support external loop synchronization. The switch requires additional parameters that specify the output index, the bit number, and the polarity. MotionFlags.ACSC_AMF_DELAY_MOTION Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The flag requires an additional parameter that specifies the motion delay. MotionFlags.ACSC_AMF_LOCALCS Interpret entered coordinates according to the Local Coordinate System. With this switch, use 2 axes motion coordinate only (X,Y). Using 3 or more coordinates causes a runtime error.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be located that contains -1 which marks the end of the array.
point	Array of the starting point coordinates. The number and order of values must correspond to the Axes array. `point` must specify a value for each element of `axes` except the last –1 element.

velocity	If the MotionFlags.ACSC_AMF_VELOCITYflag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
endVelocity	Velocity in the target point. The argument is used only if the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity. If the MotionFlags.ACSC_AMF_ENDVELOCITY flag was specified, this argument defines required velocity at the end of the current segment. Set this argument to GeneralDefinition.ACSC_NONEif not used.
junctionVelocity	If the MotionFlags.ACSC_AMF_JUNCTIONVELOCITY flag was specified, this argument defines the required velocity at the junction. Set this argument to GeneralDefinition.ACSC_NONE if not used.
angle	If the MotionFlags.ACSC_AMF_ANGLE flag was specified, this argument specifies the threshold above which a junction angle will be treated as a corner. Set this argument to GeneralDefinition.ACSC_NONE if not used.
curveVelocity	If the MotionFlags.ACSC_AMF_CURVEVELOCITY flag has been specified, this argument defines the required velocity at curvature discontinuity points. Set this argument to GeneralDefinition.ACSC_NONE if not used.
deviation	If the MotionFlags.ACSC_AMF_CORNERDEVIATION flag has been specified, this argument defines the maximal allowed trajectory deviation from the corner point. Set this argument to GeneralDefinition.ACSC_NONE if not used.
radius	If MotionFlags.ACSC_AMF_CORNERRADIUS flag has been specified, this argument defines the maximal allowed rounding radius of the additional segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
maxLength	If MotionFlags.ACSC_AMF_CORNERLENGTH flag has been specified, this argument defines the maximum length of the segment for processing automatic corner rounding. If a segment's corner attempts to exceed the maximum length, the corner will not be rounded. Set this argument to GeneralDefinition.ACSC_NONE if not used.

starvationMargin	Starvation margin in milliseconds. The controller sets the AST.#NEWSEGM bit to the starvationMargin millisecond before the starvation condition occurs. Set this argument to GeneralDefinition.ACSC_NONE if not used. By default, if this argument is not specified, the starvation margin is 500 milliseconds.
segments	String that contains the name of a one-dimensional user-defined array used to store added segments. Set this argument to None if not used. By default, if this argument is not specified, the controller allocates internal buffer for storing 50 segments only. The argument allows the user application to reallocate the buffer for storing a larger number of segments. The larger number of segments may be required if the application needs to add many very small segments in advanced. For most applications, the internal buffer size is enough and should not be enlarged. The buffer is for the controller internal use only and should not be used by the user application. The buffer size calculation rule: each segment requires about 600 bytes, so if it is necessary to allocate the buffer for 200 segments, it should be at least 600 * 200 = 120,000 bytes. The following declaration defines a 120,000 bytes buffer: real buf (15000). See XARRSIZE explanation in the ACSPL+ Command and Variable Reference Guide for details on how to declare a buffer with more than 100,000 elements
extLoopType	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the external loop type. 0 - Cancel external loop 1 – Smooth External loop (line-arc-line) 2 –Triangle External loop (line-line-line). Set this argument to GeneralDefinition.ACSC_NONE if not used.
minSegmentLength	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the Minimum Segment Length. If the lengths of both segments are more than this value, the skywriting algorithm will be applied. Set this argument to GeneralDefinition.ACSC_NONE if not used.
maxAllowedDeviation	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the maximum allowed deviation. The parameter limits the external loop deviation from the defined profile. If the value is negative there is no limitation. Set this argument to GeneralDefinition.ACSC_NONE if not used.

outputIndex	If MotionFlags.ACSC_AMF_EXT_LOOP_SYNC flag has been specified, this argument defines the output index, the index of the digital output port to assigned to synchronization (read by OUT). Set this argument to GeneralDefinition.ACSC_NONE if not used.
bitNumber	If MotionFlags.ACSC_AMF_EXT_LOOP_SYNC flag has been specified, this argument defines the bit number. Bit number (BIT_NUMBER_MIN(0) - BIT_NUMBER_MAX(16)) assigned to synchronization output. Set this argument to GeneralDefinition.ACSC_NONE if not used.
polarity	If MotionFlags.ACSC_AMF_EXT_LOOP_SYNC flag has been specified, this argument defines the polarity. POLARITY_ON(0) or POLARITY_OFF(1) - which value is considered the initial state. Set this argument to GeneralDefinition.ACSC_NONE if not used.
motionDelay	If MotionFlags.ACSC_AMF_DELAY_MOTION flag has been specified, this argument defines the actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles or 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Set this argument to GeneralDefinition.ACSC_NONE if not used.
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function itself does not specify any segment, so the created motion starts only after the first segment is specified.
The segments of motion are specified by using the SegmentLineV2, , and SegmentArc2V2 functions. The motion finishes when the EndSequenceM function is executed. If the call to **EndSequenceM** is omitted, the motion will stop at the last segment of the sequence and wait for the next segment. No transition to the next motion in the motion queue will occur until the function **EndSequenceM** is

executed.
During positioning to each point, a vector velocity profile is built using the default values of velocity, acceleration, deceleration, and jerk of the axis group. If the ACSC_AFM_VELOCITY flag is not specified, the default value of velocity is used as well. If the ACSC_AFM_VELOCITY flag is specified, the value of velocity specified in subsequent SegmentLineExt, ExtendedSegmentArc1, or ExtendedSegmentArc2 functions is used.
The function can wait for the controller response or can return immediately as specified by the `wait` argument.
The controller response indicates that the command was accepted and the motion was planned successfully. The function does not wait and does not validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.23.2 SegmentLineV2

Description

The function adds a linear segment that starts at the current point and ends at the destination point of segmented motion.

Syntax

SegmentLineV2(handle, flags, axes, point, velocity, endVelocity, time, values, variables, index, masks, extLoopType, minSegmentLength, maxAllowedDeviation, lciState, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_ENDVELOCITY: This flag requires additional parameter that specifies end velocity. The controller decelerates to the specified velocity in the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment. MotionFlags.ACSC_AMF_VARTIME: this flag requires an addition parameter that specifies the segment processing time in milliseconds. The segment processing time defines velocity at the current segment only and has no effect on subsequent segments. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions. MotionFlags.ACSC_AMF_USERVARIABLES: synchronize user variables with segment execution. This flag requires additional parameters that specify values, user variable and mask. See details in values, variables, and masks below. MotionFlags.ACSC_AMF_EXT_LOOP The switch requires additional parameters that specify the external loop type, the minimum segment length, and the maximum allowed deviation from profile. MotionFlags.ACSC_AMF_LCI_STATE: The switch requires additional parameter that specify LCI state.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be located that contains -1 which marks the end of the array.
point	Array of the starting point coordinates. The number and order of values must correspond to the Axes array. point must specify a value for each element of axes except the last –1 element.

velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
endVelocity	Velocity in the target point. The argument is used only if the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity. If MotionFlags.ACSC_AMF_ENDVELOCITY flag was specified, this argument defines required velocity at the end of the current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
time	If MotionFlags.ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to MotionFlags.ACSC_NONE if not used.
values	String that contains the name of a one-dimensional user-defined array of integer or real type with a size of 10 elements maximum. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the values to be written to the variables array at the beginning of the current segment execution. Set this argument to GeneralDefinition.ACSC_NONE if not used.
variables	String that contains the name of a one-dimensional user-defined array of the same type and size as values array. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the user-defined array, which will be written with values data at the beginning of the current segment execution. Set this argument to MotionFlags.ACSC_NONE if not used
index	If MotionFlags.ACSC_AMF_USERVARIABLES has not been specified, this argument defines the first element (starting from zero) of the variables array, to which values data will be written to. Set this argument to MotionFlags.ACSC_NONE if not used

masks	String that contains the name of a one-dimensional user defined array of integer type and same size as the values array. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the masks that are applied to values before the values are written to variables array at the beginning of the current segment execution. The masks are only applied for integer values: variables(n) = values(n) AND mask(n). If values is a real array, the masks argument should be None. Set this argument to MotionFlags.ACSC_NONE if not used.
extLoopType	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the external loop type. 0 - Cancel external loop 1 – Smooth External loop (line-arc-line) 2 –Triangle External loop (line-line-line). Set this argument to GeneralDefinition.ACSC_NONE if not used.
minSegmentLength	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the Minimum Segment Length. If the lengths of both segments are more than this value, the skywriting algorithm will be applied. Set this argument to GeneralDefinition.ACSC_NONE if not used.
maxAllowedDeviation	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the maximum allowed deviation. The parameter limits the external loop deviation from the defined profile. If the value is negative there is no limitation. Set this argument to GeneralDefinition.ACSC_NONE if not used.
lciState	If MotionFlags.ACSC_AMF_LCI_STATE flag has been specified, this argument defines the LCI state. LCI_STATE_ON(0) or LCI_STATE_OFF(1) determines the value to be considered the initial state. Set this argument to GeneralDefinition.ACSC_NONE if not used.
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function adds a linear segment that starts at the current point and ends at the destination point to segmented motion.
All axes specified in the Axes array must be specified before the call of the ExtendedSegmentedMotionV2 function. The number and order of the axes in the `axes` array must correspond exactly to the number and order of the axes of the **ExtendedSegmentedMotionV2** function.
The `point` argument specifies the coordinates of the final point. The coordinates are absolute in the plane.
ACSC_AMF_VELOCITY and ACSC_AMF_VARTIME are mutually exclusive, meaning they cannot be used together.
The function can wait for the controller response or can return immediately as specified by the `wait` argument.
The controller response indicates that the command was accepted and the segment is added to the motion buffer. The segment can be rejected if the motion buffer is full. In that case, you can call this function periodically until the function returns a non-zero value.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.23.3 SegmentArc1V2

Description

The function provides a smooth transition between two segments of segmented motion.

Syntax

SegmentArc1V2(handle, flags, axes, center, finalPoint, rotation, velocity, endVelocity, time, values, variables, index, masks, extLoopType, minSegmentLength, maxAllowedDeviation, lciState, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_ENDVELOCITY: This flag requires additional parameter that specifies end velocity. The controller decelerates to the specified velocity in the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions. MotionFlags.ACSC_AMF_VARTIME: this flag requires an addition parameter that specifies the segment processing time in milliseconds. The segment processing time defines velocity at the current segment only and has no effect on subsequent segments. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions. MotionFlags.ACSC_AMF_USERVARIABLES: synchronize user variables with segment execution. This flag requires additional parameters that specify values, user variable and mask. See details in values, variables, and masks below. MotionFlags.ACSC_AMF_EXT_LOOP The switch requires additional parameters that specify the external loop type, the minimum segment length, and the maximum allowed deviation from profile. MotionFlags.ACSC_AMF_LCI_STATE: The switch requires additional parameter that specify LCI state.
axes	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.

center	Array of the center coordinates. The number and order of values must correspond to the axes array. The center must specify a value for each element of the axes except the last–1 element
finalPoints	Array indicating the final points of the secondary axes, array size must be number of secondary axes (size of axes – 2). Set this argument to None if not used.
rotation	This argument defines the direction of rotation. If rotation is set to GeneralDefinition.ACSC_COUNTERCLOCKWISE, then the rotation is counterclockwise. If rotation is set to ACSC_ CLOCKWISE, then rotation is clockwise.
velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
endVelocity	Velocity at the target point. The argument is used only if the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity. If MotionFlags.ACSC_AMF_ENDVELOCITY flag was specified, this argument defines required velocity at the end of the current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
time	If MotionFlags.ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to MotionFlags.ACSC_NONE if not used.
values	String that contains the name of a one-dimensional user-defined array of integer or real type with a size of 10 elements maximum. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the values to be written to the variables array at the beginning of the current segment execution. Set this argument to GeneralDefinition.ACSC_NONE if not used.
variables	String that contains the name of a one-dimensional user-defined array of the same type and size as values array. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the user-defined array, which will be written with values data at the beginning of the current segment execution. Set this argument to MotionFlags.ACSC_ NONE if not used

index	If MotionFlags.ACSC_AMF_USERVARIABLES has not been specified, this argument defines the first element (starting from zero) of the variables array, to which values data will be written to. Set this argument to MotionFlags.ACSC_NONE if not used
masks	String that contains the name of a one-dimensional user defined array of integer type and same size as the values array. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the masks that are applied to values before the values are written to variables array at the beginning of the current segment execution. The masks are only applied for integer values: variables(n) = values(n) AND mask(n). If values is a real array, the masks argument should be None. Set this argument to MotionFlags.ACSC_NONE if not used.
extLoopType	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the external loop type. 0 - Cancel external loop 1 – Smooth External loop (line-arc-line) 2 – Triangle External loop (line-line-line). Set this argument to GeneralDefinition.ACSC_NONE if not used.
minSegmentLength	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the Minimum Segment Length. If the lengths of both segments are more than this value, the skywriting algorithm will be applied. Set this argument to GeneralDefinition.ACSC_NONE if not used.
maxAllowedDeviation	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the maximum allowed deviation. The parameter limits the external loop deviation from the defined profile. If the value is negative there is no limitation. Set this argument to GeneralDefinition.ACSC_NONE if not used.
lciState	If MotionFlags.ACSC_AMF_LCI_STATE flag has been specified, this argument defines the LCI state. LCI_STATE_ON(0) or LCI_STATE_OFF(1) determines the value to be considered the initial state. Set this argument to GeneralDefinition.ACSC_NONE if not used.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

All axes specified in the Axes array must be specified before the call of the **ExtendedSegmentedMotionV2** function. The number and order of the axes in the axes array must correspond exactly to the number and order of the axes of the **ExtendedSegmentedMotionV2** function. The point argument specifies the coordinates of the final point. The coordinates are absolute in the plane. ACSC_AMF_VELOCITY and ACSC_AMF_VARTIME are mutually exclusive, meaning they cannot be used together. The function can wait for the controller response or can return immediately as specified by the wait argument. The controller response indicates that the command was accepted and the segment is added to the motion buffer. The segment can be rejected if the motion buffer is full. In that case, call this function periodically until the function returns a non-zero value.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.23.4 SegmentArc2V2

Description

The function adds an arc segment to a segmented motion and specifies the coordinates of the center point and the rotation angle.

Syntax

SegmentArc2V2(handle, flags, axes, center, angle, finalPoint, velocity, endVelocity, time, values, variables, index, masks, extLoopType, minSegmentLength, maxAllowedDeviation, lciState, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_VELOCITY: the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_ENDVELOCITY: This flag requires additional parameter that specifies end velocity. The controller decelerates to the specified velocity in the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment. MotionFlags.ACSC_AMF_VARTIME: this flag requires an addition parameter that specifies the segment processing time in milliseconds. The segment processing time defines velocity at the current segment only and has no effect on subsequent segments. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions. MotionFlags.ACSC_AMF_USERVARIABLES: synchronize user variables with segment execution. This flag requires additional parameters that specify values, user variable and mask. See details in values, variables, and masks below. MotionFlags.ACSC_AMF_EXT_LOOP The switch requires additional parameters that specify the external loop type, the minimum segment length, and the maximum allowed deviation from profile. MotionFlags.ACSC_AMF_LCI_STATE: The switch requires additional parameter that specify LCI state.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be included containing -1, which marks the end of the array.
center	Array of the center coordinates. The number and order of values must correspond to the axes array. The center must specify a value for each element of the axes except the final –1 element

angle	Rotation angle in radians. Positive angle for counterclockwise rotation, negative for clockwise rotation.
finalPoints	Array indicating the final points of the secondary axes, array size must be number of secondary axes (size of axes – 2). Set this argument to None if not used.
velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
endVelocity	Velocity in the target point. The argument is used only if the MotionFlags.ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity. If MotionFlags.ACSC_AMF_ENDVELOCITY flag was specified, this argument defines required velocity at the end of the current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
time	If MotionFlags.ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to MotionFlags.ACSC_NONE if not used.
values	String that contains the name of a one-dimensional user-defined array of integer or real type with a size of 10 elements maximum. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the values to be written to the variables array at the beginning of the current segment execution. Set this argument to GeneralDefinition.ACSC_NONE if not used.
variables	String that contains the name of a one-dimensional user-defined array of the same type and size as the values array. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the user-defined array, which will be written with values data at the beginning of the current segment execution. Set this argument to MotionFlags.ACSC_NONE if not used

index	If MotionFlags.ACSC_AMF_USERVARIABLES has not been specified, this argument defines the first element (starting from zero) of the variables array, to which the values data will be written. Set this argument to MotionFlags.ACSC_NONE if not used
masks	String that contains the name of a one-dimensional user defined array of integer type and same size as the values array. If MotionFlags.ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the masks that are applied to values before the values are written to variables array at the beginning of the current segment execution. The masks are only applied for integer values: variables(n) = values(n) AND mask(n). If values is a real array, the masks argument should be None. Set this argument to MotionFlags.ACSC_NONE if not used.
extLoopType	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the external loop type. 0 - Cancel external loop 1 – Smooth External loop (line-arc-line) 2 –Triangle External loop (line-line-line). Set this argument to GeneralDefinition.ACSC_NONE if not used.
minSegmentLength	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the Minimum Segment Length. If the lengths of both segments are more than this value, the skywriting algorithm will be applied. Set this argument to GeneralDefinition.ACSC_NONE if not used.
maxAllowedDeviation	If MotionFlags.ACSC_AMF_EXT_LOOP flag has been specified, this argument defines the maximum allowed deviation. The parameter limits the external loop deviation from the defined profile. If the value is negative there is no limitation. Set this argument to GeneralDefinition.ACSC_NONE if not used.
lciState	If MotionFlags.ACSC_AMF_LCI_STATE flag has been specified, this argument defines the LCI state. LCI_STATE_ON(0) or LCI_STATE_OFF(1) determines the value to be considered the initial state. Set this argument to GeneralDefinition.ACSC_NONE if not used.

wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

Comments

All axes specified in the `axes` array must be specified before calling the function. The number and order of the axes in the `axes` array must correspond exactly to the number and order of the axes of the ExtendedSegmentedMotionV2 function.

The `point` argument specifies the coordinates of the final point. The coordinates are absolute in the plane.

ACSC_AMF_VELOCITY and ACSC_AMF_VARTIME are mutually exclusive, meaning they cannot be used together.

The function can wait for the controller response or can return immediately as specified by the Wait argument.

The controller response indicates that the command was accepted and the segment is added to the motion buffer. The segment can be rejected if the motion buffer is full. In that case, you can call this function periodically until the function returns a non-zero value.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.23.5 Stopper

Description

The function provides a smooth transition between two segments of segmented motion.

Syntax

Stopper(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be located that contains -1 which marks the end of the array.
wait	If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The controller builds the motion so that the vector velocity follows the smooth velocity diagram. The segments define the projection of the vector velocity to axis velocities. If all segments are connected smoothly, axis velocity is also smooth. However, if the user defined a path with an inflection point, axis velocity has a jump in this point. The jump can cause a motion failure due to the acceleration limit.
The function is used to avoid velocity jump in the inflection points. If the function is specified between two segments, the controller provides smooth deceleration to zero in the end of first segment and smooth acceleration to specified velocity in the beginning of second segment.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.23.6 Projection

Description

The function sets a projection matrix for segmented motion.

Syntax

Projection(handle, axes, matrix, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.

matrix	string containing the name of the matrix that provides the specified projection.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function sets a projection matrix for segmented motion.

The projection matrix connects the plane coordinates and the axis values in the axis group. The projection can provide any transformation as rotation or scaling. The number of the matrix rows must be equal to the number of the specified axes. The number of the matrix columns must equal two.

The matrix must be declared before as a global variable by an ACSPL+ program or by the DeclareVariable function and must be initialized by an ACSPL+ program or by the WriteReal function. For more information about projection, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.23.7 SmoothTransitionPointToPointMotion

Description

The SmoothTransitionPointToPointMotion command creates motion to a specified target. The **SmoothTransitionPointToPointMotion** commands work in sequence and the following **SmoothTransitionPointToPointMotion** command changes the previous target and provide a smooth transition from one motion direction to another, based on acceleration, deceleration and jerk values. The motion profile is optimized to pass on a rounded path near the breaking point, minimizing changes in speed and direction that would cause unwanted vibrations in the system.

Syntax

SmoothTransitionPointToPointMotion(handle, flags, axes, point, velocity, motionDelay, wait, failure_ check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_VELOCITY(/v): the motion will use velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_LOCALCS(/z) Interpret entered coordinates according to the Local Coordinate System. With this switch, use 2 axes motion coordinate only (X,Y). Using 3 or more coordinates causes a runtime error. MotionFlags.ACSC_AMF_EXT_DELAY_MOTION(/q) Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The switch requires additional parameter that specify the motion delay.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be included that contains -1 which marks the end of the array.
point	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the final –1 element.
velocity	If the MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
motionDelay	If the MotionFlags.ACSC_AMF_DELAY_MOTION flag has been specified, this argument defines the bit motion delay. Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles or 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Set this argument to GeneralDefinition.ACSC_NONE if not used.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

5.24 Blended Segmented Motion Functions

5.24.1 BlendedSegmentMotion

Description

The function initiates a multi-axis blended segmented motion.

Syntax

BlendedSegmentMotion(handle, flags, axes, point, segmentTime, accelerationTime, jerkTime, dwellTime, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_DWELLTIME: Dwell time between segments. This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified `dwellTime` milliseconds.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be included that contains -1 which marks the end of the array.
point	Array of the starting point coordinates. The number and order of values must correspond to the Axes array. point must specify a value for each element of axes except the final –1 element.
segmentTime	This parameter will set the default initial segment time in milliseconds.
accelerationTime	This parameter will set the default acceleration time in milliseconds.

jerkTime	This parameter will set the default jerk time in milliseconds.
dwellTime	If MotionFlags.ACSC_AMF_DWELLTIME is set, this parameter will set the initial dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified dwellTime milliseconds.
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

Comments

Blended segmented motion is a type of segmented motion that doesn't provide look-ahead capabilities, unlike extended segmented motion. Both type of motions are intended for processing a complex multi-axis trajectory and smoothing corners between segments, but do it in different ways. The extended segmented motion (ExtendedSegmentedMotionV2) allows achieving highest throughput within the defined axis limitations and the defined accuracy. The Blended segmented motion allows passing along the trajectory with the defined timing constrains.
The function itself does not specify any movement, so the created motion starts only after the first segment is specified. The segments of motion are specified by using BlendedLine, BlendedArc1, BlendedArc2 functions that follow this function.
The motion finishes when the EndSequenceM function is executed. If the call of **EndSequenceM** is omitted, the motion will stop at the last segment of the sequence and wait for the next segment. No transition to the next motion in the motion queue will occur until the function **EndSequenceM** is executed.
The function can wait for the controller response or can return immediately as specified by the Wait argument.
The controller response indicates that the command was accepted and the motion was planned successfully. The function does not wait and does not validate the end of the motion. To wait for the motion end, use the WaitMotionEnd function.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.24.2 BlendedLine

Description

The function adds a linear segment that starts at the current point and ends at the destination point of segmented motion.

Syntax

BlendedLine(handle, flags, axes, point, segmentTime, accelerationTime, jerkTime, dwellTime, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_DWELLTIME: Dwell time between segments. This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified dwellTime milliseconds.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be included that contains -1 which marks the end of the array.
point	Array of the starting point coordinates. The number and order of values must correspond to the Axes array. point must specify a value for each element of axes except the final –1 element.
segmentTime	This parameter will set the default initial segment time in milliseconds.
accelerationTime	This parameter will set the default Acceleration time in milliseconds.
jerkTime	This parameter will set the default Jerk time in milliseconds.
dwellTime	If the MotionFlags.ACSC_AMF_DWELLTIMEis set, this parameter will set the initial dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified dwellTime milliseconds.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function adds a linear segment that starts at the current point and ends at the destination point to segmented motion.

All axes specified in the `axes` array must be specified in a previous call to the BlendedSegmentMotion function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of the call to the **BlendedSegmentMotion** function.

The `point` argument specifies the coordinates of the final point. The coordinates are absolute in the plane.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the segment is added to the motion buffer. The segment can be rejected if the motion buffer is full. In that case, you can call this function periodically until the function returns a non-zero value.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.24.3 BlendedArc1

Description

The function adds to the motion path an arc segment that starts at the current point and ends at the destination point with the specified center point.

Syntax

BlendedArc1(handle, flags, axes, center, finalPoint, rotation, segmentTime, accelerationTime, jerkTime, dwellTime, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_DWELLTIME: Dwell time between segments. This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified dwellTime milliseconds.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.
center	Array of the center coordinates. The number and order of values must correspond to the axes array. The center must specify a value for each element of the axes except the final –1 element.
finalPoint	Array of the final point coordinates. The number and order of values must correspond to the axes array. The point must specify a value for each element of axes except the final –1 element.
rotation	This argument defines the direction of rotation. If rotation is set to GeneralDefinition.ACSC_COUNTERCLOCKWISE, then the rotation is counterclockwise. If rotation is set to ACSC_CLOCKWISE, then rotation is clockwise.
segmentTime	If GeneralDefinition.ACSC_AMF_BSEGTIME is set, this parameter will set the segment time, in milliseconds, for the current and all following segments – until the parameter is redefined.
accelerationTime	If GeneralDefinition.ACSC_AMF_BSEGACC is set, this parameter will set the Acceleration time, in milliseconds, for the current and all following segments – until the parameter is redefined.
jerkTime	If ACSC_AMF_BSEGJERK is set, this parameter will set the default Jerk time, in milliseconds, for the current and all following segments – until the parameter is redefined.

dwellTime	IfMotionFlags.ACSC_AMF_DWELLTIMEis set, this parameter will set the initial dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segmentfor the specified dwellTime milliseconds.
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

Comments

All axes specified in the `axes` array must be specified in a previous call to the BlendedSegmentMotion function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of the call to the **BlendedSegmentMotion** function.

The `finalPoint` argument specifies the coordinates of the final point. The coordinates are absolute in the plane.

The function can wait for the controller response or can return immediately as specified by the Wait argument.

The controller response indicates that the command was accepted and the segment is added to the motion buffer. The segment can be rejected if the motion buffer is full. In that case, you can call this function periodically until the function returns a non-zero value.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.24.4 BlendedArc2

Description

The function adds an arc segment to a segmented motion and specifies the coordinates of the center point and the rotation angle.

Syntax

BlendedArc2(handle, flags, axes, center, angle, segmentTime, accelerationTime, jerkTime, dwellTime, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_DWELLTIME: Dwell time between segments. This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified dwellTime milliseconds.
axes	Array of axis constants. Each element specifies one involved axis: Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.
center	Array of the center coordinates. The number and order of values must correspond to the axes array. The center must specify a value for each element of the axes except the final –1 element.
angle	Rotation angle in radians. Positive angle for counterclockwise rotation, negative for clockwise rotation.
segmentTime	If GeneralDefinition.ACSC_AMF_BSEGTIME is set, this parameter will set the segment time, in milliseconds, for the current and all following segments – until the parameter is redefined.
accelerationTime	If GeneralDefinition.ACSC_AMF_BSEGACC is set, this parameter will set the Acceleration time, in milliseconds, for the current and all following segments – until the parameter is redefined.
jerkTime	If ACSC_AMF_BSEGJERK is set, this parameter will set the default Jerk time, in milliseconds, for the current and all following segments – until the parameter is redefined.
dwellTime	IfMotionFlags.ACSC_AMF_DWELLTIMEis set, this parameter will set the initial dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segmentfor the specified dwellTime milliseconds.

wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

Comments

All axes specified in the axes array must be specified in a previous call to the BlendedSegmentMotion function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of the call to the **BlendedSegmentMotion** function.

The `center` argument specifies the coordinates of the Center point. The coordinates are absolute in the plane.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.25 NURBS and Smooth Path Motion

5.25.1 NurbsMotion

Description

The function creates NURBS motion. NURBS is a motion generator based on the NURBS spline specification. The algorithm accepts as parameters points, weights, and knots, and generates a spline trajectory accordingly.

Syntax

NurbsMotion(handle, flags, axes, velocity, exceptionAngle, exceptionLength, motionDelay, segments, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_VELOCITY: the motion will use the velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_EXT_DELAY_MOTION: Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The switch requires additional parameter that specifies the motion delay. MotionFlags.ACSC_AMF_NURBS_CONSIDER_ACC: Acceleration consideration. Allow the Motion Generator to deviate from specified axes acceleration parameter during velocity profile generation. MotionFlags.ACSC_AMF_NURBS_EXCEPTION_ANGLE: The suffix requires additional parameter that specifies maximum angle in a control point. The value defines exceptions from spline interpolation. If for an internal control point, directions to the previous and the next control points require direction change more than the specified angle (by modulo), the control point is processed as a corner. Actually, defining such a point divides the spline into two independent splines. MotionFlags.ACSC_AMF_NURBS_EXCEPTION_LENGTH: Specify exception length. The suffix requires an additional parameter that specifies maximum segment length. The value defines exceptions from spline interpolation. If a distance between two control points appears longer than the specified length, the trajectory between the points is considered straight. Actually, two independent splines are built before and after the segment.
axes	Array of axis constants. Each element specifies one involved axis: Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.
velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.

exceptionAngle	If MotionFlags.ACSC_AMF_NURBS_EXCEPTION_ANGLE flag was specified, this argument defines ExceptionAngle. Set this argument to GeneralDefinition.ACSC_NONEif not used.
exceptionLength	If MotionFlags.ACSC_AMF_NURBS_EXCEPTION_LENGTH flag was specified, this argument defines ExceptionLength. Set this argument to GeneralDefinition.ACSC_NONEif not used.
motionDelay	If MotionFlags.ACSC_AMF_DELAY_MOTION flag has been specified, this argument defines the bit motion delay. Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles or 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Set this argument to GeneralDefinition.ACSC_NONEif not used
segments	String that contains the name of a one-dimensional user-defined array used to store added segments. Set this argument to None if not used. By default, if this argument is not specified, the controller allocates internal buffer for storing 50 segments only. The argument allows the user application to reallocate the buffer for storing a larger number of segments. The larger number of segments may be required if the application needs to add many very small segments in advanced. For most applications, the internal buffer size is enough and should not be enlarged. The buffer is for the controller internal use only and should not be used by the user application. The buffer size calculation rule: each segment requires about 600 bytes, so if it is necessary to allocate the buffer for 200 segments, it should be at least 600 * 200 = 120,000 bytes. The following declaration defines a 120,000 bytes buffer: real buf (15000). See **XARRSIZE** explanation in the *ACSPL+ Command and Variable Reference Guide* for details on how to declare a buffer with more than 100,000 elements
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

5.25.2 NurbsPoints

Description

The function adds a new control point to the NURBS motion generator.

Syntax

NurbsPoints(handle, flags, axes, point, velocity, required_vel, knot, weight, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_VELOCITY: the motion will use the velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_REQUIRED_VELOCITY: Specify required velocity. The suffix is not compatible with ACSC_AMF_VELOCITY. The flag requires a non-zero value in the parameter that specifies required velocity. The value is considered required velocity at the current point, but does not change required velocity for subsequent points. MotionFlags.ACSC_AMF_CORNER: Mark the current point as a corner. The control point is processed as a corner. Actually, such point divides the spline into two independent splines. MotionFlags.ACSC_AMF_DUMMY: Mark the point specification as dummy. Dummy point specifications can either precede the first control point specification, or follow the last control point specification. Dummy points specification is required in rare cases where default calculation of starting/trailing knots is not suitable (see Dummy point specification). MotionFlags.ACSC_AMF_WEIGHT: Specify control point weight. The suffix requires additional parameter that specifies the weight of the control point. MotionFlags.ACSC_AMF_KNOT: Specify knot delta. The suffix requires additional parameter that specifies knot delta from the previous knot. The new knot is calculated as the previous knot plus delta.

axes	Array of axis constants. Each element specifies one involved axis: Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.
point	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the final –1 element.
velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
required_vel	If MotionFlags.ACSC_AMF_REQUIRED_VELOCITY flag was specified, this argument specifies a Required velocity in the current point. Set this argument to GeneralDefinition.ACSC_NONE if not used.
knot	If MotionFlags.ACSC_AMF_KNOT flag was specified, this argument specifies a knot delta from the previous knot. Set this argument to GeneralDefinition.ACSC_NONE if not used
weight	If MotionFlags.ACSC_AMF_WEIGHT flag has been specified, this argument defines the weight of the control point. Set this argument to GeneralDefinition.ACSC_NONE if not used
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments



KNOTS and WEIGHTS parameters should be properly calculated for NURBS. Using random values can cause unexpected behavior of the NURBS algorithm.

5.25.3 SmoothPathMotion

Description

Smooth path is a motion type that accepts line segments and generates a spline motion between the specified points.
The motion deviates from exact coordinates after interpolation.
To control the deviation, the user can generate more points along the trajectory or use the Corner specification.

Syntax

SmoothPathMotion(handle, flags, axes, point, velocity, exceptionAngle, exceptionLength, motionDelay, segments, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_VELOCITY: the motion will use the velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_DELAY_MOTION: Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The switch requires an additional parameter that specify the motion delay. MotionFlags.ACSC_AMF_NURBS_CONSIDER_ACCELERATION: Acceleration consideration. Allow the motion generator to deviate from specified axes acceleration parameter during velocity profile generation. MotionFlags.ACSC_AMF_NURBS_EXCEPTION_ANGLE: The suffix requires additional parameter that specifies maximum angle in a control point. The value defines exceptions from spline interpolation. If for an internal control point, directions to the previous and the next control points require direction change more than the specified angle (by modulo), the control point is processed as a corner. Actually, defining such a point divides the spline into two independent splines. MotionFlags.ACSC_AMF_NURBS_EXCEPTION_LENGTH: Specify exception length. The suffix requires an additional parameter that specifies maximum segment length. The value defines exceptions from spline interpolation. If a distance between two control points appears longer than the specified length, the trajectory between the points is considered straight. Actually, two independent splines are built before and after the segment.
point	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the final -1 element.
axes	Array of axis constants. Each element specifies one involved axis: Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.

exceptionAngle	If MotionFlags.ACSC_AMF_NURBS_EXCEPTION_ANGLE flag was specified, this argument defines ExceptionAngle. Set this argument to GeneralDefinition.ACSC_NONE if not used.
exceptionLength	If MotionFlags.ACSC_AMF_NURBS_EXCEPTION_LENGTH flag was specified, this argument defines ExceptionLength. Set this argument to GeneralDefinition.ACSC_NONE if not used.
motionDelay	If MotionFlags.ACSC_AMF_DELAY_MOTION flag has been specified, this argument defines the bit motion delay. Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles or 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Set this argument to GeneralDefinition.ACSC_NONE if not used
segments	String that contains the name of a one-dimensional user-defined array used to store added segments. Set this argument to None if not used. By default, if this argument is not specified, the controller allocates internal buffer for storing 50 segments only. The argument allows the user application to reallocate the buffer for storing a larger number of segments. The larger number of segments may be required if the application needs to add many very small segments in advanced. For most applications, the internal buffer size is enough and should not be enlarged. The buffer is for the controller internal use only and should not be used by the user application. The buffer size calculation rule: each segment requires about 600 bytes, so if it is necessary to allocate the buffer for 200 segments, it should be at least 600 * 200 = 120,000 bytes. The following declaration defines a 120,000 bytes buffer: real buf (15000). See **XARRSIZE** explanation in the *ACSPL+ Command and Variable Reference Guide* for details on how to declare a buffer with more than 100,000 elements
wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

None

Get extended error information by calling GetLastError.

5.25.4 SmoothPathSegment

Description

Smooth path is a motion type that accepts line segments and generates a spline motion between the specified points.
The motion deviates from exact coordinates after interpolation.
To control the deviation, the user can generate more points along the trajectory or use the Corner specification.

Syntax

SmoothPathSegment(handle, flags, axes, point, velocity, required_vel, wait, failure_check=False)

Arguments

handle	Communication handle.

flags	Bit-mapped parameter that can include one or more of the following flags: MotionFlags.ACSC_AMF_WAIT: plan the motion but don't start it until the function Go is executed. MotionFlags.ACSC_AMF_VELOCITY: the motion will use the velocity specified for each segment instead of the default velocity. MotionFlags.ACSC_AMF_DELAY_MOTION: Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The switch requires an additional parameter that specify the motion delay. MotionFlags.ACSC_AMF_ENDVELOCITY: This flag requires additional parameter that specifies end velocity. The controller decelerates to the specified velocity in the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions. MotionFlags.ACSC_AMF_MAXIMUM Use maximum velocity under axis limits. With this suffix, no required velocity should be specified. The required velocity is calculated for each segment individually on the base of segment geometry and axis velocities (VEL values) of the involved axes. MotionFlags.ACSC_AMF_JUNCTIONVELOCITY: Decelerate to corner. This flag requires additional parameter that specifies corner velocity. The controller detects corner on the path and decelerates to the specified velocity before the corner. The specified value should be less than the required velocity; otherwise the parameter is ignored. If ACSC_AMF_JUNCTIONVELOCITY flag is not specified while ACSC_AMF_ ANGLE flag is specified, zero value of corner velocity is assumed. If neither the ACSC_AMF_JUNCTIONVELOCITY nor the ACSC_ AMF_ANGLE flags are specified, the controller provides automatic calculation as described in Automatic corner processing.

MotionFlags.ACSC_AMF_ANGLE:
Do not treat junction as a corner, if junction angle is less than or equal to the specified value in radians.
This flag requires additional parameter that specifies negligible angle in radians.
If ACSC_AMF_ANGLE flag is not specified while ACSC_AMF_ JUNCTIONVELOCITY flag is specified, the controller accepts default value of 0.01 radians, about 0.57 degrees.
If neither the ACSC_AMF_JUNCTIONVELOCITY nor the ACSC_ AMF_ANGLE flags are specified, the controller provides automatic calculation as described in Automatic corner processing.

MotionFlags.ACSC_AMF_AXISLIMIT
Enable velocity limitations under axis limits.
With this flag set, setting the ACSC_AMF_VELOCITY flag will result in the requested velocity being restrained by the velocity limits of all involved axes.

MotionFlags.ACSC_AMF_CURVEVELOCITY

Decelerate to curvature discontinuity point.
This flag requires an additional parameter that specifies velocity at curvature discontinuity points.
Curvature discontinuity occurs in linear-to-arc or arc to-arc smooth junctions.
If the flag is not set, the controller does not decelerate to smooth junction disregarding curvature discontinuity in the junction.
If the flag is set, the controller detects curvature discontinuity points on the path and provides deceleration to the specified velocity.
The specified value should be less than the required velocity; otherwise the parameter is ignored.
The flag can be set together with flags ACSC_AMF_ JUNCTIONVELOCITY and/or ACS_AMF_ANGLE.
If neither of ACSC_AMF_JUNCTIONVELOCITY, ACS_AMF_ ANGLE or ACSC_ AMF_CURVEVELOCITY is set, the controller provides automatic calculation of the corner processing.

MotionFlags.ACSC_AMF_CURVEAUTO
If the Flag is specified the controller provides automatic calculations as described in Enhanced automatic corner and curvature discontinuity points processing.

MotionFlags.ACSC_AMF_CORNERDEVIATION
Use a corner rounding option with the specified permitted deviation. This flag requires an additional parameter that specifies maximal allowed deviation of motion trajectory.

axes	Array of axis constants. Each element specifies one involved axis: Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.
point	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the final –1 element.
velocity	If MotionFlags.ACSC_AMF_VELOCITY flag was specified, this argument specifies a motion velocity for current segment. Set this argument to GeneralDefinition.ACSC_NONE if not used.
required_vel	If MotionFlags.ACSC_AMF_REQUIRED_VELOCITY flag was specified, this argument specifies a Required velocity in the current point. Set this argument to GeneralDefinition.ACSC_NONE if not used.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

5.26 Points and Segments Manipulation Functions

5.26.1 AddPoint

Description

The function adds a point to a single-axis multi-point or spline motion.

Syntax

AddPoint(handle, axis, point, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

point	Coordinate of the added point.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function adds a point to a single-axis multi-point or spline motion. To add a point to a multi-axis motion, use AddPVPointM. To add a point with a specified non-default velocity or time interval use AddPVPoint or **AddPVPointM**.

The function can wait for the controller response or can return immediately as specified by the wait argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.26.2 AddPointM

Description

The function adds a point to a single-axis multi-point or spline motion.

Syntax

AddPointM(handle, axes, point, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. One additional element must be located that contains -1 which marks the end of the array.

point	Coordinate of the added point.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function adds a point to a multi-axis multi-point or spline motion. To add a point to a single-axis motion, use AddPoint. To add a point with a specified non-default velocity or time interval use ExtAddPoint or ExtAddPointM.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

All axes specified in the axes array must be specified before the call of the MultiPointM or SplineM functions. The number and order of the axes in the `axes` array must correspond exactly to the number and order of the axes of MultiPointM or SplineM functions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.26.3 ExtAddPoint

Description

The function adds a point to a single-axis multi-point or spline motion and specifies a specific velocity or motion time.

Syntax

ExtAddPoint(handle, axis, point, rate, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
point	Coordinate of the added point.
rate	If the motion was activated by the MultiPoint function with the MotionFlags.ACSC_AMF_VELOCITY flag, this parameter defines the motion velocity. If the motion was activated by the Spline function with the MotionFlags.ACSC_AMF_VARTIME flag, this parameter defines the time interval between the previous point and the present one.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function adds a point to a single-axis multi-point motion with specific velocity or to single-axis spline motion with a non-uniform time.

To add a point to a multi-axis motion, use ExtAddPointM. To add a point to a motion with default velocity or uniform time interval, the AddPoint and AddPointM functions are more convenient.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.26.4 ExtAddPointM

Description

The function adds a point to a multi-axis multi-point or spline motion and specifies a specific velocity or motion time.

Syntax

ExtAddPointM(handle, axes, point, rate, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Array of axis constants. Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array. Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
point	Coordinate of the added point.
rate	If the motion was activated by the MultiPoint function with the MotionFlags.ACSC_AMF_VELOCITY flag, this parameter defines the motion velocity. If the motion was activated by the Spline function with the MotionFlags.ACSC_AMF_VARTIME flag, this parameter defines the time interval between the previous point and the present one.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function adds a point to a multi-axis multi-point or spline motion. To add a point to a single-axis motion, use ExtAddPoint. To add a point to a motion with a default velocity or a uniform time

interval, the ExtAddPointM function is more convenient.

The function can wait for the controller response or can return immediately as specified by the Wait argument.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this function periodically in a try-exception blocks until it doesn't raise an exception when failure_check variable is True.

All axes specified in the Axes array must be specified before the call of the **MultiPointM** or **SplineM** function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of MultiPointM or SplineM functions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.26.5 EndSequence

Description

The function informs the controller, that no more points will be specified for the current single-axis motion.

Syntax

EndSequence(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The motion finishes when the **EndSequence** function is executed. If the call of **EndSequence** is omitted, the motion will stop at the last point of the sequence and wait for the next point. No

transition to the next motion in the motion queue will occur until the **EndSequence** function executes.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

This function applies to the single-axis multi-point or spline (arbitrary path) motions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.26.6 EndSequenceM

Description

The function informs the controller, that no more points or segments will be specified for the current multi-axis motion.

Syntax

EndSequenceM(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axes	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.One additional element must be included that contains -1 which marks the end of the array.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The motion finishes when the EndSequenceM function is executed. If the call of **EndSequenceM** is omitted, the motion will stop at the last point or segment of the sequence and wait for the next point. No transition to the next motion in the motion queue will occur until the **EndSequenceM** function executes.

The function can wait for the controller response or can return immediately as specified by the `wait` argument.

This function applies to the multi-axis multi-point, spline (arbitrary path) and segmented motions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.27 Data Collection Functions

5.27.1 DataCollectionExt

Description

The function initiates data collection.

Syntax

DataCollectionExt(handle, flags, axis, array, nSample, period, vars, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include one or more of the following flags: DataCollectionFlags.ACSC_DCF_SYNC: Start data collection synchronously with a motion. DataCollectionFlags.ACSC_DCF_WAIT: Create the synchronous data collection, but do not start until the Go function is called. This flag can only be used with the ACSC_DCF_ SYNC flag. DataCollectionFlags.ACSC_DCF_TEMPORAL: Temporal data collection, the sampling period is calculated automatically according to the collection time. DataCollectionFlags.ACSC_DCF_CYCLIC: Cyclic data collection uses the collection array as a cyclic buffer and continues indefinitely. When the array is full, each new sample overwrites the oldest sample in the array.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
array	String contained the name of the array that stores the collected samples. The array must be declared as a global variable by an ACSPL+ program or by the DeclareVariablefunction.
nSample	Number of samples to be collected.
period	Sampling period in milliseconds. If the DataCollectionFlags.ACSC_DCF_TEMPORAL flag is specified, this argument defines a minimal period.

vars	Variable list - String. The string contains chained names of the variables, separated by '\r'(13) character. The values of these variables will be collected in the Array. If variable name specifies an array, the name must be supplemented with indexes in order to specify one element of the array.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

When this function starts data collection without the ACSC_DCF_SYNC flag the process is called system data collection. Data collection started with the ACSC_DCF_SYNC flag is called axis data collection. Data collection started with the ACSC_DCF_CYCLIC flag is called cyclic data collection.

Unlike the standard data collection that finishes when the collection array is full, cyclic data collection does not self-terminate. Cyclic data collection uses the collection array as a cyclic buffer and can continue to collect data indefinitely. When the array is full, each new sample overwrites the oldest sample in the array. Cyclic data collection can only be terminated by calling the StopCollect function.

The array that stores the samples can have one or two dimensions. A one-dimensional array is allowed only if the variable list contains one variable name.

The number of array rows must be equal to or more than the number of variables in the variable list. The number of the array columns must be equal to or more than the number of samples specified by the `nSample` argument.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.27.2 StopCollect

Description

The function terminates data collection.

Syntax

StopCollect(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The usual system data collection finishes when the required number of samples is collected or the **StopCollect** function is executed. The application can wait for data collection end with the WaitCollectEndExt function.

Temporal data collection runs until the **StopCollect** function is executed. The function terminates the data collection prematurely. The application can determine the number of actually collected samples from the S_DCN variable and the actual sampling period from the S_DCP variable.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.27.3 WaitCollectEndExt

Description

The function waits for the end of data collection.

Syntax

WaitCollectEndExt(handle, timeout, axis, failure_check=False)

Arguments

handle	Communication handle.
timeout	Maximum wait time in milliseconds, If -1 - timeout interval never elapses.
axis	If using axis data collection – the axis number, otherwise GeneralDefinition.ACSC_NONE. Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1 etc.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If using Axis data collection (data collection is synced to axis) set `axis` to be the axis number you are collecting data from, otherwise set `axis` to be ACSC_NONE.

The function does not return while data collection is in progress and the correct parameters have been passed.

Verifies AST.#DC flag for Synced data collection, otherwise verifies S_ST.#DC system flag.
Using the function with the wrong parameters (i.e. calling the function with an axis number while performing a system data collection) will result in undefined behaviour.

5.28 Status Report Functions

5.28.1 GetMotorState

Description

The function retrieves the current motor state.

Syntax

GetMotorState(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	If using axis data collection – the axis number, otherwise GeneralDefinition.ACSC_NONE Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The return int variable representing the 32-bit of the ACSPL+ variable MST.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.28.2 GetAxisState

Description

The function retrieves the current axis state.

Syntax

GetAxisState(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	If using axis data collection – the axis number, otherwise GeneralDefinition.ACSC_NONE Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Comments

The return int variable representing the 32-bit of the ACSPL+ variable AST.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.28.3 GetIndexState

Description

The function retrieves the current set of bits that indicate the index and mark state.

Syntax

GetIndexState(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	If using axis data collection – the axis number, otherwise GeneralDefinition.ACSC_NONE Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The return int variable representing the 32-bit of the ACSPL+ variable IST.

The function retrieves the current set of bits that indicate the index and mark state.
The controller processes index/mark signals as follows:
When an index/mark signal is encountered for the first time, the controller latches feedback positions and raises the corresponding bit. As long as a bit is raised, the controller does not latch the feedback position even if the signal occurs again. To resume latching logic, the application must call the ResetIndexState function to explicitly reset the corresponding bit.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.28.4 ResetIndexState

Description

The function resets the specified bit of the index/mark state.

Syntax

ResetIndexState(handle, axis, mask, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	If using axis data collection – the axis number, otherwise GeneralDefinition.ACSC_NONE Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
mask	The parameter indicates the bit to be cleared. Only one of the following flags can be specified: IndexStates.ACSC_IST_IND – a primary encoder index of the specified axis is latched. IndexStates.ACSC_IST_IND2 – a secondary encoder index of the specified axis is latched. IndexStates.ACSC_IST_MARK – a MARK1 signal has been generated and position of the specified axis was latched. IndexStates.ACSC_IST_MARK2 – a MARK2 signal has been generated and position of the specified axis was latched
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The method resets the specified bit of the index/mark state. `mask` contains a bit that must be cleared, i.e., the method resets only that bit of the index/mark state, which corresponds to non- zero bit of `mask`. To get the current index/mark state, use the GetIndexState method. If the method fails, the Error object is filled with the Error Description.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.28.5 GetProgramState

Description

The function retrieves the current state of the program buffer.

Syntax

GetProgramState(handle, buffer, wait, failure_check=False)

Arguments

handle	Communication handle.
buffer	Number of the buffer.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The return int variable represents the 32-bit of the ACSPL+ variable IST.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29 Input/Output Access Functions

5.29.1 GetInput

Description

The function retrieves the current state of the specified digital input.

Syntax

GetInput(handle, port, bit, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the input port.

bit	Number of the specific bit.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The function retrieves the current state of the specified digital input. To get values of all inputs of the specific port, use the GetInputPort function.
Digital inputs are represented in the controller variable **IN**. For more information about digital inputs, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.2 GetInputPort

Description

The function retrieves the current state of the specified digital input port.

Syntax

GetInputPort(handle, port, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the input port.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

Digital inputs are represented in the controller variable **IN**. For more information about digital inputs, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.3 GetOutput

Description

The function retrieves the current state of the specified digital output.

Syntax

GetOutput(handle, port, bit, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the output port.
bit	Number of the specific bit.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Comments

The function retrieves the current state of the specified digital output. To get values of all outputs of the specific port, use the GetOutputPort function.

Digital outputs are represented in the controller variable **OUT**. For more information about digital outputs, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.4 GetOutputPort

Description

The function retrieves the current state of the specified digital output port.

Syntax

GetOutputPort(handle, port, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the output port.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Comments

The function retrieves the current state of the specified digital output port. To get the value of the specific output of the specific port, use the GetOutput function.

Digital outputs are represented in the controller variable **OUT**. For more information about digital outputs, see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.5 GetAnalogInputNT

Description

The function sets the specified digital output port to the specified value.

Syntax

GetAnalogInputNT(handle, port, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Index of the analog inputs port.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type

Variable that receives the current value of a specific analog input. Units: in scaling, by percent, of the signal and ranges from -100 to +100 of the maximum level.

Comments

The function retrieves the current value of the specified analog input signal from an external source such as a sensor or a potentiometer.

Analog inputs are represented in the controller variable **AIN**. For more information about analog inputs, see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.6 GetAnalogOutputNT

Description

The function retrieves the current value of the specified analog output signal that is sent to an external device such as a sensor or a potentiometer.

Syntax

GetAnalogOutputNT(handle, port, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Index of the output port.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

float type

Variable that receives the current value of a specific analog output. Units: in scaling, by percent, of the signal and ranges from -100 to +100 of the maximum level.

Comments

The function retrieves the current value of the specified analog output signal that is sent to an external device such as a sensor or a potentiometer. To write a value to the specific analog output, use the SetAnalogOutputNT function.

Analog outputs are represented in the controller variable **AOUT**. For more information about analog outputs, see *SPiiPlus ACSPL+ Programmer's Guide*

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.7 SetAnalogOutputNT

Description

The function writes the specified value to the specified analog output signal that is sent to an external device such as a sensor or a potentiometer.

Syntax

SetAnalogOutputNT(handle, port, value, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Index of the output port.
value	The value is written to the specified analog output as a percentage (range is -100% to 100%) of the maximum level.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function writes the specified value to the specified analog output signal that is sent to an external device such as a sensor or a potentiometer. To get a value of the specific analog output, use the GetAnalogOutputNT function.

Analog outputs are represented in the controller variable **AOUT**. For more information about analog outputs, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.8 GetExtInput

Description

The function retrieves the current state of the specified extended input.

Syntax

GetExtInput(handle, port, bit, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the extended input port.
bit	Number of the specific bit.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

The function retrieves the current state (0 or 1) of the extended input specified by port and bit.

Get extended error information by calling GetLastError.

Comments

The function retrieves the current state of the specified extended input. To get values of all inputs of the specific extended port, use the GetExtInputPort function.

Extended inputs are represented in the controller variable **EXTIN**. For more information about extended inputs, see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.9 GetExtInputPort

Description

The function retrieves the current state of the specified extended input port.

Syntax

GetExtInputPort(handle, port, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the extended input port.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The function retrieves the current state of the specified extended input port. To get the value of the specific input of the specific extended port, use the GetExtInput function.

Extended inputs are represented in the controller variable **EXTIN**. For more information about extended inputs, see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.10 GetExtOutput

Description

The function retrieves the current state of the specified extended output.

Syntax

GetExtOutput(handle, port, bit, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the extended output port.
bit	Number of the specific bit.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

The function retrieves the current state (0 or 1) of the specified extended output.

Get extended error information by calling GetLastError.

Comments

The function retrieves the current state of the specified extended output. To get values of all outputs of the specific extended port, use the GetExtInputPort function.

Extended outputs are represented in the controller variable **EXTOUT**. For more information about extended outputs, see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.11 GetExtOutputPort

Description

The function retrieves the current state of the specified extended output port.

Syntax

GetExtOutputPort(handle, port, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the extended output port.
bit	Number of the specific bit.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

The function retrieves the current state (0 or 1) of the specified extended output port.

Get extended error information by calling GetLastError.

Comments

The function retrieves the current state of the specified extended output port. To get the value of the specific output of the specific extended port, use the GetExtOutputPort function.

Extended outputs are represented in the controller variable **EXTOUT**. For more information about extended outputs, see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.12 SetExtOutput

Description

The function retrieves the current state of the specified extended output port.

Syntax

SetExtOutput(handle, port, bit, value, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the extended output port.
bit	Number of the specific bit.
value	The value to be written to the specified output. Any non-zero value is interpreted as 1.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function sets the specified extended output to the specified value. To set values of all outputs of the specific extended port, use the SetExtOutputPort function.

Extended outputs are represented in the controller **EXTOUT** variable. For more information about extended outputs, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.29.13 SetExtOutputPort

Description

The function sets the specified extended output port to the specified value.

Syntax

SetExtOutputPort(handle, port, value, wait, failure_check=False)

Arguments

handle	Communication handle.
port	Number of the extended output port.

value	The value written to the specified output port.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function sets the specified extended output port to the specified value. To set the value of the specific output of the specific extended port, use the SetExtOutput function.

Extended outputs are represented in the controller variable **EXTOUT**. For more information about extended outputs, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30 Safety Control Functions

5.30.1 GetFault

Description

The function retrieves the set of bits that indicate the motor or system faults.

Syntax

GetFault(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception.If False the function will ignore detected errors .

Return Value

int type.

Get extended error information by calling GetLastError.

Comments

The function retrieves the set of bits that indicate motor or system faults.
Motor faults are related to a specific motor, the power amplifier, and the Servo processor. For example: Position Error, Encoder Error, or Driver Alarm.
System faults are not related to any specific motor. For example: Emergency Stop or Memory Fault.
The return value stores set of bits that indicates the controller faults. To recognize the specific fault, constants ACSC_SAFETY_*** can be used. See For more information about the controller faults, see the SPiiPlus ACSPL+ Programmer's Guide.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.2 SetFaultMask

Description

The function sets the mask that enables or disables the examination and processing of controller faults.



Certain controller faults provide protection against potential serious bodily injury and damage to the equipment. Be aware of the implications before disabling any alarm, limit, or error.

Syntax

SetFaultMask(handle, axis, mask, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. GeneralDefinition.ACSC_NONEcorresponds to system.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type.

Get extended error information by calling GetLastError.

Comments

The function sets the mask that enables/disables the examination and processing of the controller faults. The two types of controller faults are motor faults and system faults.
The motor faults are related to a specific motor, the power amplifier or the Servo processor. For example: Position Error, Encoder Error or Driver Alarm.
The system faults are not related to any specific motor. For example: Emergency Stop or Memory Fault.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.3 GetFaultMask

Description

Certain controller faults provide protection against potential serious bodily injury and damage to the equipment. Be aware of the implications before disabling any alarm, limit, or error. The function sets the mask that enables or disables the examination and processing of the controller faults.

Syntax

GetFaultMask(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type.

Get extended error information by calling GetLastError.

Comments

The function retrieves the mask, defining which controller faults are examined and processed. If a bit of the parameter mask is zero, the corresponding fault is disabled.
The controller faults are of two types: motor faults and system faults.
The motor faults are related to a specific motor, the power amplifier, or the Servo processor. For example: Position Error, Encoder Error or Driver Alarm.
The system faults are not related to any specific motor, for example: Emergency Stop or Memory Fault.
For more information about the controller faults, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.4 EnableFault

Description

The function sets the mask that enables or disables the examination and processing of controller faults.

Syntax

EnableFault(handle, axis, fault, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. GeneralDefinition.ACSC_NONE corresponds to system.
fault	The fault to be enabled. Only one fault can be enabled at a time. To specify the fault, one of the constants ACSC_SAFETY_*** can be used.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Get extended error information by calling GetLastError.

Comments

The function enables the examination and processing of the specified motor or system fault by setting the specified bit of the fault mask to one.
The motor faults are related to a specific motor, the power amplifier, and the Servo processor. For example: Position Error, Encoder Error, and Driver Alarm.
The system faults are not related to any specific motor. For example: Emergency Stop, Memory Fault. For more information about the controller faults, see *SPiiPlus ACSPL+ Programmer's Guide*.
If the function fails, the Error object is filled with the Error Description.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.5 DisableFault

Description

The function disables the specified motor or system fault.

Syntax

DisableFault(handle, axis, fault, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
fault	The fault to be enabled. Only one fault can be enabled at a time. To specify the fault, one of the constants ACSC_SAFETY_*** can be used.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function disables the examination and processing of the specified motor or system fault by setting the specified bit of the fault mask to zero.
The motor faults are related to a specific motor, the power amplifier, and the Servo processor. For example: Position Error, Encoder Error, and Driver Alarm.
The system faults are not related to any specific motor, for example: Emergency Stop, Memory Fault.
For more information about the controller faults, see SPiiPlus ACSPL+ Programmer's Guide.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.6 SetResponseMask

Description

The function retrieves the mask that defines the motor or the system faults for which the controller provides the default response.

Syntax

SetResponseMask(handle, axis, mask, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
mask	The mask to be set. If a bit of Mask is zero, the corresponding default response is disabled. Use the ACSC_SAFETY_*** constants to set/reset a specified bit. If Mask is GeneralDefinition.ACSC_NONE , all default responses for the specified axis are enabled. If Mask is zero, all default responses for the specified axis are disabled.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function retrieves the mask that defines the motor or the system faults for which the controller provides the default response.
The default response is a controller-predefined action for the corresponding fault. For more information about the controller faults and default responses, see the *SPiiPlus ACSPL+ Programmer's Guide*.

If True when the function detects an error, it will raise an exception.

If False the function will ignore detected errors .

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.7 GetResponseMask

Description

The function retrieves the mask that defines the motor or the system faults for which the controller provides the default response.

Syntax

GetResponseMask(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type.

Get extended error information by calling GetLastError.

Comments

The function retrieves the mask that determines whether the controller will respond to a motor or system fault with the fault's default response.
For more information about the controller faults and default, responses see the SPiiPlus ACSPL+ Programmer's Guide.
If the function fails, the Error object is filled with the Error Description.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.8 DisableResponse

Description

The function disables the default response to the specified axis or system fault.

Certain controller faults provide protection against potential serious bodily injury and damage to the equipment. Be aware of the implications before disabling any alarm, limit, or error.

Syntax

DisableResponse(handle, axis, response, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
response	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. GeneralDefinition.ACSC_NONE corresponds to system.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function retrieves the mask that determines whether the controller will respond to a motor or system fault with the fault's default response.
For more information about the controller faults and default, responses see the SPiiPlus ACSPL+ Programmer's Guide.
If the function fails, the Error object is filled with the Error Description.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.9 GetSafetyInput

Description

The function retrieves the current state of the specified safety input.

Syntax

GetSafetyInput(handle, axis, input, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. GeneralDefinition.ACSC_NONE corresponds to system.
input	The specific safety input. To specify a desired motor safety input OR combination of any of the following constants can be used: SafetyControlMasks.ACSC_SAFETY_RL SafetyControlMasks.ACSC_SAFETY_LL SafetyControlMasks.ACSC_SAFETY_RL2 SafetyControlMasks.ACSC_SAFETY_LL2 SafetyControlMasks.ACSC_SAFETY_HOT SafetyControlMasks.ACSC_SAFETY_DRIVE SafetyControlMasks.ACSC_SAFETY_ES
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The function retrieves the current state of the specified safety input. To get values of all safety inputs of the specific axis, use the GetSafetyInput function.
Safety inputs are represented in the controller variables SAFIN and S_SAFIN. For more information about safety inputs, see SPiiPlus ACSPL+ Programmer's Guide.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Some safety inputs can be unavailable in a specific controller model. For example, SPiiPlus SA controller does not provide Motor Overheat, Preliminary Left Limit, and Preliminary Right Limit safety inputs.
See specific model documentation for details.

5.30.10 GetSafetyInputPort

Description

The function disables the default response to the specified axis or system fault.

Syntax

GetSafetyInputPort(handle, axis, wait, failure_check=False

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.GeneralDefinition.ACSC_NONE corresponds to system.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The method retrieves the current state of the specified safety input port.
To recognize a specific motor safety input, only one of the following properties can be used:

SafetyControlMasks.ACSC_SAFETY_RL

SafetyControlMasks.ACSC_SAFETY_LL

SafetyControlMasks.ACSC_SAFETY_NETWORK

SafetyControlMasks.ACSC_SAFETY_HOT

SafetyControlMasks.ACSC_SAFETY_DRIVE

The function retrieves the current state of the specified safety input port. To get the state of the specific safety input of a specific axis, use the GetSafetyInput function.
Safety inputs are represented in the controller variables SAFIN and S_SAFIN.
For more information about safety inputs, see SPiiPlus ACSPL+ Programmer's Guide.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Some safety inputs can be unavailable in a specific controller model. For example, SPiiPlus SA controller does not provide Motor Overheat, Preliminary Left Limit, and Preliminary Right Limit safety inputs.
See specific model documentation for details.

5.30.11 GetSafetyInputPortInv

Description

The function retrieves the set of bits that define inversion for the specified safety input port.

Syntax

GetSafetyInputPortInv(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

ine inversion for the specified safety input port. To recognize a specific bit, use the following properties

SafetyControlMasks.ACSC_SAFETY_RL

SafetyControlMasks.ACSC_SAFETY_LL

SafetyControlMasks.ACSC_SAFETY_NETWORK

SafetyControlMasks.ACSC_SAFETY_HOT

SafetyControlMasks.ACSC_SAFETY_DRIVE

Use the SafetyControlMasks.ACSC_SAFETY_ES property to recognize an inversion for the specific system safety input port. See Safety Control Masks for a detailed description of these properties.

The function retrieves the set of bits that define inversion for the specified safety input port. To set the specific inversion for the specific safety input port, use the GetSafetyInputPortInv function.
If a bit of the retrieved set is zero, the corresponding signal is not inverted and therefore high voltage is considered an active state.
If a bit is raised, the signal is inverted and low voltage is considered an active state. The inversions of safety inputs are represented in the controller variables SAFINI and S_SAFINI. For more information about safety inputs, see the SPiiPlus ACSPL+ Programmer's Guide.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Some safety inputs can be unavailable in a specific controller model. For example, the SPiiPlus SA controller does not provide Motor Overheat, Preliminary Left Limit, and Preliminary Right Limit safety inputs.
See specific model documentation for details.

5.30.12 SetSafetyInputPortInv

Description

The function sets the set of bits that define inversion for the specified safety input port.

Syntax

SetSafetyInputPortInv(handle, axis, value, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
value	The specific inversion. To set a specific bit, use the following constants: SafetyControlMasks.ACSC_SAFETY_RL SafetyControlMasks.ACSC_SAFETY_LL SafetyControlMasks.ACSC_SAFETY_RL2 SafetyControlMasks.ACSC_SAFETY_LL2 SafetyControlMasks.ACSC_SAFETY_HOT SafetyControlMasks.ACSC_SAFETY_DRIVE To set an inversion for the specific system safety input port, use only the ACSC_ SAFETY_ES constant. See Safety Control Masks for a detailed description of these constants.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function sets the bits that define inversion for the specified safety input port. To retrieve an inversion for the specific safety input port, use the GetSafetyInputPortInv function.
If a bit of the set is zero, the corresponding signal will not be inverted and therefore high voltage is considered an active state. If a bit is raised, the signal will be inverted and low voltage is considered an active state.
The inversions of safety inputs are represented in the controller variables SAFINI and S_SAFINI. For more information about safety, inputs see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.13 FaultClear

Description

The function clears the current faults and the result of the previous fault stored in the MERR variable.

Syntax

FaultClear(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.

value	The specific inversion. To set a specific bit, use the following constants: SafetyControlMasks.ACSC_SAFETY_RL SafetyControlMasks.ACSC_SAFETY_LL SafetyControlMasks.ACSC_SAFETY_RL2 SafetyControlMasks.ACSC_SAFETY_LL2 SafetyControlMasks.ACSC_SAFETY_HOT SafetyControlMasks.ACSC_SAFETY_DRIVE To set an inversion for the specific system safety input port, use only the ACSC_ SAFETY_ES constant. See Safety Control Masks for a detailed description of these constants.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function clears the current faults of the specified axis and the result of the previous fault stored in the **MERR** variable.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.30.14 FaultClearM

Description

The function clears the current faults and results of previous faults stored in the **MERR** variable for multiple axes.

Syntax

FaultClearM(handle, axes, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. one additional element must be located that contains -1 which marks the end of the array.
value	The specific inversion. To set a specific bit, use the following constants: SafetyControlMasks.ACSC_SAFETY_RL SafetyControlMasks.ACSC_SAFETY_LL SafetyControlMasks.ACSC_SAFETY_RL2 SafetyControlMasks.ACSC_SAFETY_LL2 SafetyControlMasks.ACSC_SAFETY_HOT SafetyControlMasks.ACSC_SAFETY_DRIVE To set an inversion for the specific system safety input port, use only the ACSC_ SAFETY_ES constant. See Safety Control Masks for a detailed description of these constants.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If the reason for the fault is still active, the controller will set the fault immediately after this command is performed. If cleared fault is Encoder Error, the feedback position is reset to zero.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.31 Wait-for-Condition Functions

5.31.1 WaitMotionEnd

Description

The function waits for the end of a motion.

Syntax

WaitMotionEnd(handle, axis, timeout, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc.
timeout	Maximum waiting time in milliseconds. If timeout is -1, the function's time-out interval never elapses.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function does not return while the specified axis is involved in a motion, the motor has not settled in the final position and the specified time-out interval has not elapsed.
The function differs from the WaitLogicalMotionEnd function. Examining the same motion, the WaitMotionEnd function will return latter. The WaitLogicalMotionEnd function returns when the generation of the motion finishes. On the other hand, the WaitMotionEnd function returns when the generation of the motion finishes and the motor has settled in the final position.

5.31.2 WaitLogicalMotionEnd

Description

The function waits for the logical end of a motion.

Syntax

WaitLogicalMotionEnd(handle, axis, timeout, failure_check=False)

Arguments

handle	Communication handle.
axis	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_ AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. The axis must be either a single axis not included in any group or a leading axis of a group.
timeout	Maximum waiting time in milliseconds. If timeout is -1, the function's time-out interval never elapses.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function does not return while the specified axis is involved in a motion and the specified timeout interval has not elapsed.
The function differs from the WaitMotionEnd function. Examining the same motion, the **WaitMotionEnd** function will return later. The **WaitLogicalMotionEnd** function returns when the generation of the motion finishes. On the other hand, the **WaitMotionEnd** function returns when the generation of the motion finishes and the motor has settled in the final position.

5.31.3 WaitForAsyncCall

Description



Not supported as of this version.

The function waits for completion of asynchronous call and retrieves a data.

Syntax

WaitForAsyncCall(handle, buf, wait, timeout, failure_check=False)

Arguments

handle	Communication handle
buf	The buffer that receives controller response. This parameter must be the same Buffer that was return by the asynchronous function. If the SPiiPlus Python function does not accept a buffer as a parameter, buf has to be None object
wait	It should be the identical wait object that was passed to the asynchronous SPiiPlus Python function.
timeout	Maximum waiting time in milliseconds. If timeout is -1, the function's timeout interval never elapses.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

If the function fails, the return value is zero. The Ret field of `wait` contains the error code that the non-waiting call caused.

If `wait.Ret` is zero, the call succeeded: no errors occurred, and the function should return the result of the asynchronous call.

Get extended error information by calling GetLastError.

Remarks

The function waits for completion of asynchronous call, corresponds to the `wait` parameter, and retrieves controller response to the buffer pointed by `buf`. The `wait` and `buf` parameters must be the same objects passed to SPiiPlus Python function when asynchronous call was initiated.

If the call to the SPiiPlus Python function was successful, the function retrieves controller response to the buffer `buf`.
If the call of SPiiPlus Python function does not return a response (for example: Enable, Jog, etc.) `buf` must be None.

If the call of SPiiPlus Python function returned an error, the function retrieves this error code in the `Ret` member of the wait parameter. If the SPiiPlus python function has not been completed in `Timeout` milliseconds, the function aborts specified asynchronous call and returns the ErrorCodes.ACSC_TIMEOUT error. If the call of SPiiPlus Python function has been aborted by the CancelOperation_ function, the function returns the ErrorCodes.ACSC_OPERATIONABORTED error.

5.31.4 WaitMotorEnabled

Description

The function waits for the specified state of the specified motor.

Syntax

WaitMotorEnabled(handle, axis, state, timeout, failure_check=False)

Arguments

handle	Communication handle.
axis	Array of axis constants. Each element specifies one involved axis: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc.
state	1 – the function wait for the motor enabled 0 – the function wait for the motor disabled.
timeout	Maximum waiting time in milliseconds. If timeout is -1, the function's timeout interval never elapses.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function does not return while the ACSPL+ program in the specified buffer is in progress and the specified time-out interval has not elapsed.

5.31.5 WaitInput

Description

The function waits for the specified state of digital input.

Syntax

WaitInput(handle, port, bit, state, timeout, failure_check=False)

Arguments

handle	Communication handle.
port	Number of input port: 0 corresponds to IN0, 1 – to IN1, etc.
bit	Selects one bit from the port, from 0 to 31.
state	Specifies a desired state of the input, 0 or 1.
timeout	Maximum waiting time in milliseconds. If timeout is -1, the function's timeout interval never elapses.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The basic configuration of the SPiiPlus PCI model provides only 16 inputs. Therefore, the `port` must be 0, and the `bit` can be set to any value from 0 to 15.

5.31.6 WaitUserCondition

Description

The function waits for the specified state of digital input.

Syntax

WaitUserCondition(handle, userCondition, timeout, failure_check=False)

Arguments

handle	Communication handle.
userCondition	Name of the function which is used as the condition.

timeout	Maximum waiting time in milliseconds. If timeout is -1, the function's timeout interval never elapses.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function calls the **UserCondition** function in order to determine the termination moment. While the **UserCondition** function returns zero and the specified time-out interval has not elapsed, the **WaitUserCondition** function calls **UserCondition** periodically. Once the **UserCondition** function returns non-zero results, the **WaitUserCondition** function also returns.
The **UserCondition** function accepts as argument the `handle` parameter, passed to the function **WaitUserCondition**. If the condition is satisfied, the **UserCondition** function returns 1, if it is not satisfied – 0. In case of an error the UserCondition function returns –1.

Example of user condition function in Windows:

```
@ctypes.WINFUNCTYPE(ctypes.c_int, ctypes.c_int)
 user_cond_func(hc)
    print("in user condition function")
    time.sleep(1)
  return 1
```

5.31.7 WaitMotorCommutated

Description

The function waits for the specified motor to be commutated.

Syntax

WaitMotorCommutated(handle, axis, state, timeout, failure_check=False)

Arguments

handle	Communication handle.
userCondition	Name of the function which is used as the condition.
timeout	Maximum waiting time in milliseconds. If timeout is -1, the function's timeout interval never elapses.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function does not return while the specified motor is not in the desired state and the specified time-out interval has not elapsed. The function examines the **MFLAGS**.#BRUSHOK flag.

5.32 Callback Registration Functions

5.32.1 InstallCallback

Description

The function installs a user-defined callback function for the specified interrupt condition with userdefined parameter.

Syntax

InstallCallback(handle, callback, cardContext, interrupt, failure_check=False)

Arguments

handle	Communication handle.
callback	A user-defined function that accepts an argument of integer type and returns the result of integer type. If Callback is None, the function resets previous installed callback for the corresponding Interrupt. If Callback has no functionality and is set only for corresponding Waitxxx function, this parameter may be set as dummy_callback_ext. The callback function is defined as : @ctypes.WINFUNCTYPE(ctypes.c_int, ctypes.c_uint64, ctypes.POINTER (ctypes.c_int)) def callback_example(param, p): # function block
cardContext	Additional parameter that will be passed to the callback function.
interrupt	Number specifies an interrupt. Use the Interrupt class.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function installs a user-defined callback function `callback` for the specified interrupt condition `interrupt`.
The library calls the `callback` function when the specified interrupt occurs. The bit-mapped parameter `param` of the function `callback` identifies for which axis/buffer/input the interrupt was generated.

One callback can be installed for each interrupt, for same communication channel. The library creates a separate thread for each interrupt. Therefore, each callback function is called in a separate thread so that the callbacks do not delay one another.

User on Callback registration defines the parameter `userParameter`. It is especially useful when several SPiiPlus cards are used. That allows setting the same function as a Callback on a specific interrupt, for all the cards. Inside that function user can see by the `userParameter`, which card caused the interrupt.

To uninstall a specific callback, call the function `InstallCallback` with the parameter Callback equals to None and the parameter Interrupt equals the specified interrupt type. This action will uninstall the callback for specified communication channel.

5.32.2 InstallCallbackExt

Description

The function installs a user-defined callback function for the specified interrupt condition with user defined parameter, with extension to 128 axes.

Syntax

InstallCallbackExt(handle, callback, cardContext, interrupt, failure_check=False)

Arguments

handle	Communication handle.
callback	A user-defined function that accepts an argument of integer type and returns the result of integer type. If Callback is None, the function resets previous installed callback for the corresponding Interrupt. If Callback should have no functionality and is set only for corresponding Waitxxx function, this parameter may be set as dummy_callback_ext. The callback function is defined as : @ctypes.WINFUNCTYPE(ctypes.c_int, SPiiPlusPython.AXMASK_EXT_c, ctypes.POINTER(ctypes.c_int)) def callback_example(param, p): # function block
cardContext	Additional parameter that will be passed to the callback function.
interrupt	Number specifies an interrupt. Use the Interrupt class.

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

This function extends the functionality of InstallCallback work with up to 128 axes.

5.32.3 SetCallbackMask

Description

The function sets the mask for specified callback.

Syntax

SetCallbackMask(handle, interrupt, mask, failure_check=False)

Arguments

handle	Communication handle.
interrupt	See Callback Interrupts.
mask	The mask to be set. If some bit equals to 0 then interrupt for corresponding axis/buffer/input does not occur – interrupt is disabled. Use AxisMasks.ACSC_MASK_*** constants to set/reset a specified bit. See Callback Interrupt Masks for a detailed description of these constants. If mask is GeneralDefinition.ACSC_NONE, the interrupts for all axes/buffers/inputs are enabled. If mask is 0, the interrupts for all axes/buffers/inputs are disabled. As default all bits for each interrupts are set to one.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function sets the bit mask for specified interrupt. To get current mask for specified interrupt, call GetCallbackMask.
Using a mask, you can reduce the number of calls of your callback function. The callback will be called only if the interrupt is caused by an axis/buffer/input that corresponds to non-zero bit in the related mask.

5.32.4 SetCallbackMaskExt

Description

The function sets the mask for the specified callback, up to 128 axes.

Syntax

SetCallbackMaskExt(handle, interrupt, mask, failure_check=False)

Arguments

handle	Communication handle.
interrupt	See Callback Interrupts.
mask	The mask to be set. If some bit equals to 0 then interrupt for corresponding axis/buffer/input does not occur – interrupt is disabled. Use AxisMasks.ACSC_MASK_*** constants to set/reset a specific bit. If Mask.AXMASK64 is GeneralDefinition.ACSC_NONE, the interrupts for all 0..63 axes/buffers/inputs are enabled. If Mask.AXMASK128 is GeneralDefinition.ACSC_NONE, the interrupts for all 64..127 axes are enabled. If Mask.AXMASK64 is 0, the interrupts for all 0..63 axes/buffers/inputs are disabled. If Mask.AXMASK128 is 0, the interrupts for axes 64..127 axes are disabled. As default all bits for each interrupt are set to one.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The function sets the bit mask for specified interrupt. To get current mask for specified interrupt, call GetCallbackMaskExt.

5.32.5 GetCallbackMask

Description

The function retrieves the mask for specified interrupt.

Syntax

GetCallbackMask(handle, interrupt, failure_check=False)

Arguments

handle	Communication handle.
interrupt	See Callback Interrupts.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type representing a 64-bit mask

Get extended error information by calling GetLastError.

Comments

The function retrieves the bit mask for the specified interrupt. To set the mask for specified interrupt, use SetCallbackMask.

5.32.6 GetCallbackMaskExt

Description

The function retrieves the mask for specified interrupt, up to 128 axes.

Syntax

GetCallbackMaskExt(handle, interrupt, failure_check=False)

Arguments

handle	Communication handle.
interrupt	See Callback Interrupts.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

type AXMASK_EXT

Get extended error information by calling GetLastError.

Comments

The function retrieves the bit mask for specified interrupt. To set the current mask for the specified interrupt, call SetCallbackMaskExt.

5.32.7 SetCallbackPriority

Description



The function can be used only with the PCI communication channel.

The function sets the priority for all callback threads.

Syntax

SetCallbackPriority(handle, priority, failure_check=False)

Arguments

handle	Communication handle.
priority	Specifies the priority value for the callback thread. This parameter can be one of the operating system defined priority levels. For example the Win32 API defines the following priorities: THREAD_PRIORITY_ABOVE_NORMAL THREAD_PRIORITY_BELOW_NORMAL THREAD_PRIORITY_HIGHEST THREAD_PRIORITY_IDLE THREAD_PRIORITY_LOWEST THREAD_PRIORITY_NORMAL THREAD_PRIORITY_TIME_CRITICAL As default, all callback threads have a normal priority.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

The operating system uses the priority level of all executable threads to determine which thread gets the next slice of CPU time. Basically, threads are scheduled in a round-robin fashion at each priority level, and only when there are no executable threads at a higher level does scheduling of threads at a lower level take place.

When manipulating priorities, be very careful to ensure that a high-priority thread does not consume all of the available CPU time. See the relevant operating system guides for details.

5.33 Variables Management Functions

5.33.1 DeclareVariable

Description

The function creates the persistent global variable.

Syntax

DeclareVariable(handle, type, name, wait, failure_check=False)

Arguments

handle	Communication handle
type	Type of the variable. For an integer variable the type parameter must be 1. For a real variable, the type parameter must be 2. For a static integer variable the type parameter must be 3. For a static real variable, the type parameter must be 4.

name	String contained name of the variable.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None.

Comments

The function creates the persistent global variable specified by the parameter name of type specified by the parameter type. The variable can be used as any other standard or global variable.

If it is necessary to declare a one or two-dimensional array, the parameter Name should also contains the dimensions in brackets.

The lifetime of a persistent global variable is not connected with any program buffer. The persistent variable survives any change in the program buffers and can be erased only by the ClearVariables function.

The function can wait for the controller response or can return immediately as specified by the Wait argument.

If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.

5.33.2 ClearVariables

Description

The function creates the persistent global variable.

Syntax

ClearVariables(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

The function deletes all persistent global variables created by the DeclareVariable function. The function can wait for the controller response or can return immediately as specified by the *wait* argument.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.34 Service Functions

5.34.1 GetFirmwareVersion

Description

The function retrieves the firmware version of the controller.

Syntax

GetFirmwareVersion(handle, count, wait, failure_check=False)

Arguments

handle	Communication handle.
count	Max length of the return string.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

String type

Get extended error information by calling GetLastError.

Comments

The function retrieves the character string that contains the firmware version of the controller. The function will not copy more than `count` characters to the return string. If the buffer is too small, the firmware version may be truncated.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.34.2 GetSerialNumber

Description

The function retrieves the controller serial number.

Syntax

GetSerialNumber(handle, count, wait, failure_check=False)

Arguments

handle	Communication handle.
count	Max length of the return string.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

String type

Get extended error information by calling GetLastError.

Comments

The function retrieves the character string that contains the firmware version of the controller. The function will not copy more than *count* characters to the return string. If the buffer is too small, the firmware version can be truncated.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.34.3 GetBuffersCount

Description

The function returns the number of available ACSPL+ programming buffers.

Syntax

GetBuffersCount(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.34.4 GetAxesCount

Description

The function returns the number of available axes.

Syntax

GetAxesCount(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.34.5 GetDBufferIndex

Description

The function returns the index of the D-Buffer.

Syntax

GetDBufferIndex(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.34.6 GetUMDVersion

Description

The function retrieves the SPiiPlus User Mode Driver version number.

Syntax

GetUMDVersion()

Arguments

This function has no arguments.

Return Value

Int type

The return value is the represent 32-bit unsigned integer value, which specifies the binary version number.

Get extended error information by calling GetLastError.

Comments

The SPiiPlus User Mode Driver version consists of four (or fewer) numbers separated by points: #.#.#.#. The binary version number is represented by 32-bit unsigned integer value. Each byte of this value specifies one number in the following order: high byte of high word – first number, low byte of high word – second number, high byte of low word – third number and low byte of low word – forth number. For example version "2.10" has the following binary representation (hexadecimal format): 0x020A0000.
First two numbers in the string form are obligatory. Any release version of the library consists of two numbers. The third and fourth numbers specify an alpha or beta version, special or private build, or other version.

5.35 Error Diagnostic Functions

5.35.1 GetMotorError

Description

The function retrieves the reason for motor disabling.

Syntax

GetMotorError(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. GeneralDefinition.ACSC_NONEcorresponds to system.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The function retrieves the reason for motor disabling. If the motor is enabled the parameter, error is zero. If the motor was disabled, the parameter error contains the reason for motor disabling.
See SPiiPlus ACSPL+ Programmer's Guide for all available motor error code descriptions.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.35.2 GetMotionError

Description

The function retrieves the error code of the last program error encountered in the specified buffer.

Syntax

GetMotionError(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. GeneralDefinition.ACSC_NONE corresponds to system.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Int type

Get extended error information by calling GetLastError.

To get the error explanation, use the GetErrorString function.

Comments

The function retrieves the error code of the last program error encountered in the specified buffer. If the program is running, the parameter error is zero. If the program terminates for any reason, the parameter error contains the termination code.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.35.3 GetEtherCATError

Description

The function is used to retrieve the last EtherCAT error.

Syntax

GetEtherCATError(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Int type

Get extended error information by calling GetLastError.

Comments

Any EtherCAT error sets ECST.#OP to false and the error code is latched in the ECERR variable.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.35.4 GetProgramError

Description

The function retrieves the reason for motor disabling.

Syntax

GetProgramError(handle, ProgramBuffer, wait, failure_check=False)

Arguments

handle	Communication handle.
ProgramBuffer	Number of the program buffer.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Integer type - retrieves the error code of the last program error encountered in the specified buffer.

Comments

If the program is running, the method returns zero. If the program terminates for any reason, the method returns the termination code.

Get extended error information by calling GetLastError.

5.36 Shared Memory Functions

5.36.1 GetSharedMemoryAddress

Description

The function reads the address of shared memory variable.

Syntax

GetSharedMemoryAddress(handle, nBuf, var, wait, failure_check=False)

Arguments

handle	Communication handle.
nBuf	Number of program buffer for local variable or GeneralDefinition.ACSC_None for global and standard variable.
var	String that contains the name of the variable.
data	Value to be written.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Int type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.36.2 ReadSharedMemoryInteger

Description

The function reads value(s) from an integer shared memory variable.

Syntax

ReadSharedMemoryInteger(handle, address, from1, to1, from2, to2, failure_check=False)

Arguments

handle	Communication handle.
address	Shared memory address of the variable to be read.
from1, to1	Index range (first dimension)
From2, to2	Index range (second dimension).
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.36.3 WriteSharedMemoryInteger

Description

The function writes value(s) to the integer shared memory variable.

Syntax

WriteSharedMemoryInteger(handle, address, from1, to1, from2, to2, values, failure_check=False)

Arguments

handle	Communication handle.
address	Shared memory address of the variable that should be read.
from1, to1	Index range (first dimension)
From2, to2	Index range (second dimension).
values	The values to write.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.36.4 ReadSharedMemoryReal

Description

The function reads value(s) from a real shared memory variable.

Syntax

ReadSharedMemoryReal(handle, address, from1, to1, from2, to2, failure_check=False)

Arguments

handle	Communication handle.
address	Shared memory address of the variable that should be read.
from1, to1	Index range (first dimension)

From2, to2	Index range (second dimension).
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.36.5 WriteSharedMemoryReal

Description

The function writes value(s) to the real shared memory variable.

Syntax

WriteSharedMemoryReal(handle, address, from1, to1, from2, to2, values, failure_check=False)

Arguments

handle	Communication handle.
address	Shared memory address of the variable that should be read.
from1, to1	Index range (first dimension)
From2, to2	Index range (second dimension).
values	The values to write.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37 EtherCAT Functions

5.37.1 GetEtherCATState

Description

The function is used to retrieve the EtherCAT state.

Syntax

GetEtherCATState(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

Comments

The return int variable representing the 32-bit of the ACSPL+ variable **ESCT**.

For more information about digital outputs, see *SPiiPlus ACSPL+ Programmer's Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.2 MapEtherCATInput

Description



The function call is valid only when the EtherCAT state is operable (that is, **ECST**.OP=1)

The function is used for raw mapping of network input variables of any size. Once the function is called successfully, the firmware copies the value of the network input variable at the corresponding EtherCAT offset into the ACSPL+ variable, every controller cycle.

Syntax

MapEtherCATInput(handle, flags, offset, variableName, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter. Currently the value should be 0.
offset	Internal EtherCAT offset of network input variable extracted from the SPiiPlus MMI Application Studio Communication Terminal **#ETHERCAT** command.
variableName	Valid name of ACSPL+ variable, global or standard.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.3 MapEtherCATOutput

Description



The function call is valid only when the EtherCAT state is operable (that is, **ECST**.OP=1)

The function is used for raw mapping of network output variables of any size. Once the function is called successfully, the firmware copies the value of specified ACSPL+ variable into the network output variable at the corresponding EtherCAT offset, every controller cycle.

Syntax

MapEtherCATOutput(handle, flags, offset, variableName, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter. Currently the value should be 0.
offset	Internal EtherCAT offset of network input variable extracted from the SPiiPlus MMI Application Studio Communication Terminal **#ETHERCAT** command.
variableName	Valid name of ACSPL+ variable, global or standard.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.4 UnmapEtherCATInputsOutputs

Description



The function call is valid only when the EtherCAT state is operable (that is, **ECST**.OP=1)

The function resets all previous mapping defined by MapEtherCATInput and MapEtherCATOutput functions.

Syntax

UnmapEtherCATInputsOutputs(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.5 GetEtherCATSlaveIndex

Description

The function returns the index of EtherCAT slave according to the parameters that are specified.

Syntax

GetEtherCATSlaveIndex(handle, vendorID, productID, count, wait, failure_check=False)

Arguments

handle	Communication handle.
vendorID	EtherCAT Vendor ID of the slave device.
productID	ID EtherCAT Product ID of the slave device.
count	Internal count of the device within those devices having the same Product and Vendor IDs.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.6 GetEtherCATSlaveOffsetV2

Description

The function returns offset of a network variable of the specified EtherCAT slave in specific ECAT network.

Syntax

GetEtherCATSlaveOffsetV2(handle, flags, variableName, slaveIndex, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_BIT_OFFSET - bit offset of the variable. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together. 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0).
variableName	Name of the EtherCAT network variable.
slaveIndex	Index of the required EtherCAT slave, can be determined by GetEtherCATSlaveIndex function.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.7 GetEtherCATSlaveVendorIDV2

Description

The function returns the Vendor ID of the specified EtherCAT slave in the specific ECAT network.

Syntax

GetEtherCATSlaveVendorIDV2(handle, flags, slaveIndex, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together. 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0).
slaveIndex	Index of the required EtherCAT slave, can be determined by GetEtherCATSlaveIndex function.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.8 GetEtherCATSlaveProductIDV2

Description

The function returns the Product ID of the specified EtherCAT slave in the specific EtherCAT network.

Syntax

GetEtherCATSlaveVendorIDV2(handle, flags, slaveIndex, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together. 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0).
slaveIndex	Index of the required EtherCAT slave, can be determined by GetEtherCATSlaveIndex function.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float Type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.9 GetEtherCATSlaveRevisionV2

Description

The function returns the revision of the specified EtherCAT slave in the specific EtherCAT network.

Syntax

GetEtherCATSlaveRevisionV2(handle, flags, slaveIndex, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together. 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0).
slaveIndex	Index of the required EtherCAT slave, can be determined by GetEtherCATSlaveIndex function.

wait	If Wait is SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCKobject, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detected error, it will raise exception. If False the function will ignore detected errors.

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.10 GetEtherCATSlaveType

Description

The function returns the type of the specified EtherCAT slave.

Syntax

GetEtherCATSlaveType(handle, vendorID, productID, wait, failure_check=False)

Arguments

handle	Communication handle.
vendorID	EtherCAT Vendor ID of the slave device.
productID	Product ID of the specified EtherCAT slave.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

The type of specified EtherCAT slave.
The value can be one of the following:

- 0 – ACS device.
- 1 – Non-ACS Servo.
- 2 – Non-ACS Stepper.
- 3 – Non-ACS I/O.
- -1 – Device not found at slave index.

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.11 GetEtherCATSlaveRegister

Description

The function returns value of the ESC Error Counters Registers of specified EtherCAT slave in specific EtherCAT network.

Syntax

GetEtherCATSlaveRegister(handle, flags, slaveIndex, offset, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together. 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0).
slaveIndex	Index of the required EtherCAT slave, can be determined by GetEtherCATSlaveIndex function.
offset	Register offset in the ESC memory.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

To see the table lists supported error counter registers see the EtherCAT Error Counter Registers table in *ACSPL+ Commands Variables Reference Guide*.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.12 GetEtherCATSlaveStateV2

Description

The function returns machine state of the specified EtherCAT slave in specific EtherCAT network.

Syntax

GetEtherCATSlaveStateV2(handle, flags, slaveIndex, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together. 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0).
slaveIndex	Index of the required EtherCAT slave, can be determined by GetEtherCATSlaveIndex function.
offset	Register offset in the ESC memory.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Returns the state of specified EtherCAT slave. The value can be one of the following:

- 1 – INIT
- 2 – PREOP
- 4 – SAFEOP
- 8 – OP
- -1 – Device not found at slave index

If specified slave does not exist or is not accessible, -1 is returned.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.37.13 GetEtherCATSlavesCount

Description

The function is used to retrieve the number of slaves in the specified EtherCAT network.

Syntax

GetEtherCATSlavesCount(handle, flags, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier. EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together. 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0).

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Returns the number of slaves in the specified EtherCAT network If there are no slaves in the system, -1 is returned.

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38 Position Event Generation (PEG) Functions

5.38.1 AssignPegNTV2

Description



This function can be used only with the SPiiPlus family of controllers.

This function is used for engine-to-encoder assignment as well as for the additional digital outputs assignment for use as PEG state and PEG pulse outputs.

Syntax

AssignPegNTV2(handle, flags, axis, engToEncBitCode, gpOutsBitCode, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include the following flag: MotionFlags.ACSC_AMF_FASTLOADINGPEG If included, fast loading of Random PEG arrays is activated.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_ AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONEcorresponds to system.

engToEncBitCode	Bit code for engines-to-encoders mapping according to the ASSIGNPEG chapter in the *PEG and MARK Operations Application Notes*.
gpOutsBitCode	General Purpose outputs assignment to use as PEG state and PEG pulse outputs according to the ASSIGNPEG chapter in the*PEG and MARK Operations Application Notes*.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38.2 AssignPegOutputsNT

Description



This function can be used only with the SPiiPlus family of controllers.

The function is used for setting output pins assignment and mapping between FGP_OUT signals to the bits of the ACSPL+ OUT(x) variable, where x is the index that has been assigned to the controller in the network during System Configuration.

OUT is an integer array that can be used for reading or writing the current state of the General Purpose outputs - see the *SPiiPlus Command & Variable Reference Guide*.

Syntax

AssignPegOutputsNT(handle, axis, outputIndex, bitCode, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
outputIndex	0 for OUT_0, 1 for OUT_1, ..., 9 for OUT_9.
bitCode	Bit code for engine outputs to physical outputs mapping according to the ASSIGNPEG chapter in the*PEG and MARK Operations Application Notes*.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38.3 AssignFastInputsNT

Description



While this method is not related to PEG activity, it is included with the PEG methods for the sake of completeness, since many times fast inputs are used in applications that use PEG functionality.

The method is used to switch MARK_1 physical inputs to ACSPL+ variables as fast General Purpose inputs for SPiiPlus family controllers.

The method is used for setting input pins assignment and mapping between FGP_IN signals to the bits of the ACSPL+ IN(x) variable, where x is the index that has been assigned to the controller in the

network during System Configuration.
IN is an integer array that can be used for reading the current state of the General Purpose inputs.

See the *SPiiPlus Command & Variable Reference Guide*.

Syntax

AssignFastInputsNT(handle, axis, inputIndex, bitCode, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONEcorresponds to system.
inputIndex	0 for OUT_0, 1 for OUT_1, ..., 9 for OUT_9.
bitCode	Bit code for mapping engines inputs to physical inputs the **ASSIGNPEG** and **ASSIGNPOUTS** chapters in *the PEG and MARK Operations Application Notes*.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38.4 PegIncNTV2

Description

The function is used for setting the parameters for the Incremental PEG mode. The Incremental PEG function is defined by first point, last point and the interval.

Syntax

PegIncNTV2(handle, flags, axis, width, firstPoint, interval, lastPoint, errMapAxis1, axisCoord1, errMapAxis2, axisCoord2, errMapMaxSize, minDirDistance, tbNumber, tbPeriod, wait, failure_check=False)

Arguments

handle	Communication handle.

flags	Bit-mapped parameter that can include following flags: MotionFlags.ACSC_AMF_WAIT - the execution of the PEG is delayed until the STARTPEGNT function is executed. MotionFlags.ACSC_AMF_INVERT_OUTPUT - the PEG pulse output is inverted. MotionFlags.ACSC_AMF_ACCURATE - error accumulation is prevented by taking into account the rounding of the distance between incremental PEG events. You must use this switch if interval does not match a whole number of encoder counts. Using this switch is recommended for any application that uses the PegIncNTV2 command, whether or not interval defines a whole number of encoder counts. MotionFlags.ACSC_AMF_SYNCHRONOUS - PEG starts synchronously with the motion sequence. MotionFlags.ACSC_AMF_ENDLESS This flag supports endless incremental PEG. If the flag is set in in the PegIncNTV2 function, the last_point parameter is optional and ignored now , and the PEG never stops by position. To stop PEG use the StopPegNT command. MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION1D This flag supports 1D/2D dynamic error compensation for Incremental PEG. 1D/2D error compensation can be defined by the 1D error mapping functions documented in Dynamic Error Compensation section. MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D This flag supports 2D dynamic error compensation for Incremental PEG. See Dynamic Error Compensation for dynamic error compensation function details. This flag requires 2 additional function arguments: ErrMapAxis1 and AxisCoord1. MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D This flag supports 3D dynamic error compensation for Incremental PEG. See Dynamic Error Compensation for dynamic error compensation function details. This flag must be used in combination with MotionFlags.ACSC_ AMF_DYNAMIC_ERROR_COMPENSATION2D and requires 4 additional function arguments: ErrMapAxis1, AxisCoord1, ErrMapAxis2, and AxisCoord2. MotionFlags.ACSC_AMF_MAXIMUM_ARR_SIZE This flag supports 1D/2D dynamic error compensation for Incremental PEG. The ErrMapMaxSize parameter must have a relevant value with this flag is set. MotionFlags.ACSC_AMF_MIN_AXIS_DIRECTION This flag signals that the MinDirDistance parameter indicates a value defining actual motion as opposed to jitter. If motion along the axis when error compensation is in force is greater than this value, that motion is used to determine the direction of motion.

axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONEcorresponds to system.
width	Width of desired pulse in milliseconds.
firstPoint	Position where the first pulse is generated.
interval	Distance between the pulse-generating points.
lastPoint	Position where the last pulse is generated. If the MotionFlags.ACSC_AMF_ENDLESS flag is declared, then this parameter should be set to GeneralDefinition.ACSC_NONE.
errMapAxis1	The index of the first static axis when error mapping is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D flag alone or in combination with MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D.
axisCoord1	The predefined location value of the first static axis when error map correction compensation is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D flag alone or in combination with MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D.
errMapAxis2	The index of the second static axis when error mapping is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D and MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D flags.
axisCoord2	The predefined location value of the second static axis when error map correction compensation is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D and MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D flags.
errMapMaxSize	Maximum array size contained error correction data. Default is 512. Range from 1 to **XARRSIZE**. A negative value or 0 is interpreted as default.
minDirDistance	Limit defining actual motion as opposed to jitter. If motion along the axis is greater than this value, that motion is used to determine the direction of motion.
tbNumber	Number of time-based pulses generated after each encoder-based pulse.
tbPeriod	Period of time-based pulses.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38.5 PegRandomNTV2

Description

The new function includes new parameters supporting error mapping.
The function sets the parameters for the Random PEG mode. The Random PEG function specifies an array of points where position-based events are to be generated, and includes parameters for error mapping support.

Syntax

PegRandomNTV2(handle, flags, axis, width, mode, firstIndex, lastIndex, pointArray, stateArray, errMapAxis1, axisCoord1, errMapAxis2, axisCoord2, minDirDistance, tbNumber, tbPeriod, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped parameter that can include following flags: Motion Flags the execution of the PEG is delayed until the StartPegNT function is executed. MotionFlags.ACSC_AMF_INVERT_OUTPUT the PEG pulse output is inverted. MotionFlags.ACSC_AMF_SYNCHRONOUS PEG starts synchronously with the motion sequence. MotionFlags.ACSC_AMF_DYNAMICLOADINGGPEG If the MotionFlags.ACSC_AMF_DYNAMICLOADINGGPEG flag is included, dynamic loading of positions is implemented. MotionFlags.ACSC_AMF_MODULE : A new MotionFlags.ACSC_AMF_ MODULE flag is introduced to support modulo axis. The Positions Arrays loaded once, provides pulses every Modulo cycle. The Position values should be inside the Modulo range. The PEG engine must be assigned to Modulo axis. MotionFlags.ACSC_AMF_DYNAMIC_ERROR_ COMPENSATION1D This flag supports 1D/2D dynamic error compensation for Incremental PEG. 1D/2D error compensation can be defined by the 1D error mapping functions documented in Dynamic Error Compensation section. MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D This flag supports 2D dynamic error compensation for Incremental PEG. See Dynamic Error Compensation for dynamic error compensation function details. This flag requires 2 additional function arguments: ErrMapAxis1 and AxisCoord1. MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D This flag supports 3D dynamic error compensation for Incremental PEG. See Dynamic Error Compensation for dynamic error compensation function details. This flag must be used in combination with MotionFlags.ACSC_ AMF_DYNAMIC_ERROR_COMPENSATION2D and requires 4 additional function arguments: ErrMapAxis1, AxisCoord1, ErrMapAxis2, and AxisCoord2. MotionFlags.ACSC_AMF_MAXIMUM_ARR_SIZE This flag supports 1D/2D dynamic error compensation for Incremental PEG. The ErrMapMaxSize parameter must have a relevant value with this flag is set. MotionFlags.ACSC_AMF_MIN_AXIS_DIRECTION This flag signals that the MinDirDistance parameter indicates a value defining actual motion as opposed to jitter. If motion along the axis when error compensation is in force is greater than this value, that motion is used to determine the direction of motion.

axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
width	Width of desired pulse in milliseconds.
mode	Output signal configuration according to the **ASSIGNPEG** chapter in *PEG and MARK Operations Application Notes*.
firstIndex	Index of position in pointArray where the first pulse is generated.
lastIndex	Index of position in pointArray where the last pulse is generated.
pointArray	String containing the name of the real array that stores positions at which PEG pulse are to be generated The array must be declared as a global variable by an ACSPL+ program or by the DeclareVariable function.
stateArray	String containing the name of the integer array that stores desired output state at each position. The array must be declared as a global variable by an ACSPL+ program or by the DeclareVariable function. If output state change is not desired, this parameter should be NULL.
errMapAxis1	The index of the first static axis when error mapping is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D flag alone or in combination with MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D.
axisCoord1	The predefined location value of the first static axis when error map correction compensation is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D flag alone or in combination with MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D.
errMapAxis2	The index of the second static axis when error mapping is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D and MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D flags.
axisCoord2	The predefined location value of the second static axis when error map correction compensation is used with PEG. Use with the MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION2D and MotionFlags.ACSC_AMF_DYNAMIC_ERROR_COMPENSATION3D flags.

minDirDistance	Limit defining actual motion as opposed to jitter. If motion along the axis is greater than this value, that motion is used to determine the direction of motion.
tbNumber	Number of time-based pulses generated after each encoder-based pulse.
tbPeriod	Period of time-based pulses.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38.6 WaitPegReadyNT

Description

The function waits for the all values to be loaded and the PEG engine to be ready to respond to movement on the specified axis for SPiiPlus family controllers.
The method can be used in both the Incremental and Random PEG modes.

Syntax

WaitPegReadyNT(handle, axis, timeout, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONEcorresponds to system.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38.7 StartPegNT

Description



This function can be used only with the SPiiPlus family of controllers.

The function is used to initiate the PEG process.

Syntax

StartPegNT(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1,etc. GeneralDefinition.ACSC_NONE corresponds to system.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.38.8 StopPegNT

Description



This function can be used only with the SPiiPlus family of controllers.

The function is used to terminate the PEG process immediately on the specified axis. The function can be used in both the Incremental and Random PEG modes.

Syntax

StopPegNT(handle, axis, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39 Dynamic Error Compensation

5.39.1 DynamicErrorCompensationOn

Description

DynamicErrorCompensationOn function receives axis index and zone index parameters. The function activates error correction for the mechanical error compensation for the specified zone.

Syntax

DynamicErrorCompensationOn(handle, axis, zone, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.2 DynamicErrorCompensationOff

Description

DynamicErrorCompensationOff function receives axis index and zone index parameters . The function deactivates error mapping correction for the mechanical error compensation for the specified zone.

Syntax

DynamicErrorCompensationOff(handle, axis, zone, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.3 DynamicErrorCompensation1D

Description

The DynamicErrorCompensation1D function configures and activates 1D error correction for the mechanical error compensation for the specified zone, so that the compensated reference position will be calculated by subtracting the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value. The calculation assumes fixed intervals between points inside the zone.

Syntax

DynamicErrorCompensation1D(handle, flags, axis, zone, base0, step0, correctionMapVariable, referencedAxisOrAnalogInput, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_PREVENT_COMP_INDEX_MARK_PEG Prevent applying dynamic error compensation on INDEX, MARK, and PEG values. DynamicErrorCompensationFlags.ACSC_DECOMP_REFERENCED_AXIS Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ANALOG_INPUT Specifies that the mechanical error compensation will be calculated based on the feedback from the analog input indicated by the optional parameter. 0, //No flags are set
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
base0	A real number representing the axis command that corresponds to the first point in correction table for mechanical error compensation.
step0	A real number representing the fixed interval distance between the two adjacent axis commands.
correctionMapVariable	The name of a real one-dimensional array that specifies correction table for mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput	The index of the axis, or the index of the analog input that the mechanical error compensation will be calculated based on its feedback.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.4 DynamicErrorCompensationN1D

Description

The DynamicErrorCompensationN1D function configures and activates 1D error correction for the mechanical error compensation for the specified zone, so that the compensated reference position will be calculated by subtracting the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value. The calculation is based on an arbitrary network of points inside the zone.

Syntax

DynamicErrorCompensationN1D(handle, flags, axis, zone, axisCommands, correctionMapVariable, referencedAxisOrAnalogInput, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_ PREVENT_COMP_INDEX_MARK_PEG Prevent applying dynamic error compensation on INDEX, MARK, and PEG values. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ ANALOG_INPUT Specifies that the mechanical error compensation will be calculated based on the feedback from the analog input indicated by the optional parameter. 0, //No flags are set
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
axisCommands	The name of a real one-dimensional array that specifies axis command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMapVariable	The name of a real one-dimensional array that specifies correction table for mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput	The index of the axis, or the index of the analog input that the mechanical error compensation will be calculated based on its feedback.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.5 DynamicErrorCompensationA1D

Description

The DynamicErrorCompensationA1D function configures and activates 1D error correction for the mechanical error compensation for the specified zone, so that the compensated reference position will be calculated by multiplying the scaling factor by the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensationA1D(handle, flags, axis, zone, scalingFactor, offset, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_PREVENT_COMP_INDEX_MARK_PEG Prevent applying dynamic error compensation on INDEX, MARK, and PEG values.
axis	Axis constant: Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. GeneralDefinition.ACSC_NONE corresponds to system.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
scalingFactor	The scaling factor for the linear alignment that will be used for mechanical error compensation. The allowed range for the scaling factor is 0>2.
offset	The offset for the linear alignment that will be used for mechanical error compensation. The offset is actually the mechanical error compensation for the 0-point location.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.6 DynamicErrorCompensation2D

Description

The DynamicErrorCompensation2D function configures and activates 2D error correction for the mechanical error compensation of the 'axis0' command for the specified zone, so that the compensated reference position will be calculated by subtracting the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensation2D(handle, flags, axis0, axis1, zone, base0, step0, base1, step1, correctionMapVariable, referencedAxisOrAnalogInput0, referencedAxisOrAnalogInput1, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00 The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01 The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ ANALOG_INPUT Specifies that the mechanical error compensation will be calculated based on the feedback from the analog input indicated by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT Specifies that the second optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
Axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
base0	A real number representing the 'axis0' command that corresponds to the first point in correction table for mechanical error compensation.

step0	A real number representing the fixed interval distance between the two adjacent 'axis0' commands.
base1	A real number representing the 'axis1' command that corresponds to the first point in correction table for mechanical error compensation.
step1	A real number representing the fixed interval distance between the two adjacent 'axis1' commands.
correctionMapVariable	The name of a real one-dimensional array that specifies correction table for mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.7 DynamicErrorCompensationN2D

Description

The **DynamicErrorCompensationN2D** function configures and activates 2D error correction for the mechanical error compensation of the 'axis0' parameter for the specified zone, so that the compensated reference position will be calculated by subtracting the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value. The calculation is based on an arbitrary network of points inside the zone.

Syntax

DynamicErrorCompensationN2D(handle, flags, axis0, axis1, zone, axis0Commands, axis1Commands, correctionMapVariable, referencedAxis0rAnalogInput0, referencedAxis0rAnalogInput1, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00 The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01 The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ ANALOG_INPUT Specifies that the mechanical error compensation will be calculated based on the feedback from the analog input indicated by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT Specifies that the second optional parameter will be regarded as an analog input index. 0, //No flags are set

axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
Axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
axis0Commands	The name of a real one-dimensional array that specifies 'axis0' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis1Commands	The name of a real one-dimensional array that specifies 'axis1' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMapVariable	The name of a real one-dimensional array that specifies correction table for mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.8 DynamicErrorCompensationA2D

Description

The **DynamicErrorCompensationA2D** function configures and activates 2D error correction for the mechanical error compensation of the specified axis for the specified zone, so that the compensated reference position will be calculated by taking into account the angle for the orthogonality correction so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensationA2D(handle, flags, axis0, axis1, zone, angle, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00 The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01 The mechanical error compensation will be applied to 'axis1'. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
Axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
angle	The angle for the orthogonality correction that will be used for mechanical error compensation. The allowed range for the angle is [-45°, 45°].

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.9 DynamicErrorCompensation3D2

Description

The **DynamicErrorCompensation3D2** function configures and activates 3D error correction for the mechanical error compensation of the 'axis0', 'axis1', and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensation3D2(handle, flags, axis0, axis1, axis2, zone, base0, step0, base1, step1, base2, step2, correctionMap0Variable, correctionMap1Variable, referencedAxisOrAnalogInput0, referencedAxisOrAnalogInput1, referencedAxisOrAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00 The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01 The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02 The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
base0	A real number representing the 'axis0' command that corresponds to the first point in correction table for mechanical error compensation.
step0	A real number representing the fixed interval distance between the two adjacent 'axis0' commands.
base1	A real number representing the 'axis1' command that corresponds to the first point in correction table for mechanical error compensation.
step1	A real number representing the fixed interval distance between the two adjacent 'axis1' commands.
correctionMap0Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '0' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '1' coordinate. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.10 DynamicErrorCompensation3D3

Description

The **DynamicErrorCompensation3D3** function configures and activates 3D error correction for the mechanical error compensation of the 'axis0', 'axis1', and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensation3D3(handle, flags, axis0, axis1, axis2, zone, base0, step0, base1, step1, base2, step2, correctionMap0Variable, correctionMap1Variable, correctionMap2Variable, referencedAxisOrAnalogInput0, referencedAxisOrAnalogInput1, referencedAxisOrAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00: The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT: Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT: Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT: Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
base0	A real number representing the 'axis0' command that corresponds to the first point in correction table for mechanical error compensation.
step0	A real number representing the fixed interval distance between the two adjacent 'axis0' commands.
base1	A real number representing the 'axis1' command that corresponds to the first point in correction table for mechanical error compensation.
step1	A real number representing the fixed interval distance between the two adjacent 'axis1' commands.
base2	A real number representing the 'axis2' command that corresponds to the first point in correction table for mechanical error compensation.
step2	A real number representing the fixed interval distance between the two adjacent 'axis2' commands.
correctionMap0Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '0' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '1' coordinate. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap2Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '2' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.

referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.11 DynamicErrorCompensation3D5

Description

The **DynamicErrorCompensation3D5** function configures and activates 3D error correction for the mechanical error compensation of the 'axis0', 'axis1', and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensation3D5(handle, flags, axis0, axis1, axis2, zone, base0, step0, base1, step1, base2, step2, correctionMap0Variable, correctionMap1Variable, correctionMap2Variable, correctionMap3Variable, correctionMap4Variable, referencedAxisOrAnalogInput0, referencedAxisOrAnalogInput1, referencedAxisOrAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00: The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS: Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT: Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT: Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
base0	A real number representing the 'axis0' command that corresponds to the first point in correction table for mechanical error compensation.
step0	A real number representing the fixed interval distance between the two adjacent 'axis0' commands.
base1	A real number representing the 'axis1' command that corresponds to the first point in correction table for mechanical error compensation.
step1	A real number representing the fixed interval distance between the two adjacent 'axis1' commands.
base2	A real number representing the 'axis2' command that corresponds to the first point in correction table for mechanical error compensation.
step2	A real number representing the fixed interval distance between the two adjacent 'axis2' commands.
correctionMap0Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '0' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '1' coordinate. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap2Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '2' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).

correctionMap3Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '3' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap4Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '4' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.12 DynamicErrorCompensation3DA

Description

The DynamicErrorCompensation3DA function configures and activates 3D error correction for the mechanical error compensation of the 'axis0', 'axis1', and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensation3DA(handle, flags, axis0, axis1, axis2, zone, base0, step0, base1, step1, base2, step2, correctionMap0Variable, correctionMap1Variable, correctionMap2Variable, correctionMap3Variable, correctionMap4Variable, correctionMap5Variable, correctionMap6Variable, correctionMap7Variable, correctionMap8Variable, correctionMap9Variable, referencedAxisOrAnalogInput0, referencedAxisOrAnalogInput1, referencedAxisOrAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00: The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS: Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT: Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT: Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT: Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
base0	A real number representing the 'axis0' command that corresponds to the first point in correction table for mechanical error compensation.
step0	A real number representing the fixed interval distance between the two adjacent 'axis0' commands.
base1	A real number representing the 'axis1' command that corresponds to the first point in correction table for mechanical error compensation.
step1	A real number representing the fixed interval distance between the two adjacent 'axis1' commands.
base2	A real number representing the 'axis2' command that corresponds to the first point in correction table for mechanical error compensation.
step2	A real number representing the fixed interval distance between the two adjacent 'axis2' commands.
correctionMap0Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '0' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '1' coordinate. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap2Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '2' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).

correctionMap3Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '3' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap4Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '4' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap5Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '5' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap6Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '6' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap7Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '7' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap8Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '8' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
correctionMap9Variable	The name of a real two-dimensional array that specifies correction table for mechanical error compensation in relation to axis 2 step '9' coordinate. The array type should be GLOBAL REAL STATIC (defined in DBuffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.

referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.39.13 DynamicErrorCompensationN3D2

Description

The **DynamicErrorCompensationN3D2** function configures and activates 3D error correction for the mechanical error compensation of the 'axis0', 'axis1', and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensationN3D2(handle, flags, axis0, axis1, axis2, zone, axis0Commands, axis1Commands, axis2Commands, correctionMap0Variable, correctionMap1Variable, referencedAxisOrAnalogInput0, referencedAxisOrAnalogInput1, referencedAxisOrAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00: The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS: Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT: Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT: Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT: Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
axis0Commands	The name of a real one-dimensional array that specifies 'axis0' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis1Commands	The name of a real one-dimensional array that specifies 'axis1' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis2Commands	The name of a real one-dimensional array that specifies 'axis2' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap0Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the first specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the second specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

5.39.14 DynamicErrorCompensationN3D3

Description

The **DynamicErrorCompensationN3D3** function configures and activates 3D error correction for the mechanical error compensation of the 'axis0', 'axis1', and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensationN3D3(handle, flags, axis0, axis1, axis2, zone, axis0Commands, axis1Commands, axis2Commands, correctionMap0Variable, correctionMap1Variable, correctionMap2Variable, referencedAxis0rAnalogInput0, referencedAxis0rAnalogInput1, referencedAxis0rAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00: The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS: Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT: Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT: Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT: Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
axis0Commands	The name of a real one-dimensional array that specifies 'axis0' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis1Commands	The name of a real one-dimensional array that specifies 'axis1' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis2Commands	The name of a real one-dimensional array that specifies 'axis2' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap0Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the first specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the second specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap2Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the third specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.

referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

5.39.15 DynamicErrorCompensationN3D5

Description

The **DynamicErrorCompensationN3D5** function configures and activates 3D error correction for the mechanical error compensation of the 'axis0', 'axis1', and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensationN3D5(handle, flags, axis0, axis1, axis2, zone, axis0Commands, axis1Commands, axis2Commands, correctionMap0Variable, correctionMap1Variable, correctionMap2Variable, correctionMap3Variable, correctionMap4Variable, referencedAxisOrAnalogInput0, referencedAxisOrAnalogInput1, referencedAxisOrAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00: The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS: Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT: Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT: Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT: Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
axis0Commands	The name of a real one-dimensional array that specifies 'axis0' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis1Commands	The name of a real one-dimensional array that specifies 'axis1' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis2Commands	The name of a real one-dimensional array that specifies 'axis2' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap0Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the first specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the second specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap2Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the third specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap3Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the fourth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).

correctionMap4Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the fifth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Return Value

None

Get extended error information by calling GetLastError.

5.39.16 DynamicErrorCompensationN3DA

Description

The **DynamicErrorCompensationN3DA** function configures and activates 3D error correction for the mechanical error compensation of the 'axis0' , 'axis1' , and 'axis2' parameters for the specified zone, so that the compensated reference position will be calculated by adding the linearly (by default) interpolated error from the desired position so that the actual value will be closer to the desired value.

Syntax

DynamicErrorCompensationN3DA(handle, flags, axis0, axis1, axis2, zone, axis0Commands, axis1Commands, axis2Commands, correctionMap0Variable, correctionMap1Variable, correctionMap2Variable, correctionMap3Variable, correctionMap4Variable, correctionMap5Variable, correctionMap6Variable, correctionMap7Variable, correctionMap8Variable, correctionMap9Variable, referencedAxis0rAnalogInput0, referencedAxis0rAnalogInput1, referencedAxis0rAnalogInput2, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DynamicErrorCompensationFlags.ACSC_DECOMP_00: The mechanical error compensation will be applied to 'axis0' (default). DynamicErrorCompensationFlags.ACSC_DECOMP_01: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_02: The mechanical error compensation will be applied to 'axis1'. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS: Specifies that the mechanical error compensation will be calculated based on the feedback from the axis specified by the optional parameter. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_FIRST_ANALOG_ INPUT: Specifies that the first optional parameter will be regarded as an analog input index. DynamicErrorCompensationFlags.ACSC_DECOMP_ REFERENCED_AXIS | ACSC_DECOMP_SECOND_ ANALOG_ INPUT: Specifies that the second optional parameter will be regarded as an analog input index. ACSC_DECOMP_REFERENCED_AXIS | ACSC_DECOMP_ THIRD_ANALOG_ INPUT: Specifies that the third optional parameter will be regarded as an analog input index. 0, //No flags are set
axis0	The index of the first axis that the mechanical error compensation will be applied to. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

axis1	The index of the second axis participating in 2D mechanical error compensation. Valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1
axis2	The index of the second axis participating in 3D mechanical error compensation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.
axis0Commands	The name of a real one-dimensional array that specifies 'axis0' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis1Commands	The name of a real one-dimensional array that specifies 'axis1' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
axis2Commands	The name of a real one-dimensional array that specifies 'axis2' command values used for correction table of mechanical error compensation. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap0Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the first specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap1Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the second specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap2Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the third specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).

correctionMap3Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the fourth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap4Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the fifth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap5Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the sixth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap6Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the seventh specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap7Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the eighth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap8Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the ninth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
correctionMap9Variable	The name of a real two-dimensional array that specifies a correction table for mechanical error compensation in relation to the tenth specified coordinate of the Z-axis. The array type should be GLOBAL REAL STATIC (defined in D-Buffer).
referencedAxisOrAnalogInput0	The index of the first axis, or the index of the first analog input whose feedback will be used to calculate the mechanical error compensation.

referencedAxisOrAnalogInput1	The index of the second axis, or the index of the second analog input whose feedback will be used to calculate the mechanical error compensation.
referencedAxisOrAnalogInput2	The index of the third axis, or the index of the third analog input whose feedback will be used to calculate the mechanical error compensation.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

5.39.17 DynamicErrorCompensationRemove

Description

The **DynamicErrorCompensationRemove** function receives axis index and zone index parameters. The function deactivates error correction for the mechanical error compensation for the specified zone.

Syntax

DynamicErrorCompensationRemove(handle, axis, zone, wait, failure_check=False)

Arguments

handle	Communication handle.
axis	Axis constant: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 – to axis 1, etc. For the axis constants see Axis Definitions.
zone	The zone index, valid numbers are: 0, 1, 2, ... up to the maximum number of zones (up to 10) minus 1. If '-1' is specified, all zones of specified axis will be affected.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Comments

None

5.40 Emergency Stop Functions

5.40.1 RegisterEmergencyStop

Description

The function initiates the Emergency Stop functionality for calling application.

Syntax

RegisterEmergencyStop(failure_check=False)

Arguments

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

SPiiPlus UMD (User Mode Driver) and the Python package provide the user application with the ability to open/close the Emergency Stop button. Clicking the Emergency Stop button sends a stop command to all motions and motors to all channels, thereby stopping all motions and disabling all motors.



Figure 5-1. Emergency Stop Button

In the SPiiPlus UMD, when it has been selected, the Emergency Stop button stops all motions and disables all motors. Previously only the SPiiPlus MMI provided such functionality. Now such functionality is also available for user applications.

Calling **RegisterEmergencyStop** will cause an Emergency Stop button to appear in the right bottom corner of the computer screen. If this button has already been activated by another application, a new button does not appear, but all functionality is available for the new application. Clicking the Emergency Stop button causes the stopping of all motions and motors command to all channels that are used in the calling application.

Calling **RegisterEmergencyStop** works only if the local host SPiiPlus UMD is running, even if it is used through a remote connection, because the Emergency Stop button is part of the local SPiiPlus UMD. If the local SPiiPlus UMD is not running, the function fails.

Only a single call is required per application. It can be placed anywhere in code, even before the opening of communication with controllers.

An application can remove the Emergency Stop button by calling UnregisterEmergencyStop. The Emergency Stop button disappears if there are no additional registered applications using it. Termination of SPiiPlus UMD also removes the Emergency Stop button, so restarting the SPiiPlus UMD requires calling **RegisterEmergencyStop** again.

Registering the Emergency Stop button more than once per application is meaningless, but the function succeeds anyway. In order to ensure that the Emergency Stop button is active, call **RegisterEmergencyStop** after each call to any of the **OpenComm***()** functions.

5.40.2 UnregisterEmergencyStop

Description

The function terminates the Emergency Stop functionality for the calling application.

Syntax

UnregisterEmergencyStop(failure_check=False)

Arguments

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Comments

The SPiiPlus User Mode Driver (UMD) and Python package provide user applications with the ability to activate or deactivate the Emergency Stop button displayed in the UMD. Calling **UnregisterEmergencyStop** will cause applications not to respond to the user clicking the Emergency Stop button displayed on the screen. If there are no other applications that have registered the Emergency Stop functionality (through the RegisterEmergencyStop), the button will disappear.

Unregistering Emergency Stop more than once per application is meaningless, but the function will succeed anyway.

5.41 Application Save/Load Functions

5.41.1 AnalyzeApplication

Description

The function analyzes an application file and returns information about the file's components, such as, saved ACSPL+ programs, configuration parameters, user files, etc.

Syntax

AnalyzeApplication(handle, fileName, wait, failure_check=False)

Arguments

handle	Communication handle.
fileName	Filename (with included path) of the Application file.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If `fileName` is None, the current Controller Application will be analyzed; otherwise, the file specified by `fileName`, from the local hard disk, will be analyzed.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.41.2 LoadApplication

Description

The function loads a section of data from the host PC disk and saves it in the controller's files.

Syntax

LoadApplication(handle, fileName, wait, failure_check=False)

Arguments

handle	Communication handle.
fileName	Filename (with included path) of the Application file.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

object of type ACSC_APPSL_INFO

Comments

If `fileName` is None, the current Controller Application will be analyzed; otherwise, the file specified by `fileName`, from the local hard disk, will be analyzed.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

The user should reboot the controller after the function operation to update the controller configuration.

To reboot the controller the user can use the ControllerReboot function.

5.41.3 SaveApplication

Description

The function saves a user application from the controller to a file on the host PC.

Syntax

SaveApplication(handle, fileName, info, wait, failure_check=False)

Arguments

handle	Communication handle.
fileName	Filename (with included path) of the Application file.
info	An application information descriptor, defined by the ACSC_ APPSL_INFO object. The AnalyzeApplication function is solely responsible for initializing this structure.

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.41.4 FreeApplication

Description

The function frees memory previously allocated by the AnalyzeApplication function.

Syntax

FreeApplication(info, failure_check=False)

Arguments

info	An application information descriptor, defined by the ACSC_ APPSL_INFO object. The AnalyzeApplication function is solely responsible for initializing this structure.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.42 Reboot Functions

5.42.1 ControllerReboot

Description

The function reboots controller and waits for process completion.

Syntax

ControllerReboot(handle, timeout, failure_check=False)

Arguments

handle	Communication handle.
timeout	Maximum waiting time in milliseconds.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.42.2 ControllerFactoryDefault

Description

The function reboots controller, restores factory default settings and waits for process completion.

Syntax

ControllerFactoryDefault(handle, timeout, failure_check=False)

Arguments

handle	Communication handle.
timeout	Maximum waiting time in milliseconds.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.43 Host-Controller File Operations

5.43.1 CopyFileToController

Description

The function copies file from host PC to controller's non-volatile memory.

Syntax

CopyFileToController(handle, sourceFileName, destinationFileName, wait, failure_check=False)

Arguments

handle	Communication handle.
sourceFileName	String that contains name of the source file on host PC.
destinationFileName	String that contains name of the destination file in controller's flash.
timeout	Maximum waiting time in milliseconds.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.



The flash memory is rated to support about 100,000 write operations.
ACS recommends against frequent, repeated writing to the flash memory.

5.43.2 DeleteFileFromController

Description

The function deletes user files from controller's non-volatile memory.

Syntax

DeleteFileFromController(handle, fileName, wait, failure_check=False)

Arguments

handle	Communication handle.
fileName	String that contains name of the source file on host PC.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.



The flash memory is rated to support about 100,000 write operations.
ACS recommends against frequent, repeated writing to the flash memory.

5.44 Save to Flash

5.44.1 ControllerSaveToFlash

Description

The function saves user application to the controller's non-volatile memory.

Syntax

ControllerSaveToFlash(handle, parameters, buffers, sPPrograms, userArrays, failure_check=False)

Arguments

handle	Communication handle.
parameters	Array of parameters constants. Each element specifies system parameters or one involved axis: ACSC_SYSTEM corresponds to system parameters; Axis.ACSC_AXIS_0 corresponds to axis 0, Axis.ACSC_AXIS_1 to axis 1, etc. If all parameters need to be specified, Axis.ACSC_PAR_ALL should be used. After the last axis, one additional element must be located that contains -1 which marks the end of the array.
buffers	Array of buffer constants. Each element specifies one involved buffer: ProgramBuffer.ACSC_BUFFER_0 corresponds to buffer 0, ProgramBuffer.ACSC_BUFFER_1 to buffer 1, etc. If all buffers need to be specified, ProgramBuffer.ACSC_BUFFER_ALL should be used. After the last buffer, one additional element must be located that contains -1 which marks the end of the array.
sPPrograms	Array of Servo Processor (SP) constants. Each element specifies one involved SP: ServoProcessor.ACSC_SP_0 corresponds to SP 0, ServoProcessor.ACSC_SP_1 to SP 1, etc. If all SPs need to be specified, ACSC_SP_ALL should be used. After the last SP, one additional element must be located that contains -1 which marks the end of the array.
userArrays	User Arrays list - Pointer to the null-terminated string. The string contains chained names of user arrays, separated by '\r'(13) character. If there is no need to save user arrays to controller's non-volatile memory, this parameter should be None.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments



The flash memory is rated to support about 100,000 write operations.
ACS recommends against frequent, repeated writing to the flash memory.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.45 SPiiPlusSC Management

5.45.1 StartSPiiPlusSC

Description

The function starts the SPiiPlusSC controller.

Syntax

StartSPiiPlusSC(failure_check=False)

Arguments

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.45.2 StopSPiiPlusSC

Description

The function stops the SPiiPlusSC controller.

Syntax

StopSPiiPlusSC(failure_check=False)

Arguments

failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If wait is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item until a call to the WaitForAsyncCall or GetResult function.

5.46 Servo Processor Data Collection

5.46.1 SPDataCollectionStart

Description

SPDataCollectionStart (Servo Processor Data Collection) performs fast data collection and accumulates data about the specified Servo Processor variable with a constant maximum sampling rate of 20kHz. A typical use for **SPDataCollectionStart** is for collecting position error (**PE**) and feedback position (**FPOS**) data at the fast Servo Processor rate.
The Servo Processor value is different from the MPU value. The Servo Processor always uses counts and not units. The Servo Processor position value is not affected by a **SET FPOS** command. An offset is added at the MPU level only. The formula (found in our manuals) is:

FPOS = FP*EFAC + EOFFS

where **FPOS** is the MPU variable and **FP** is the Servo Processor calculated value.
SPDataCollection terminates:

- When SPDataCollectionStop
- After accumulating the defined number of samples

Syntax

SPDataCollectionStart(handle, flags, array, nSample, period, sP_Index, sP_Address1, sP_Address2, sP_Address3, sP_Address4, wait, failure_check=False)

Arguments

handle	Communication handle.
flags	DataCollectionFlags.ACSC_SPDCF_REAL_TYPE - defines the array as real.
array	Array name, up to XARRSIZE variable value. Array should be declared as STATIC.
nSample	The number of samples to collect, the maximum value depends on the size of the array.
period	The time, in milliseconds, that each sample is taken.
sP_Index	The index of the Servo Processor to be sampled.
sP_Address1	The address of the Servo Processor variable in the Servo Processor to sample.
sP_Address2	Optional Add address of another Servo Processor variable in the Servo Processor to sample. In this case, the array should be defined as (2)(N)
sP_Address3	Optional Add address of a third Servo Processor variable in the Servo Processor to sample In this case, the array should be defined as (3)(N).

sP_Address4	Optional Add address of a fourth Servo Processor variable in the Servo Processor to sample In this case, the array should be defined as (4)(N).
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments



Memory addresses may vary between SPiiPlus products and revisions. ACS strongly recommends defining a variable to represent SP_Address as the return value of **GetSPAddress**. SPDataCollection can then use this variable in any SPiiPlus product or revision.

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Only one SPDC command per Servo Processor can run at a given time.

Variable	Axis	Servo Processor Variable
Position Error	0,1,2,3	axes[0].PE
Feedback Position	0,1,2,3	axes[0].fpos
Feedback Velocity	0,1,2,3	axes[0].fvel
Sin Analog Input	0,1,2,3	axes[0].sin
Cos Analog Input	0,1,2,3	axes[0].cos
Phase A Current	0,1,2,3	axes[0].is
Phase B Current	0,1,2,3	axes[0].it

Variable	Axis	Servo Processor Variable
Current Command	0,1,2,3	axes[0].command

5.46.2 SPDataCollectionStop

Description

The **SPDataCollectionStop** function Immediately terminates the data collection of SPDataCollectionStart(Servo processor data collection) for the specified servo processor.

Syntax

SPDataCollectionStop(handle, sP_Index, wait, failure_check=False)

Arguments

handle	Communication handle.
sP_Index	The index of the Servo Processor.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

5.46.3 GetSPAddress

Description

GetSPAddress reads a value from the specified SP address.

Syntax

GetSPAddress(handle, flags, sP_Index, sP_Variable, failure_check=False)

Arguments

handle	Communication handle.
flags	Bit-mapped argument that can include one or more of the following flags: EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 - first network specifier EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 - second network specifier EtherCATFlags.ACSC_ETHERCAT_NETWORK_0 and EtherCATFlags.ACSC_ETHERCAT_NETWORK_1 flags must not be specified together 0, //No flags are set (equivalent to flag ACSC_ETHERCAT_NETWORK_0)
sP_Index	The SP index in the system.
sP_Variable	String representing the name of an SP variable.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

int type

Get extended error information by calling GetLastError.

5.46.4 WaitSPDataCollectionEnd

Description

The function waits for the end of Servo Processor data collection.

Syntax

WaitSPDataCollectionEnd(handle, timeout, sP_Index, failure_check=False)

Arguments

handle	Communication handle.
timeout	Maximum wait time in milliseconds. If -1 - timeout interval never elapses.
sP_Index	The SP index in the Processor.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

Float type

Get extended error information by calling GetLastError.

Comments

The function does not return while data collection is in progress and the correct parameters have been passed.

5.47 Controller Protection Functions

5.47.1 DefineControllerProtection

Description

The function applies protection for controller, reboots controller and waits for process completion.

Syntax

DefineControllerProtection(handle, flags, noEditBuffers, noViewBuffers, standardVariables, password, timeout, failure_check=False)

Arguments

handle	Communication handle.
flags	ProtectionFlags.ACSC_PRIMARY_PROTECTION - defines primary protection. ProtectionFlags.ACSC_SECONDARY_PROTECTION - defines secondary protection. ProtectionFlags.ACSC_ALLOW_SYSTEM_RECONFIGURATION - Allows system reconfiguration. If one of the protections does not allow system reconfiguration of the system, then the other cannot allow it either. ProtectionFlags.ACSC_VARIABLES_PROTECTION – define variable protection. (For primary protection only). ProtectionFlags.ACSC_PRIMARY_PROTECTION and ProtectionFlags.ACSC_SECONDARY_PROTECTION flags must not be specified together.
noEditBuffers	[Optional] Array of buffer constants. Each element specifies one involved buffer: ProgramBuffer.ACSC_BUFFER_0 corresponds to buffer 0, ProgramBuffer.ACSC_BUFFER_1 to buffer 1, etc.

noViewBuffers	[Optional] Array of buffer constants. Each element specifies one involved buffer: ProgramBuffer.ACSC_BUFFER_0 corresponds to buffer 0, ProgramBuffer.ACSC_BUFFER_1 to buffer 1, etc.
standardVariables	[Optional] Array of variables to be protected. Only with primary protection. The string contains chained variables names separated by '\r'(13) character. Set this argument to None if not used.
password	A string (must be 4 or more characters).
timeout	Maximum waiting time in milliseconds. As part of the process, controller will be rebooted. Recommended value 150000. Depends on network configuration.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

Application and removal of controller protection involves writing to flash memory. Exceeding the rated number of flash memory writes (about 100,000) risks damage to the memory device.

5.47.2 RemoveControllerProtection

Description

The function applies protection for controller, reboots controller and waits for process completion.

Syntax

RemoveControllerProtection(handle, flags, password, timeout, failure_check=False)

Arguments

handle	Communication handle.
flags	ProtectionFlags.ACSC_PRIMARY_PROTECTION - Removes primary protection. ProtectionFlags.ACSC_SECONDARY_PROTECTION - Removes secondary protection.
password	A string (must be 4 or more characters).
timeout	Maximum waiting time in milliseconds. As part of the process, controller will be rebooted. Recommended value 150000. Depends on network configuration.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

Application and removal of controller protection involves writing to flash memory. Exceeding the rated number of flash memory writes (about 100,000) risks damage to the memory device.

5.47.3 TemporarilyDisableVariableProtection

Description

The function allows modification and saving of protected variables. Related only for primary protection.

Syntax

TemporarilyDisableVariableProtection(handle, password, wait, failure_check=False)

Arguments

handle	Communication handle.
password	A string (must be 4 or more characters).

wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Application and removal of controller protection involves writing to flash memory. Exceeding the rated number of flash memory writes (about 100,000) risks damage to the memory device.

5.47.4 RestoreVariableProtection

Description

The function restores variable protection that was temporarily disabled.

Syntax

RestoreVariableProtection(handle, wait, failure_check=False)

Arguments

handle	Communication handle.
wait	If Wait is a SYNCHRONOUS object, the function returns when the controller response is received. If Wait points to a valid ACSC_WAITBLOCK object, the function returns immediately. The calling thread must then call the GetResult function to retrieve the operation result. If Wait is IGNORE, the function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
failure_ check	If True when the function detects an error, it will raise an exception. If False the function will ignore detected errors .

Return Value

None

Get extended error information by calling GetLastError.

Comments

If *wait* is a valid ACSC_WAITBLOCK structure, the calling thread must not use or discard the wait item before calling either the WaitForAsyncCall or the GetResult function.

Application and removal of controller protection involves writing to flash memory. Exceeding the rated number of flash memory writes (about 100,000) risks damage to the memory device.

5.48 FRF Functions

5.48.1 FRF_Measure

Description

This function initializes FRF measurement.

Syntax

FRF_Measure(handle, inputParams)

Arguments

handle	Communication handle.
frf_input	Array of input parameters. FRF_INPUT type.

Return Value

On success:

The function returns an **FRF_OUTPUT** object.

On failure:

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.1", 701)
frf_input: sp.FRF_INPUT = sp.FRF_InitInput()
frf_input.axis = 0
frf_input.loopType = sp.FRF_LOOP_TYPE.PositionVelocity
frf_input.excitationType = sp.FRF_EXCITATION_TYPE.WhiteNoise
# Chirp type - Irrelevant for WhiteNoise excitation signal
frf_input.chirpType = sp.FRF_CHIRP_TYPE.LinearChirp
frf_input.filterType = sp.FRF_WINDOW_TYPE.Hamming
frf_input.overlap = sp.FRF_OVERLAP.HalfSignal
frf_input.frequencyDistributionType = sp.FRF_FREQUENCY_DISTRIBUTION_
TYPE.Logarithmic
# Frequency range parameters
frf_input.startFreqHz = 30
frf_input.endFreqHz = 3000
frf_input.freqPerDec = 50
```

```
frf_input.highResolutionStart = 500
frf_input.highResolutionFreqPerDec = 500
frf_input.excitationAmplitudePercentIp = 1
frf_input.durationSec = 1
frf_input.numberOfRepetitions = 5
frf_input.userDefinedExcitationSignal = None
# Recalculate parameters
frf_input.inRaw = None
frf_input.outRaw = None
frf_input.recalculate = True
frf_input.recalculate = False
frf_output = sp.FRF_Measure(hc, frf_input)
```

5.48.2 FRF_Stop

Description

Function aborts FRF measurement. If FRF was aborted "outputParams" will not contain valid data (all pointers will be set to nullptr) and errorCode will reflect that FRF was aborted by the user.

Syntax

FRF_Stop(handle, axis)

Arguments

handle	Communication handle
axis	axis to abort

Return Value

On success:

The function returns None object.

On failure:

The function return AcsError object and an Integer representing the error code.

Example

```
class ThreadWithReturnValue(threading.Thread):
    def __init__(self, group=None, target=None, name=None,
                 args=(), kwargs={}, Verbose=None):
        threading.Thread.__init__(self, group, target, name, args,
                                  kwargs)
        self._return = None

    def run(self):
        if self._target is not None:
            self._return = self._target(*self._args,
                                        **self._kwargs)
```

```
    def join(self, *args):
        threading.Thread.join(self, *args)
        return self._return

def stop_measure(hc, axis):
    sp.FRF_Stop(hc, axis)
    print('measuring has been stopped')

def measure(hc, frf_input):
    print('start measuring')
    frf_output = sp.FRF_Measure(hc, frf_input)
    return frf_output

hc = sp.OpenCommEthernetTCP("192.168.56.1", 701)
frf_input.axis = 0
frf_input.loopType = sp.FRF_LOOP_TYPE.PositionVelocity
frf_input.excitationType = sp.FRF_EXCITATION_TYPE.WhiteNoise
frf_input.chirpType = sp.FRF_CHIRP_TYPE.LinearChirp
frf_input.filterType = sp.FRF_WINDOW_TYPE.Hamming
frf_input.overlap = sp.FRF_OVERLAP.HalfSignal
frf_input.frequencyDistributionType =
sp.FRF_FREQUENCY_DISTRIBUTION_TYPE.Logarithmic
frf_input.startFreqHz = 30
frf_input.endFreqHz = 3000
frf_input.freqPerDec = 50
frf_input.highResolutionStart = 500
frf_input.highResolutionFreqPerDec = 500
frf_input.excitationAmplitudePercentIp = 1
frf_input.durationSec = 1
frf_input.numberOfRepetitions = 5
frf_input.userDefinedExcitationSignal = None
frf_input.inRaw = None
frf_input.outRaw = None
frf_input.recalculate = True
frf_input.recalculate = False
# create separate thread for stop and measure
stop_thread = ThreadWithReturnValue(target=stop_measure,
                                    args=(hc, 0))
measure_thread = ThreadWithReturnValue(target=measure, args=(hc,
                                    frf_input))
# start both operation concurrently
measure_thread.start()
time.sleep(0.1)
stop_thread.start()

# return value should give error code which indicating
# measurement was aborted
frf_output = measure_thread.join()
stop_thread.join()
```

5.48.3 FRF_CalculateMeasurementDuration

Description

This function calculates the required duration of measurement (DurationSec parameter of FRFInput class) in order to satisfy the specified frequency resolution.

Syntax

FRF_CalculateMeasurementDuration(params)

Arguments

params	Object contains parameters to calculate duration. **FRF_DURATION_CALCULATION_PARAMETERS** type.

Return Value

On success:

The function float type represents the calculated duration.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
input_par = sp.FRF_DURATION_CALCULATION_PARAMETERS()
input_par.startFreqHz = 10
input_par.endFreqHz = 1000
input_par.freqPerDec = 100
input_par.highResolutionStart = 500
input_par.highResolutionFreqPerDec = 500
input_par.frequencyDistributionType = sp.FRF_FREQUENCY_DISTRIBUTION_
TYPE.Logarithmic
# irrelevant if linear
input_par.frequencyHzResolutionForLinear = 0.1
duration: float = sp.FRF_CalculateMeasurementDuration(input_par)
```

5.48.4 FRF_InitInput

Description

Initializes the inputParams structure of the **FRF_Measure** function to valid initial values.

Syntax

FRF_InitInput()

Return Value

On success:

The function returns FRF_INPUT object.

On failure:

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
frf_input = sp.FRF_InitInput()
```

5.48.5 FRF_ReadServoParameters

Description

The function reads all required servo parameters for calculation of Controller, OpenLoop, ClosedLoop etc.

Syntax

FRF_ReadServoParameters(handle, axis)

Arguments

handle	Communication handle.
axis	Axis for parameter retrieval.

Return Value

On success:

The function returns SERVO_PARAMETERS object.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
axis = 0
hc = sp.OpenCommEthernetTCP("10.0.0.10", 701)
servoParameters = sp.FRF_ReadServoParameters(hc, axis)
```

5.48.6 FRF_CalculateControllerFRD

Description

The function calculates controller FRD (Frequency Response Data) based on servo parameters, servo loop and frequency vector.

Syntax

FRF_CalculateControllerFRD(servoParameters, loopType, frequencyHz)

Arguments

servoParameters	Servo parameters for axis. SERVO_PARAMETERS type.
loopType	Loop type to calculate. FRF_LOOP_TYPE type.
frequencyHz	List of input frequency values

Return Value

On success:

The function returns calculated FRD object of the controller.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
axis = 0
hc = sp.OpenCommEthernetTCP("10.0.0.10", 701)
servoParameters = sp.FRF_ReadServoParameters(hc, axis)
frequencies = [100, 200, 300]
controller = sp.FRF_CalculateControllerFRD(servoParameters, sp.FRF_LOOP_
TYPE.PositionVelocity, frequencies)
```

5.48.7 FRF_CalculateOpenLoopFRD

Description

This function calculates open loop FRD based on servo parameters, servo loop and frequency vector.

Syntax

FRF_CalculateOpenLoopFRD(servoParameters, plant, loopType)

Arguments

servoParameters	Servo parameters for axis. SERVO_PARAMETERS type.
plant	plant FRD information. FRD type.
loopType	Loop type to calculate. FRF_LOOP_TYPE type.

Return Value

On success:

The function returns calculated FRD object of the controller.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
# Example shows how open loop may be calculated based on measured plant
# and controller with
# adjusted parameters
```

```
hc = sp.OpenCommEthernetTCP("10.0.0.10", 701)
frf_input = sp.FRF_InitInput()
frf_output = sp.FRF_Measure(hc, frf_input)
axis = 0
servoParameters = sp.FRF_ReadServoParameters(hc, axis)
servoParameters.SLVKP *= 1.1
open_loop = sp.FRF_CalculateOpenLoopFRD(servoParameters, frf_
output.plant, frf_input.loopType)
```

5.48.8 FRF_CalculateClosedLoopFRD

Description

This function calculates closed loop FRD based on servo parameters, servo loop and frequency vector.

Syntax

FRF_CalculateClosedLoopFRD(servoParameters, plant, loopType):

Arguments

servoParameters	Servo parameters for axis. SERVO_PARAMETERS type.
plant	plant FRD information. FRD type.
loopType	Loop type to calculate. FRF_LOOP_TYPE type.

Return Value

On success:

The function returns calculated FRD object of the controller.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
# Example shows how open loop may be calculated based on measured plant
# and controller with
# adjusted parameters

hc = sp.OpenCommEthernetTCP("10.0.0.10", 701)
axis = 0
frf_input = sp.FRF_InitInput()
frf_output = sp.FRF_Measure(hc, frf_input)
servoParameters = sp.FRF_ReadServoParameters(hc, axis)
origSLVKP = servoParameters.SLVKP
servoParameters.SLVKP = origSLVKP * 1.1
```

```
close_loop = sp.FRF_CalculateClosedLoopFRD(servoParameters, frf_
output.plant, frf_input.loopType)
```

5.48.9 FRF_CalculateStabilityMargins

Description

The function calculates required stability margins based on the frequency response data of the open loop.

Syntax

FRF_CalculateStabilityMargins(openLoop)

Arguments

openLoop	open loop FRD parameters. FRD type.

Return Value

On success:

The function returns calculated FRF_STABILITY_MARGINS object.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.1", 701)
frf_input: sp.FRF_INPUT = sp.FRF_InitInput()
frf_output = sp.FRF_Measure(hc, frf_input)
axis = 0
servoParameters = sp.FRF_ReadServoParameters(hc, axis)
servoParameters.SLVKP *= 1.1
open_loop = sp.FRF_CalculateOpenLoopFRD(servoParameters, frf_
output.plant, frf_input.loopType)
stability_margin = sp.FRF_CalculateStabilityMargins(open_loop)
```

5.48.10 FFT

Description

Function calculates fast Fourier transform.

Syntax

FFT(in_par)

Arguments

inputParams	Array of input for FFT.

Return Value

On success:

Two numpy ndarray objects which the first is the real part and the second is the imaginary parts of the calculation.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
input_list = [-1, 2, 3, 0]
real, imag = sp.FFT(input_list)
```

5.48.11 JitterAnalysis

Description

Function executes jitter analysis.

Syntax

JitterAnalysis(inputParams):

Arguments

inputParams	Data for analysis. JITTER_ANALYSIS_INPUT type.

Return Value

On success:

The function returns calculated JITTER_ANALYSIS_OUTPUT object.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.1", 701)
jitterInput = sp.JITTER_ANALYSIS_INPUT()
axis = 0
dataCollectionArray = "dataCollectionPE(100000)"
try:
    sp.DeclareVariable(hc, sp.AcsplVariableType.ACSC_REAL_TYPE,
       dataCollectionArray, sp.SYNCHRONOUS, True)
except:
    pass
sampleDuration = 5

CTIME = sp.ReadReal(hc, -1, "CTIME", -1, -1, -1, -1, sp.SYNCHRONOUS,
True)
EFAC = sp.ReadReal(hc, -1, "EFAC", -1, -1, -1, -1, sp.SYNCHRONOUS, True)
numberOfSamples = sampleDuration * 20000
# issue servo processor data collection
transaction = "SPDC/r dataCollectionPE, 100000, 1/20000, 0,
       getspa(0,\"axes[0].PE\")"
```

```
response = sp.Transaction(hc, transaction, len(transaction), 100,
       sp.SYNCHRONOUS, True)
time.sleep(1)
jitter = sp.ReadReal(hc, -1, "dataCollectionPE", 0, numberOfSamples    - 1,
-1, -1, sp.SYNCHRONOUS, True)
jitterInput.desiredFrequencyResolutionHz = 1
jitterInput.frequencyBandsHz = [0, 100, 150, 300, 500, 1000]
jitterInput.jitter = [0]*numberOfSamples
for k in range(numberOfSamples):
    jitterInput.jitter[k] = jitter[k] * EFAC

jitterInput.jitterFrequencyBandsCumulativeAmplitudeRMSthreshold = [1e-6,
2e-6, 1e-6, 2e-6, 1e-6]
jitterInput.samplingFrequencyHz = 20000
jitterInput.windowType = sp.FRF_WINDOW_TYPE.Hanning
jitterOutput = sp.JitterAnalysis(jitterInput)
```

5.48.12 FRF_CrossCouplingMeasure

Syntax

FRF_CrossCouplingMeasure(handle, inputParams)

Arguments

handle	Communication handle.
inputParams	FRF_CROSS_COUPLING_INPUT type.

Return Value

On success:

The function returns calculated FRF_CROSS_COUPLING_OUTPUT object.

On failure:

The function returns an AcsError object and an Integer representing an error code.

Example

```
hc = sp.OpenCommEthernetTCP("10.0.0.1", 701)
sp.Enable(hc, sp.Axis.ACSC_AXIS_0, sp.SYNCHRONOUS, True)
sp.WaitMotorEnabled(hc, sp.Axis.ACSC_AXIS_0, 1, 5000, True)
sp.Enable(hc, sp.Axis.ACSC_AXIS_2, sp.SYNCHRONOUS, True)
sp.WaitMotorEnabled(hc, sp.Axis.ACSC_AXIS_2, 1, 5000, True)
sp.CommutExt(hc, sp.Axis.ACSC_AXIS_0, sp.GeneralDefinition.ACSC_NONE,
sp.GeneralDefinition.ACSC_NONE,
             sp.GeneralDefinition.ACSC_NONE, sp.SYNCHRONOUS, True)
sp.CommutExt(hc, sp.Axis.ACSC_AXIS_2, sp.GeneralDefinition.ACSC_NONE,
sp.GeneralDefinition.ACSC_NONE,
             sp.GeneralDefinition.ACSC_NONE, sp.SYNCHRONOUS, True)
sp.WaitMotorCommutated(hc, 0, 1, 5000, True)
```

```
sp.WaitMotorCommutated(hc, 2, 1, 5000, True)
input = sp.FRF_CROSS_COUPLING_INPUT()
axes = [0, 2]
input.axes = axes
input.axesLength = 2
input.startFreqHz = 10
input.endFreqHz = 1000
input.freqPerDec = 30
input.frequencyDistributionType = sp.FRF_FREQUENCY_DISTRIBUTION_
TYPE.Linear
input.excitationAmplitudePercentIp = [0, 0]
input.excitationAmplitudePercentIp[0] = 1
input.excitationAmplitudePercentIp[1] = 1
input.numberOfRepetitions = 3
input.durationSec = 0.02
input.highResolutionStart = 10000
input.highResolutionFreqPerDec = 500
input.crossCouplingType = sp.FRF_CROSS_COUPLING_TYPE.CompleteOpen
cross_coupling = sp.FRF_CrossCouplingMeasure(hc, input)
```

5.48.13 FRF_CalculateSLVRAT

Description

The function calculates the **SLVRAT** parameter and returns it as float value.

Syntax

FRF_CalculateSLVRAT(inputParams)

Arguments

inputParams	FRF_CALCULATE_SLVRAT_DATA type.

Return Value

On success:

The function returns float value.

On failure:

The function returns an AcsError object and an Integer representing an error code.



5 HaTnufa St.
Yokne'am Illit 2066717
Israel
Tel: (+972) (4) 654 6440 Fax: (+972) (4) 654 6443